# Differential Privacy with OpenDP

**The OpenDP library from a first release to synthetic data**

Manuel V. Loureiro

Version 2026-11-04

# Table of contents

**Preface** **1**
How this book is organized . . . . . 1

**I. Getting started** **3**

**1. A first release** **5**
1.1. Install the library . . . . . 5
1.2. One noisy number . . . . . 6
1.3. The five steps of an analysis . . . . . 6
1.4. A count and a mean . . . . . 8
1.5. What the context refuses . . . . . 9
1.6. Summary . . . . . 9
1.7. References . . . . . 10

**2. The data and the context** **11**
2.1. The Labour Force Survey sample . . . . . 11
2.2. Loading the data lazily . . . . . 12
2.3. A context on the frame . . . . . 13
2.4. A query is not a frame . . . . . 15
2.5. A margin, in preview . . . . . 16
2.6. Summary . . . . . 17
2.7. References . . . . . 17

**II. Tabular data** **19**

**3. Essential statistics** **21**
3.1. The context for this chapter . . . . . 21
3.2. Counting responses . . . . . 22
3.3. Where bounds come from . . . . . 23
3.4. The sum of hours . . . . . 24
3.5. The mean . . . . . 25
3.6. The median and the quartiles . . . . . 26
3.7. A check against the truth . . . . . 28
3.8. Summary . . . . . 28
3.9. References . . . . . 29

**4. Grouping** **30**
4.1. Why grouping keys are dangerous . . . . . 30
4.2. Stable keys . . . . . 31

4.3. Explicit keys . . . 32
4.4. Invariant keys . . . 33
4.5. Margins as public descriptors . . . 35
4.6. The noise per group and the ungrouped count . . . 35
4.7. Checking the release against the truth . . . 36
4.8. Summary . . . 36
4.9. References . . . 37

**5. Identifier truncation and bounds** **38**
5.1. A privacy unit with an identifier . . . 38
5.2. Truncating each person's rows . . . 40
5.3. Truncating the groups a person touches . . . 41
5.4. One row per person . . . 43
5.5. Bounds in the privacy unit . . . 44
5.6. The check against the truth . . . 46
5.7. Summary . . . 46
5.8. References . . . 47

**6. Preparing microdata** **48**
6.1. Prepare inside the query . . . 48
6.2. Columns, filters and groups . . . 49
6.3. The types OpenDP tracks . . . 50
6.4. Casting, filling and clipping . . . 51
6.5. Predicates and arithmetic . . . 53
6.6. Strings and dates . . . 54
6.7. A worked preparation: hours lost to absence . . . 55
6.8. What a refused expression looks like . . . 57
6.9. Summary . . . 57
6.10. References . . . 58

**7. Assessing utility and choosing the noise** **59**
7.1. What an accuracy statement means . . . 59
7.2. A target error bar on the survey . . . 60
7.3. Laplace under pure DP, Gaussian under zCDP . . . 61
7.4. One person in many groups: L1 against L2 . . . 62
7.5. Stating the guarantee in epsilon . . . 63
7.6. Signed and unsigned counts . . . 64
7.7. Which noise to choose . . . 65
7.8. The releases of one run . . . 66
7.9. Summary . . . 66
7.10. References . . . 67

**III. The framework** **68**

**8. A framework to understand differential privacy** **70**
8.1. Two kinds of object . . . 70
8.2. Functions . . . 71

8.3. Domains . . . 71
8.4. Metrics . . . 72
8.5. Measures . . . 73
8.6. Stability maps and privacy maps . . . 74
8.7. Chaining a transformation and a measurement . . . 75
8.8. The typical workflow in both APIs . . . 76
8.9. Summary . . . 78
8.10. References . . . 79

**9. Transformations** **81**
9.1. Constructors and chains . . . 81
9.2. Preprocessing . . . 82
9.3. The sum . . . 84
9.4. The mean and the variance . . . 87
9.5. Quantile scores . . . 88
9.6. Reading a chain . . . 89
9.7. Summary . . . 90
9.8. References . . . 91

**10. Measurements** **92**
10.1. Additive noise . . . 92
10.2. Canonical noise . . . 95
10.3. Thresholded noise . . . 95
10.4. The approximate Laplace projection . . . 97
10.5. Noisy max and selection . . . 98
10.6. Randomized response . . . 100
10.7. Summary . . . 102
10.8. References . . . 103

**11. Combinators** **105**
11.1. Chaining . . . 105
11.2. Non-adaptive composition . . . 107
11.3. Adaptive composition . . . 108
11.4. Odometers . . . 109
11.5. Privacy filters . . . 110
11.6. Chaining and nesting compositors . . . 111
11.7. Measure casting . . . 112
11.8. Amplification by subsampling . . . 113
11.9. Summary . . . 114
11.10. References . . . 115

**12. Utilities and the Context API inside** **117**
12.1. Types . . . 117
12.2. Parameter search . . . 119
12.3. Serialization . . . 120
12.4. The compositor behind a context . . . 121
12.5. The query behind a release . . . 122
12.6. Nested compositors and other measures . . . 123

12.7. Summary . . . 125
12.8. References . . . 126

**IV. Extending the library** **127**

**13. Plugins** **129**
13.1. Trust and the honest-but-curious flag . . . 129
13.2. A measurement plugin . . . 130
13.3. A transformation plugin . . . 132
13.4. Your own domains, metrics and measures . . . 133
13.5. Registering a constructor for the Context API . . . 134
13.6. A Theil-Sen regression . . . 135
13.7. Choosing grouping columns privately . . . 138
13.8. Summary . . . 141
13.9. References . . . 141

**14. Statistical modelling** **142**
14.1. A model or a table? . . . 142
14.2. Linear regression . . . 143
14.3. Principal component analysis . . . 145
14.4. The NumPy domain underneath . . . 146
14.5. What a reviewer should ask . . . 147
14.6. The releases of one run . . . 148
14.7. Summary . . . 149
14.8. References . . . 150

**15. Limitations and troubleshooting** **151**
15.1. What OpenDP does not promise . . . 151
15.2. The feature flags . . . 152
15.3. What lies outside the guarantee . . . 154
15.4. The error messages you will meet . . . 155
15.5. Reading an OpenDPException . . . 157
15.6. What a reviewer verifies . . . 158
15.7. Summary . . . 159
15.8. References . . . 160

**V. Synthetic data** **161**

**16. Contingency tables** **163**
16.1. From grouped counts to marginals . . . 163
16.2. The context under zCDP, with a delta . . . 164
16.3. Keys and cuts . . . 165
16.4. The Fixed algorithm and its workload . . . 166
16.5. The released table . . . 167
16.6. Adaptive estimation . . . 167
16.7. Checking the release against the truth . . . 168
16.8. Summary . . . 170

16.9. References . . . . . 171

**17. Synthetic data with AIM and MST** **172**
17.1. When you do not know the questions . . . . . 172
17.2. The AIM release . . . . . 173
17.3. A spanning tree of pairs with MST . . . . . 175
17.4. A first look at the one-way marginals . . . . . 175
17.5. Nulls and the key set . . . . . 176
17.6. Guaranteeing marginals with Sequential . . . . . 177
17.7. Generating the rows . . . . . 178
17.8. The privacy statement . . . . . 179
17.9. The measured marginals against the real ones . . . . . 180
17.10. Summary . . . . . 181
17.11. References . . . . . 182

**18. Evaluating and tuning synthetic data** **184**
18.1. What good means . . . . . 184
18.2. The workload error . . . . . 186
18.3. A query nobody measured . . . . . 187
18.4. Train on synthetic, test on real . . . . . 188
18.5. What the budget and the algorithm buy . . . . . 189
18.6. How many rows, and how to draw them . . . . . 190
18.7. What to publish . . . . . 191
18.8. Summary . . . . . 192
18.9. References . . . . . 193

**Appendices** **195**

**A. Glossary** **195**

**B. Index** **196**

# Preface

This book teaches the OpenDP library, from installing it and releasing one noisy number to generating a synthetic copy of a dataset. OpenDP is a library for differential privacy (DP), the guarantee that a release reveals almost nothing about any one person in the data. The library's documentation is a reference, organized by what the library contains. This book is a course, organized by what you learn next, and it follows the documentation closely enough that every chapter names the pages it covers, so you can read the two side by side.

Every piece of code here has been run. A chapter is built around one script in the repository's `code/` directory, which a test runs and checks against every number the chapter quotes. A short block on a page is either a snippet that runs on its own or an excerpt of that script, so the code you read is the code that produced the output next to it. OpenDP draws its noise from a cryptographic generator that cannot be seeded, so a released value is different on every run. A chapter therefore quotes the numbers that are the same on every run, such as noise scales, error bars and thresholds, and shows released values in a block marked as one run.

One dataset runs through the book: the sample of the French Labour Force Survey (LFS) that the OpenDP documentation uses in its own tutorials, about 3.8 million quarterly survey responses from 1.7 million people. Each chapter releases something new about it, so by the end you will have counted, summed, grouped, bounded and modelled it, and finally replaced it with a synthetic table.

You need Python and enough NumPy or Polars to read a short query. You do not need to know differential privacy already. Each chapter says in a sentence what the results it uses mean, and links to the full edition of *Privacy-Preserving Machine Learning* for their derivations, which this book never repeats.

## How this book is organized

1. **Getting started** installs the library, releases a first number, and sets up the context that mediates every later query to the data.
2. **Tabular data** releases counts, sums, means, quantiles and grouped tables with Polars, bounds what one person can contribute, prepares the data inside the privacy guarantee, and reads error bars.
3. **The framework** opens the box: domains, metrics and measures, transformations and measurements, how they chain and compose, and the utilities that search for parameters.
4. **Extending the library** writes new transformations and measurements as plugins, fits regression and principal component models, and knows what the library does not promise.
5. **Synthetic data** releases contingency tables and generates synthetic microdata with the AIM and MST algorithms, then measures how good the copy is.

The code runs on OpenDP 0.16.0 with Polars 1.36.1, pinned in the repository so that every number stays reproducible. To rerun a chapter, run `uv run python code/<part>/opendp_<topic>.py` from the repository's root.

# Part I.

# Getting started

This part takes you from an empty Python environment to a context that holds a real dataset and a privacy budget. It assumes nothing about differential privacy, and its one idea is this. A differentially private release is a randomized computation whose noise is calibrated to how much one person can change the answer. OpenDP does that calibration for you, and it keeps count of what you spend.

The first chapter installs the library and releases one noisy number, first with a mechanism built by hand and then with the Context API. With the Context API, you state the privacy unit and the budget, and the library finds the noise. On the way, the chapter gives the five steps of every analysis in OpenDP.

The second chapter loads the dataset of the whole book, the French Labour Force Survey (LFS) sample of the OpenDP tutorials. Then it builds a context on it. A person who answers the survey in many quarters owns many rows, and the chapter explains why the privacy unit must cover all of them. It also shows how a context splits one budget over several queries, and what a margin is.

Table 1.: The chapters of this part and the questions they answer.

| Chapter | Question it answers |
|---|---|
| A first release | How do I install OpenDP, and what happens when I release one noisy number? |
| The data and the context | How do I put a real dataset behind a context, and what do its unit, budget and margins mean? |

At the end of this part, you can explain the privacy unit and the budget of a release in one sentence each. You can also build a context for a dataset in which one person owns many rows.

# 1. A first release

Suppose you hold the scores of a hundred people. You want to publish their number and average without letting anyone work out an individual's score. This chapter makes those releases with OpenDP and introduces the idea behind the library. A **differentially private** release is a randomized computation whose noise is calibrated to how much one person can change the answer. OpenDP does that calibration, refuses computations it cannot calibrate, and tracks the privacy budget you spend.

Following the documentation's quickstart, the chapter first builds a Laplace mechanism by hand and reads its guarantee. It then uses an OpenDP context to release a count and a mean.

The script opendp_first_release.py runs every step on OpenDP 0.16.0, with a hundred integers from a seeded generator in place of the scores. The noise scales, the error bars and the privacy maps that it prints are the same on every run. The released values are not, because OpenDP's noise comes from a cryptographic generator without a seed. So this page shows them in one block, marked as one run.

```
uv run python code/part01-getting-started/opendp_first_release.py
```

## 1.1. Install the library

OpenDP is a Python package with a Rust core. It reads tables through Polars, so install the Polars extra:

```
pip install 'opendp[polars]'
```

The repository pins `opendp[polars]==0.16.0` and Polars 1.36.1; all numbers in this book use those versions. Next, import the prelude, which gathers the library's modules under one name, and enable the `contrib` feature.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.__version__)  # 0.16.0
```

The feature flag marks components whose proofs have not completed review. Every component needs a written proof that it does what it claims, and that proof must be reviewed. Until then, OpenDP marks the component `contrib` and runs it only if you opt in. In version 0.16.0, almost everything in this book is still `contrib`, so every script enables the feature.

## 1.2. One noisy number

The simplest private computation adds noise to one number. OpenDP builds it from a **space**, a pair of a domain and a metric. A constructor then turns the space into a **measurement**, which is a randomized function with a privacy guarantee.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(laplace.map(1.0))  # 1.0
print(laplace.map(2.0))  # 2.0
print(laplace.check(1.0, 0.5))  # False
```

The domain says what the input can be, here any finite float. The metric says how to compare two inputs, here by their absolute difference. The constructor `then_laplace` adds a draw from the Laplace distribution with scale 1 to its input, and the `>>` operator attaches it to the space.

The guarantee is in the **privacy map**. Its argument is a bound on how far apart two inputs can be, which differential privacy calls the **sensitivity** of the released quantity. Its value is the largest privacy loss $\varepsilon$ that the mechanism can have on two such inputs. For Laplace noise, the map is the sensitivity divided by the scale, so at scale 1 the cost equals the sensitivity. The `check` method asks whether the map at a given sensitivity stays within a given $\varepsilon$. If you double the scale, the cost halves, as the script shows.

```
laplace scale 1: map(1.0)=1.0 map(2.0)=2.0
laplace scale 1: check(1.0, 1.0)=True check(1.0, 0.5)=False
laplace scale 2: map(1.0)=0.5
```

The value $\varepsilon$ promises that a change to one person's data changes the probability of any output by a factor of at most $e^{\varepsilon}$. The full edition's chapter on the Laplace and Gaussian mechanisms derives this guarantee. Note where it comes from: not from the noise alone, but from the noise together with a bound on what one person can change. You gave that bound as the argument of `map`, and the rest of the chapter shows how the library finds it for you.

## 1.3. The five steps of an analysis

The documentation's typical workflow has five steps. Before accessing the data, decide on the privacy unit, the budget and the public information you will use. Then give the data to the library through a context, and submit queries through that context.

The first step is to identify the **privacy unit**, which is what the guarantee protects. The documentation calls it the unit of privacy. Here each person contributes one row, so the unit is one row. Two datasets are **neighbours** when one is the other with one more or one less row. OpenDP writes this as a distance of 1 under the symmetric distance.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1))  # (SymmetricDistance(), 1)
print(dp.loss_of(epsilon=1.0))  # (MaxDivergence, 1.0)
print(dp.loss_of(epsilon=1.0, delta=1e-7))  # (Approximate(MaxDivergence), (1.0,
 ↪  1e-07))
print(dp.loss_of(rho=0.1))  # (ZeroConcentratedDivergence, 0.1)
```

The second step is to set the **privacy budget**, which the documentation calls the privacy loss. It is a measure, which says how to compare the distributions of two releases, together with a bound on that comparison. Pure differential privacy compares them by the max divergence and bounds it by $\varepsilon$, so `loss_of(epsilon=1.0)` gives the printed pair. The other two lines show budgets of later chapters. A slack $\delta$ gives approximate differential privacy. A budget $\rho$ gives zero-concentrated differential privacy, which Assessing utility and choosing the noise introduces. A budget of $\varepsilon = 1$ is a common start, and the OpenDP deployments registry lists the values of real releases.

The third step is to collect **public information**: facts about the data that you do not protect, because they are known anyway. Here they are the bounds on a score, 0 to 100, and the midpoint 50 as the value for a missing score. Take bounds from the domain or from another public source, never from the private data, because a bound read off the data leaks it.

The fourth step is to **mediate access**. The script builds a **context**, which holds the data, the privacy unit, the budget and the rule that splits the budget. Every later computation on the data goes through the context.

```
def make_context(data: list[float]) -> dp.Context:
    """The context: the data behind a privacy unit of one row, a budget of epsilon = 1,
     ↪  split evenly over 3 queries."""
    return dp.Context.compositor(
        data=data,
        privacy_unit=dp.unit_of(contributions=1),  # neighbours add or remove one
 ↪  person's row
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 3
    )
```

The context is a **compositor**, because the costs of several releases add up. Three queries at a third of $\varepsilon$ each cost $\varepsilon$ in total, by the basic composition rule that the full edition's chapter on properties proves. The script prints the split.

```
privacy unit: (SymmetricDistance(), 1)
privacy loss: (MaxDivergence, 1.0)
bounds: (0.0, 100.0)  imputed value: 50
budget per query: [0.3333, 0.3333, 0.3333]
```

The fifth step is to submit queries, which the next section does. Before that, adopt one habit from the documentation: when the context exists, delete your own reference to the data. Anything that you compute outside the context is outside the guarantee.

## 1.4. A count and a mean

You build a query by chaining methods on `context.query()`. Each method is a constructor of the framework without its space, because the context knows the space. The count query counts the rows and adds Laplace noise.

```
count_query = context.query().count().laplace()
count_scale = count_query.param()  # the scale the budget allows: sensitivity 1 over
 ↪  epsilon / 3
count_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=count_scale, alpha=0.05)
dp_count = count_query.release()
```

Nothing in `laplace()` gives a scale. The context finds the smallest scale whose privacy map, at the unit's distance, fits within the query's share of the budget, and `param` reports it. For the count, the unit's distance of one row is a sensitivity of 1, and the share is $\varepsilon/3$, so the scale is 3. The count is an integer, so the noise is discrete Laplace, and the discrete accuracy function turns the scale into an error bar. With probability 0.95, the released count is within about 9.4 of the true count.

```
count: scale=3.0000 95% accuracy=9.4457
```

Read those two numbers before you release anything. They depend on the budget and the sensitivity, not on the data, so they cost nothing.

The mean needs more preparation, because the mean of a list is not a well-behaved function of the list until you fix three things.

```
mean_query = (
    context.query()
    .impute_constant(IMPUTED)  # a missing value becomes the public midpoint
    .clamp(BOUNDS)  # every value is clipped into the public bounds
    .resize(size=dp_count, constant=IMPUTED)  # the mean needs a known size: the
 ↪  released count
    .mean()
    .laplace()
)
mean_scale = mean_query.param()
dp_mean = mean_query.release()
```

Imputation removes missing values, because the mean of a list with a hole in it has no value. Clamping bounds every value, because without bounds one person could move the mean by any amount, and no finite noise would hide them. Resizing fixes the number of rows, because the mean divides by it, and the sensitivity of a mean has a bound only when that divisor is known. The size is the count released just before. A released value is public, so using it again costs nothing, by the post-processing property that the full edition proves. If the noisy count is too small, the resize pads the list with the imputed value, and if it is too large, the resize drops rows. So the released mean is the mean of a slightly changed list, and the error bar below does not cover that change.

So the scale of the mean changes from run to run. The data are a public toy, so the script also computes the scale and its error bar at the true size of 100.

```
mean at size 100: scale=3.0000 95% accuracy=8.9872
```

An error bar of about 9 points on a range of 100 is wide. The cause is the small budget: a third of $\varepsilon = 1$, on rows whose mean one person can move by a fixed fraction of the range. Essential statistics shows how a larger dataset, with a sum released together with a count, does better.

## 1.5. What the context refuses

After the count and the mean, two of the three shares are spent, and the script reads them back from the context.

```
budget spent after two queries: [0.3333, 0.3333]
```

A third query takes the last share. A fourth has no share to take, so the context raises an error and releases nothing.

```
try:
    context.query().count().laplace().release()
    refusal = "released"
except ValueError as err:
    refusal = str(err)
```

```
fourth query: Privacy allowance has been exhausted
```

This refusal is the reason to use a context. With the framework alone, nothing stops you from calling a measurement twice and spending twice. With a context, the budget is a hard limit, and your guarantee is what you declared at the start.

Here are the releases of one run.

```
released (one run):
  count 105  (true 100, within 9.4 with probability 0.95)
  mean scale 2.8571  mean 53.32  (true 50.93)
  third query, a second count 108
```

The count is within its error bar, as it must be about 19 times in 20. The mean's scale is below 3 because the released count is above 100, and the mean is also within its bar. You can do this check only because the data are a toy. On real data, make these decisions on public or historical data during development, and release the real data once.

## 1.6. Summary

Table 1.1.: The calls of a first release, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.enable_features("contrib")` | Switches on the components whose proofs are not yet vetted | Put it at the top of every script; leave it out to see what is vetted |
| `space >> dp.m.then_laplace(scale)` | A Laplace measurement on a domain and a metric | The scale is the sensitivity over $\varepsilon$ |
| `measurement.map(d_in)` | The largest $\varepsilon$ for inputs at most `d_in` apart | Read it before releasing; it costs nothing |
| `measurement.check(d_in, d_out)` | Whether the map at `d_in` stays within `d_out` | What the context runs for you when it searches for a scale |
| `dp.unit_of(contributions=1)` | The privacy unit: neighbours add or remove one row | Choose it before anything else; one row per person is the default case |
| `dp.loss_of(epsilon=1.0)` | The privacy budget under pure differential privacy | $\varepsilon = 1$ is a common start; the registry lists real values |
| `dp.Context.compositor(...)` | Holds the data, the unit, the budget and its split | One context per release; delete your own reference to the data |
| `context.query().count().laplace()` | A query whose scale the context fills in from the budget | Read `param()` and the accuracy before `release()` |
| `impute_constant`, `clamp`, `resize` | Make the mean's sensitivity finite | Bounds and imputed values come from public information |
| A fourth query on a budget split in three | `ValueError: Privacy allowance has been exhausted` | The refusal is the point of the context |

## 1.7. References

- OpenDP documentation: "Quickstart", "Typical Workflow", and "Context" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism and the definition it satisfies.
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the framework of domains, metrics, measures and maps behind every call.
- The full edition's chapters on the definition in the full edition, post-processing and composition in the full edition and the Laplace and Gaussian mechanisms in the full edition derive the results this chapter states.

# 2. The data and the context

The first chapter released a count and a mean from a hundred numbers that fit in one line of Python. Real data are different: a table of millions of rows, read from a file, in which one person appears many times. This chapter puts such a table behind a context. The one idea it adds is that the privacy unit must cover everything that one person can contribute. So a person who answers a survey every quarter needs a unit of many rows, and every noise scale grows with it.

The chapter follows the documentation's introduction to tabular data and its page on how OpenDP's Polars differs from plain Polars. You meet the dataset of the whole book and load it lazily. Then you build a context with a unit of 36 rows and a budget split five ways. You see how a query on a context differs from a query on a frame, and you add the margin that sums need. The first release on this data is the subject of Essential statistics.

The script opendp_data_context.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. Every number on this page is the same on every run: the sizes of the sample, the budget splits, the noise scales and the margin. The page quotes no released value, because OpenDP's noise cannot be seeded and the releases start in the next chapter.

```
uv run python code/part01-getting-started/opendp_data_context.py
```

## 2.1. The Labour Force Survey sample

Every quarter, Eurostat and the national statistical offices survey the working hours of people across the European Union in the Labour Force Survey (LFS). Eurostat publishes a public sample of the microdata. For its tutorials, the OpenDP project took a few columns of the French sample over the nine years from 2005 to 2013. Statistical disclosure control already protects these public microdata, with the classical tools of recoded categories, suppressed rare values and a little noise. So no release from this sample can harm anyone, and you can compare every release with the truth. That is why the book uses it.

Table 2.1.: The columns of the LFS sample, with the coding of the survey's user guide.

| Column | Meaning | Coding |
|---|---|---|
| `PIDENT` | Person identifier | An integer, generated by the OpenDP project |
| `YEAR` | Reference year | 2005 to 2013 |
| `QUARTER` | Reference quarter | 1 to 4 |
| `SEX` | Sex | 1 male, 2 female |
| `AGE` | Age during the reference week | Single years, as a float with a few missing values |

| Column | Meaning | Coding |
|---|---|---|
| `HWUSUAL` | Hours per week usually worked in the main job | 1 to 98 hours; 0 when the hours vary too much to give; 99 when not applicable; missing when there was no answer |
| `HWACTUAL` | Hours actually worked in the reference week | Coded like `HWUSUAL` |
| `ILOSTAT` | Labour status during the reference week | 1 worked for pay; 2 had a job but was absent; 3 laid off; 4 conscript; 5 neither worked nor had a job; 9 under 15 years old |

The survey publishes no person identifier, so the OpenDP project generated `PIDENT`. This synthetic column links rows as if they were one person's answers over time. It shows how the library handles a person who owns many rows, and it points to no real person. The script reads the sizes of the sample before any context exists, which is permitted only because the sample is public.

```
rows: 3,811,932  people: 1,682,306
years: 2005 to 2013  quarters: 36
responses per person: max 15  distinct quarters per person: max 4
```

About 3.8 million responses come from about 1.7 million identifiers, so a typical identifier appears two or three times. The busiest appears 15 times, in at most four distinct quarters, so the synthetic identifier sometimes links several rows of one quarter. The rest of the book does not use these numbers. The privacy unit comes from the survey's public design, and the numbers only confirm that the design's bound holds with room to spare.

## 2.2. Loading the data lazily

Polars can read a file in two ways. An eager read loads the whole table into memory at once. A lazy scan records only where the file is and what its columns are, and it reads the rows when a query runs. OpenDP targets the lazy API, so the script opens the sample with `scan_parquet` and gets a `LazyFrame`. To print its schema, Polars reads only the file's header.

```
frame = pl.scan_parquet(practitioner_datasets.france_lfs())  # a LazyFrame: the file is
 ↪  read only when a query runs
print("schema: " + ", ".join(f"{name} {dtype}" for name, dtype in
 ↪  frame.collect_schema().items()))
```

```
schema: PIDENT Int64, YEAR Int64, QUARTER Int64, SEX Int64, AGE Float64, HWUSUAL
 ↪  Float64, HWACTUAL Float64, ILOSTAT Int64
```

The documentation reads the same sample as a CSV file, through `dp.examples.get_france_lfs_path()` and `scan_csv`. The repository caches it as Parquet, for a reason that a warning in the documentation gives. A CSV file has no types, so Polars infers them from the values, and the type of a column then depends on the data. A column of whole numbers becomes an integer column, but with one decimal value in it, it becomes a float column.

```
import io

import polars as pl

integers = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"))
decimals = pl.scan_csv(io.BytesIO(b"hours\n35\n40.5\n"))
strings = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"), infer_schema=False)
print(integers.collect_schema())  # Schema({'hours': Int64})
print(decimals.collect_schema())  # Schema({'hours': Float64})
print(strings.collect_schema())  # Schema({'hours': String})
```

This matters for privacy because the guarantee covers only the queries that go through the context. Loading the data is outside it, and the documentation leaves that step to a trusted curator. Suppose that one person's unusual value changes an inferred type or makes the parse fail, and that someone outside the trusted environment sees the difference. Then the data have leaked through a channel that the guarantee never counted. So use one of two safe habits. Read a format that carries its own schema, such as Parquet. Or load every CSV column as a string with `infer_schema=False`, as the last line of the snippet does, and cast it yourself.

Give the context the `LazyFrame` itself, not a collected table. Then OpenDP can read the plan of every query before it touches a row, and the 3.8 million rows stay out of memory until a release needs them.

## 2.3. A context on the frame

The context of the first chapter took a list, and this one takes the frame. Its arguments are the same, but each has a new value, and each needs a sentence.

```
def make_context(frame: pl.LazyFrame, margins: Sequence[dp.polars.Margin] | None =
↪  None) -> dp.Context:
    """A context on the LazyFrame: a privacy unit of 36 rows, a budget of epsilon = 1
    ↪  split evenly over 5 queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
        margins=margins,  # public facts about the frame; the count needs none
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (MaxDivergence, 1.0)
budget split evenly over 5: [0.2, 0.2, 0.2, 0.2, 0.2]
budget split by weights [2, 1, 1, 1]: [0.4, 0.2, 0.2, 0.2]
```

The privacy unit is `dp.unit_of(contributions=36)`, and the 36 comes from the survey's design. The unit is what the guarantee protects, so it must cover everything that one person can contribute. The survey runs every quarter for nine years, and a person answers at most once a quarter, so a person owns at most 36 rows. So two datasets are **neighbours** when one is the other with up to 36 rows of one person more or less. OpenDP writes this as a symmetric distance of 36. Identifier truncation and bounds replaces this fixed bound by `dp.unit_of(contributions=1, identifier="PIDENT")`, so that the library enforces the bound and does not trust it.

The unit has a price. One person more or less changes a count by up to 36. So every count here is 36 times as sensitive as with a unit of one row, and every noise scale grows by the same factor. This price is necessary. A unit of one row would protect each quarter's answer separately, and for a person with 15 rows the guarantee would weaken to 15 times the budget. That is the group-privacy property, which the full edition's chapter on properties proves.

The budget is `dp.loss_of(epsilon=1.0)`, pure differential privacy with a privacy parameter $\varepsilon$ of 1. It promises that a change to all 36 rows of one person changes the probability of an output by a factor of at most $e^{\varepsilon}$. The full edition's chapter on the definition sets this out. The documentation calls this argument the privacy loss. This book calls it the budget, because the context spends it.

The context must split the budget, because the losses of several releases add up. With `split_evenly_over=5`, each query gets $\varepsilon/5$, and the five shares of 0.2 sum to the whole budget, by the basic composition rule of the same chapter on properties. Without a limit on the number of queries, the total loss would have no bound. The documentation advises an even split when the queries are alike, and weights when some matter more. The script builds a second context only to print a weighted split.

```
weighted = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_by_weights=WEIGHTS,  # the first query gets twice the share of each of the
 ↪  other three
)
```

The first query gets 0.4 and the other three get 0.2 each, so the weights are shares of the budget, not values of $\varepsilon$. The documentation recommends one context for all your queries.

```
context = make_context(frame)
del frame  # from here on, every computation on the data goes through a context
```

The last line is a habit that the documentation insists on: when the context exists, delete your own reference to the frame. Anything that you compute from the frame itself is outside the guarantee. The sizes printed above are such a computation, permitted here only because the sample is public.

## 2.4. A query is not a frame

With a `LazyFrame` behind it, `context.query()` returns a `LazyFrameQuery`. The documentation's page on Polars versus OpenDP gives four ways in which it differs from the frame, and a tiny frame shows the first two.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35, 40, 0, 99]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=5,
)
query = context.query().select(dp.len(signed=True))
print(type(query).__name__)  # LazyFrameQuery
scale = query.summarize()["scale"].item()
print(f"scale {scale:.1f}")  # scale 5.0
```

The first difference is how you specify the data. You never touch the frame again, and you write every query against `context.query()`, which the documentation calls a mock for the real data. The script shows how good a mock it is.

```
query type: LazyFrameQuery
query schema: Schema({'PIDENT': Int64, 'YEAR': Int64, 'QUARTER': Int64, 'SEX': Int64,
 ↪  'AGE': Float64, 'HWUSUAL': Float64, 'HWACTUAL': Float64, 'ILOSTAT': Int64})
collect before release: You must call `.release()` before executing a query.
plain Polars mean: MakeMeasurement("Expr is not recognized at this time:
 ↪  col("AGE").mean(). If you would like to see this supported, please file an issue.")
```

The mock knows the schema of the frame, so the library checks a query against the real columns and types. The documentation describes the mock as an empty `LazyFrame` with extra methods. In the installed version it is a class of its own, not a subclass of `LazyFrame`, but you use it in the same way.

The second difference is how you construct a query, because OpenDP extends the Polars API in two places. The query gets the methods `summarize`, which reports the noise that each aggregate will get, and `release`, which spends the budget. Expressions get the namespace `.dp`, whose methods are the aggregates that add noise, such as `dp.len`, `.dp.sum` and `.dp.median`. In the snippet, the count's sensitivity of 1 over a share of $\varepsilon/5$ gives a scale of 5.

The third difference is how you run a query. In plain Polars you call `collect` on a lazy frame. In OpenDP you must call `release` first, as the refusal in the text block shows. The release takes the query's share of the budget and returns a `OnceFrame`, which hands out its rows once. A second `collect` fails, so keep the collected frame, not the released query.

```
second collect: FailedFunction("OnceFrame has been exhausted")
```

The fourth difference is what a query can contain. OpenDP makes guarantees only about the query plans and expressions that it has a proof for, so it acts as an allow-list. It refuses a plain Polars `mean` of the age column, with the message in the text block above, because that expression adds no noise. The private mean is `.dp.mean`, which the next chapter uses. OpenDP also changes the arguments of some expressions that it accepts, mostly so that a failure cannot raise an error that depends on the data.

## 2.5. A margin, in preview

A count needs no public information beyond the unit, but the sum of working hours in the next chapter needs one more fact. The script asks for that sum on the context above and prints the refusal. Then it builds a context that carries the missing fact.

```
margin = dp.polars.Margin(
    # at most 150,000 responses per quarter (50,000 households of under three people,
    ↪ both public figures)
    # over the 36 quarters of the survey: a bound on the length of the frame that costs
    ↪ no privacy
    max_length=QUARTER_RESPONSES * CONTRIBUTIONS,
)
bounded = make_context(frame, margins=[margin])
```

```
margin: Margin(by=[], max_length=5400000, max_groups=None, invariant=None)
sum without a margin: MakeTransformation("must specify 'max_length' in a margin with
↪ by=[]")
sum with the margin: scale=14,405.2  (36 x 80 over epsilon / 5 = 14,400)
```

A **margin** is a set of facts about the frame, grouped by some columns, that you declare public. With `by=[]` the group is the whole frame, and `max_length` is a bound on its number of rows. A sum needs that bound because OpenDP's guarantee also covers overflow and floating-point rounding. A sum of millions of floats loses a little precision, and the proof that this loss leaks nothing needs to know how many numbers were added. Without the bound, the library refuses the sum, as the first line of the block shows. With it, a sum of clipped usual hours gets a scale a little above 14,400, the plain sensitivity over the share. The small extra pays for the rounding.

The bound must be public, and the documentation shows how to find one. Eurostat publishes that the French survey reaches about 50,000 households a quarter, and public statistics put the average household below three people. So 150,000 responses a quarter is a generous bound, and over the 36 quarters it gives 5.4 million rows. This bound does not come from the row count printed earlier, and it must not, because a bound read from the data leaks the data. You give away a fact that an adversary could look up anyway, and in exchange the library can prove that arithmetic cannot weaken the guarantee. A margin can also bound the number of groups and declare the keys or the lengths of groups invariant, which Grouping takes up.

## 2.6. Summary

Table 2.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| The LFS sample | Public microdata of the French Labour Force Survey, 3,811,932 responses from 1,682,306 synthetic identifiers | Already protected by statistical disclosure control, so releases can be checked against the truth |
| `pl.scan_parquet(path)` | A `LazyFrame` that reads rows only when a query runs | Give the context the lazy frame, not a collected table |
| `pl.scan_csv` and schema inference | Types inferred from the values, so they depend on the data | Read a format with a schema, or load strings with `infer_schema=False` and cast |
| `dp.unit_of(contributions=36)` | The privacy unit: neighbours add or remove up to 36 rows of one person | Take the bound from the design of the data, never from the data |
| `dp.loss_of(epsilon=1.0)` | The budget, under pure differential privacy | One person's data changes any output's probability by at most $e^{\varepsilon}$ |
| `split_evenly_over`, `split_by_weights` | How the budget is divided; `context.d_mids` lists the shares | Weights are shares, not values of $\varepsilon$; the shares sum to the budget |
| `del frame` | Removes your own reference to the data | Anything computed outside the context is outside the guarantee |
| `context.query()` | A `LazyFrameQuery`: a mock that knows the schema and accepts only proven expressions | Build and summarize against the mock; `collect` works only after `release` |
| `summarize()` | Scale and distribution of each aggregate, before any release | It depends on the budget and the unit, not on the data |
| `release().collect()` | Spends the share and returns the released frame once | Keep the collected frame; a second `collect` fails |
| `dp.polars.Margin(max_length=A public)` | bound on the length of the frame, which sums need | Build it from published figures, as 150,000 responses a quarter here |

## 2.7. References

- OpenDP documentation: "Working with Tabular Data" (the dataset description, loading data, and mediating access with `Context`), "Polars vs. OpenDP" and "Context" in the API user guide, and the "Why OpenDP?" paragraph of the front page.

- Eurostat, *EU Labour Force Survey*, public microdata and the EU-LFS database user guide: the survey, its columns and their coding.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the scales of this chapter.
- The full edition's chapters on the definition in the full edition and on post-processing, group privacy and composition in the full edition derive the results this chapter states.

# Part II.

# Tabular data

This part does the everyday work of a private analysis: statistics on a table. OpenDP reads the lazy queries of Polars, a dataframe library, before they run. So every filter, cast and aggregation is part of the privacy guarantee. The thread of the part is that every statistic needs two things from you: a bound on what one person can contribute, and a share of the budget. The library turns them into a noise scale, which you read before a release.

The chapters follow the documentation's tutorial on tabular data. Essential statistics releases counts, sums, means, medians and quantiles, with the noise scale and error bar that `summarize` reports. Grouping releases one row per group, with public keys or keys that the library selects privately. Identifier truncation and bounds truncates the rows of a person who owns many, and bounds the groups that the person touches. Preparing microdata covers the expressions that OpenDP allows before an aggregation, and the data types it knows. Assessing utility and choosing the noise turns scales into error bars and back, and compares Laplace with Gaussian noise under zero-concentrated differential privacy.

Table 2.3.: The chapters of this part and the questions they answer.

| Chapter | Question it answers |
|---|---|
| Essential statistics | How do I release a count, a sum, a mean or a quantile, and how large is its error? |
| Grouping | How do I release one statistic per group, when the groups are known and when they are not? |
| Identifier truncation and bounds | What do I do when one person owns many rows? |
| Preparing microdata | Which transformations may I apply before aggregating, and how do types and nulls behave? |
| Assessing utility and choosing the noise | How do I read an error bar, and when is Gaussian noise the better choice? |

At the end of this part, you can release a table of statistics on a dataset in which each person owns many rows. Each cell has an error bar, and the whole table has a guarantee.

# 3. Essential statistics

With a context on a table, you can ask the everyday questions. How many responses are there, how many hours do people work in total and on average, and what is a typical week? This chapter answers them on the Labour Force Survey (LFS) sample with OpenDP's Polars queries. Each answer is a statistic with noise. The library finds the noise scale from a bound on what one person can change and from a share of the budget. You read that scale and its error bar before you release anything.

The chapter follows the documentation's tutorial on essential statistics and its page on aggregation. You count rows, decide where bounds come from, sum a column, release a mean in two ways, and choose a median and quartiles from candidates. The practitioner edition's workflow on releasing statistics does the same work on a retail dataset.

The script opendp_essential_statistics.py runs every query on OpenDP 0.16.0. It reads the sample lazily through `practitioner_datasets.france_lfs()`, as the documentation reads the same file through `dp.examples.get_france_lfs_path()`. The scales, error bars, refusals and true values that it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_essential_statistics.py
```

## 3.1. The context for this chapter

The context is the one that the previous chapter builds, with a privacy unit of 36 rows, one per quarter. Its budget is a privacy parameter $\varepsilon$ of 1 under pure differential privacy (DP), split evenly over five queries. It also has that chapter's margin, a public bound on the number of rows. The new argument `invariant` is for the section on the mean.

```
def make_context(frame: pl.LazyFrame, invariant: Literal["keys", "lengths"] | None =
↪  None) -> dp.Context:
    """The chapter's context: 36 rows per person, epsilon = 1 split evenly over 5
    ↪  queries, and one public margin."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
        margins=[dp.polars.Margin(max_length=MAX_LENGTH, invariant=invariant)],  # a
↪  public bound on the rows
    )
```

The chapter's five releases come from this one context. A query takes its share of the budget when you build it, so the comparisons on this page, which only read scales, use new contexts with the same settings. A real analysis builds one context and plans its shares before the first query.

## 3.2. Counting responses

The first release on the survey is its number of rows. The query selects `dp.len`, OpenDP's private version of Polars' `len`, and the context fills in the noise.

```
count_query = context.query().select(dp.len(signed=True))
count_summary = count_query.summarize(alpha=0.05)
dp_count = count_query.release().collect().item()
```

Before `release`, call `summarize`. It describes the measurement that the context built, without running it, one row per statistic.

```
shape: (1, 5)
┌────────┬──────────────┬──────────────────┬─────────┬──────────────┐
│ column ┆ aggregate    ┆ distribution     ┆ scale   ┆ accuracy     │
│ ---    ┆ ---          ┆ ---              ┆ ---     ┆ ---          │
│ str    ┆ str          ┆ str              ┆ f64     ┆ f64          │
╞════════╪══════════════╪══════════════════╪═════════╪══════════════╡
│ len    ┆ Frame Length ┆ Integer Laplace  ┆ 180.0   ┆ 539.731115   │
└────────┴──────────────┴──────────────────┴─────────┴──────────────┘
```

The aggregate is the length of the frame, and the distribution is integer Laplace, because a count is an integer. The scale comes from the privacy unit. One person more or less changes up to 36 rows, so the **sensitivity** of the count, the most that one person can change it, is 36. The Laplace scale is the sensitivity over the query's share of $\varepsilon$, so 36 over 0.2. The full edition's chapter on the Laplace and Gaussian mechanisms derives why that scale gives $\varepsilon$-DP. The accuracy is the error bar at the level that you passed as `alpha`: with probability 0.95, the released count is within 539.73 of the true one.

The error bar depends on `alpha`, not on the data, so both numbers below cost nothing to read. If you ask for more confidence, the bar widens, and at `alpha=0.01` the same scale gives 829.43.

```
count: scale=180.0 95% accuracy=539.73 99% accuracy=829.43
sensitivity 36 over epsilon 0.2: scale 180.0
```

The bar covers the noise and only the noise. It does not cover the sampling error between this sample and the population of France, or the recoding and suppression that Eurostat applied before publication. And it counts responses, not people. The argument `signed=True` keeps the released count a signed integer, so a small count can come out negative when the noise is larger. Without the argument, the library clamps a negative count to zero, which biases small counts upwards, and it warns that the default will change.

```
without signed=True: FutureWarning: The default value is currently signed=False, and
 ↪  counts are clamped at zero, consistent with past releases, but the default will be
 ↪  changing to signed=True.
```

A column has its own counts, which the documentation's aggregation page lists. On a column, `.dp.len` counts the rows with nulls, `.dp.count` the rows that are not null, and `.dp.null_count` the rows that are. So the column counts answers to the hours question, and its null count measures non-response. Then `.dp.n_unique` counts distinct values, with null as one value. One person's rows change each count by at most 36, so each has the scale of the frame length when it is alone in a query. The excerpt asks for all five in one query.

```
five_counts = variants.query().select(
    dp.len(signed=True),  # rows of the frame
    hwusual.dp.len(signed=True).alias("expr_len"),  # rows of the column, nulls
 ↪  included
    hwusual.dp.count(signed=True).alias("count"),  # rows of the column that are not
 ↪  null
    hwusual.dp.null_count(signed=True).alias("null_count"),  # rows of the column that
 ↪  are null
    hwusual.dp.n_unique(signed=True).alias("n_unique"),  # distinct values, null among
 ↪  them
)
five_summary = five_counts.summarize(alpha=0.05)
```

```
HWUSUAL len alone: scale=180.0
HWUSUAL count alone: scale=180.0
HWUSUAL null_count alone: scale=180.0
HWUSUAL n_unique alone: scale=180.0
five counts in one query: scale=900.0 each, 95% accuracy=2696.7
```

Five statistics in one query share its budget equally, so each gets a fifth of it and five times the scale. Every query on this page divides its share evenly among its statistics, unless you set their scales yourself. Grouping gets the null and the non-null count together for one count's noise, by grouping on whether the value is null.

## 3.3. Where bounds come from

Every statistic after the count needs the range of its values. A sum or a mean needs a lower and an upper bound, and a median needs a list of candidates. The bounds set the sensitivity, because one person's rows can move a sum by at most their number times the width of the bounds. That is the clipping idea, which the full edition's chapter on sensitivity derives. Bounds that are too wide add more noise than the data need. Bounds that are too narrow clip real values and bias the result, and the error bar does not show that bias.

Take bounds from domain knowledge, from a comparable public dataset, or from a share of the budget spent on a coarse quantile query. Never take them from the private data. A bound read off the data, such as its maximum, depends on one person's value and leaks it, outside the guarantee.

This page uses domain knowledge: a working week of 0 to 80 hours, with the whole hours from 20 to 59 as candidates.

## 3.4. The sum of hours

The column `HWUSUAL` holds the hours that a person usually works in a week, with 99 for "not applicable". So the query first filters out those rows, and OpenDP accounts for the filter as part of the query. Then `.dp.sum` clips every value into the bounds, sums, and adds noise.

```
hours: Any = pl.col("HWUSUAL").cast(int)  # Any: the .dp namespace is added at run time
sum_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)  # 99 means "not applicable"
    .select(hours.fill_null(35).dp.sum(bounds=(0, 80)))  # fill, clip, sum, add noise
)
sum_summary = sum_query.summarize(alpha=0.05)
dp_sum = sum_query.release().collect().item()
```

```
sum HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
sensitivity 36 x 80 = 2,880 over epsilon 0.2: scale 14,400
```

One person owns at most 36 rows of at most 80 hours each, so the sensitivity is their product. An error bar of 43,139 hours is small against a total near 56 million. The noise does not grow with the number of rows, so the relative error decreases as the table grows.

The sum needs the margin, and it says so when the margin is missing.

```
without a margin: must specify 'max_length' in a margin with by=[]
```

As the previous chapter explains, the library must know the largest number of rows to account for overflow and rounding in a sum. The `by=[]` in the message names the margin of the whole table, and Grouping adds margins per group. The refusal does not depend on the data, as a snippet on three rows shows.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35.0, 39.0, 40.0]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
)
try:
    context.query().select(pl.col("hours").dp.sum(bounds=(0.0, 80.0))).summarize()
except dp.OpenDPException as err:
    print(err.message)  # must specify 'max_length' in a margin with by=[]
```

The integer cast is the second adjustment. The column holds floats, and a float sum pays for the rounding of up to `max_length` additions. At this chapter's bound the price is small, but a careless bound of a billion rows raises the float scale to 192,035.68. The cast removes the rounding and its dependence on `max_length`, so cast whenever the values are whole numbers.

```
float sum, max_length 5,400,000: HWUSUAL: Sum, Float Laplace, scale=14,405.18 95%
 ↪  accuracy=43,154.1
float sum, max_length 1,000,000,000: HWUSUAL: Sum, Float Laplace, scale=192,035.68 95%
 ↪  accuracy=575,287.5
int sum, max_length 1,000,000,000: HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95%
 ↪  accuracy=43,139.0
```

The third adjustment is `fill_null`, because a sum has no value when a row is null. If you write the clip and the sum by hand, the library refuses a nullable column. The shortcut `.dp.sum` instead fills a null with the midpoint of the bounds, 40 here. With `fill_null(35)` you choose the legal working week in France, a public fact. On this data the choice has no effect: Polars drops a row whose test `!= 99` is null, so the filter drops every null. To confirm the midpoint rule, the script also sums the whole column, nulls included.

```
sum without fill_null: scale=14,400.0 (accepted; a null becomes the midpoint 40)
clip, sum and laplace by hand, without fill_null: input data (HWUSUAL) might contain
 ↪  nulls. Preprocess your data with `.fill_null`.
```

The error bar is about the *clipped* sum. A value above 80 becomes 80 before the sum, and no error bar reports that. Here no applicable value is above the bound, as the check at the end shows.

## 3.5. The mean

A mean is a sum divided by a count, and under the add/remove relation both are private. The documentation recommends that you release the two in one query and divide afterwards. The division of two released numbers is post-processing, which costs nothing, as the full edition's chapter on properties proves.

```
joint_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.sum(bounds=(0, 80)), dp.len(signed=True))  # a null becomes the
 ↪  midpoint, 40
)
joint_summary = joint_query.summarize(alpha=0.05)
joint = joint_query.release().collect().with_columns(mean=pl.col("HWUSUAL") /
 ↪  pl.col("len"))
```

```
joint HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95% accuracy=86,277.6
joint len: Frame Length, Integer Laplace, scale=360.00 95% accuracy=1,079.0
```

The two statistics split the query's share evenly, so each has twice the scale that it had alone. But the pair costs only one share, and one released count can also divide the sums of other columns. On this context, `.dp.mean` releases the same noisy sum and noisy length, as the script shows.

```
dp.mean on the add/remove context: HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95%
 ↪  accuracy=86,277.6
dp.mean on the add/remove context: HWUSUAL: Length, Integer Laplace, scale=360.00 95%
 ↪  accuracy=1,079.0
```

The alternative is to declare the number of rows public, which the documentation calls bounded DP. The neighbouring relation becomes replace-one: two neighbours have the same number of rows and differ in one person's values. In a margin, `invariant="lengths"` makes the number of rows public, and `.dp.mean` then adds noise to the sum only.

```
applicable_frame = frame.filter(pl.col("HWUSUAL") != 99.0)  # filtered outside the
 ↪  context: see the text
bounded = make_context(applicable_frame, invariant="lengths")  # the number of rows is
 ↪  public
mean_query = bounded.query().select(hours.dp.mean(bounds=(0, 80)))
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
bounded mean HWUSUAL: Sum, Integer Laplace, scale=7,200.00 95% accuracy=21,569.8
bounded mean HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
```

The sum's scale is half of the scale with a whole share under add/remove. When the length is fixed, a changed row counts as one removal and one addition, so a distance of 36 means at most 18 changed rows. The length has a scale of zero, because it is released exactly. That is the weaker guarantee. The release still hides what a person answered, but not whether the person is in the table.

The excerpt shows a second cost: the filter on the code 99 runs outside the context. Inside the context, a filter makes the number of rows unknown again. Then the invariant is lost, and `.dp.mean` falls back to a noisy sum and a noisy length, as the script shows. A filter outside the context is sound only if the number of applicable responses is truly public. If it is not, use the joint query.

```
dp.mean after a filter inside the bounded context: HWUSUAL: Sum, Integer Laplace,
 ↪  scale=28,800.00 95% accuracy=86,277.6
dp.mean after a filter inside the bounded context: HWUSUAL: Length, Integer Laplace,
 ↪  scale=360.00 95% accuracy=1,079.0
```

## 3.6. The median and the quartiles

You cannot release a median by adding noise to it. When few values lie near the middle, one person's rows can move the median by any amount, so no finite scale covers every dataset. So

OpenDP chooses the median from **candidates** that you supply. Each candidate gets a score for how unbalanced the counts below and above it are. OpenDP adds exponential noise to every score and releases the candidate with the smallest noisy score. That is the exponential mechanism, which the full edition's chapter on selection derives. The candidates here are the whole hours from 20 to 59.

```
candidates = list(range(20, 60))  # whole hours from 20 to 59
median_query = context.query().filter(pl.col("HWUSUAL") !=
 ↪  99.0).select(hours.dp.median(candidates))
median_summary = median_query.summarize(alpha=0.05)
dp_median = median_query.release().collect().item()
```

```
median HWUSUAL: 0.5-Quantile, ExponentialMin, scale=360.00 95% accuracy=none
score sensitivity 36: scale 2 x 36 / 0.2 = 360
median without a margin: Must know max_length
```

The aggregate reads 0.5-quantile because `.dp.median` calls `.dp.quantile` with `alpha=0.5`. The scale is twice the score's sensitivity over the query's share, a factor that comes from the analysis of the exponential mechanism. The accuracy is empty because the noise goes into the scores, not into the answer, so there is no error bar in hours. Instead, a candidate whose score is far above the best one is very unlikely to win, as the check at the end shows. The quantile also needs the margin, because the scores are integers whose size depends on the number of rows.

A quantile other than the median takes `alpha`, the share of values below it. The excerpt asks for the three quartiles in one query.

```
quartile_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.quantile(a, candidates).alias(f"{a}-Quantile") for a in [0.25,
 ↪  0.5, 0.75])
)
quartile_summary = quartile_query.summarize()
dp_quartiles = quartile_query.release().collect().row(0)
```

```
quartiles 0.25-Quantile: 0.25-Quantile, ExponentialMin, scale=3,240.00 95%
 ↪  accuracy=none
quartiles 0.5-Quantile: 0.5-Quantile, ExponentialMin, scale=1,080.00 95% accuracy=none
quartiles 0.75-Quantile: 0.75-Quantile, ExponentialMin, scale=3,240.00 95%
 ↪  accuracy=none
0.25-quantile alone: scale=1,080.0
```

Three quantiles in one query split its share three ways, so the median's scale triples. The quartiles' scales are three times larger again, but the median did not get more of the budget. A quantile's score is an integer scaled by the denominator of `alpha`, four for a quartile and two for the median. So one person's rows move a quartile's score three times as far. The last line confirms the even split: the first quartile alone gets a third of its scale in the trio.

## 3.7. A check against the truth

The sample is public, so the script computes the true values and reads one run's releases against them. On private data, do this check on public or historical data during development, and release the real data once.

```
true responses: 3,811,932
true responses with HWUSUAL not 99: 1,496,192 (nulls among them: 0; in the whole
 ↪  column: 20,819)
true sum of hours after the filter, clipped to [0, 80]: 56,315,594 (values above 80: 0)
true sum of the whole column, clipped: 240,742,034 with nulls as 40, 239,909,274 with
 ↪  nulls dropped
true mean: 37.64  (bounded mean 95% accuracy: 0.0144)
true quartiles: 35, 37, 40
score gap from the true quartile to the nearest other candidate: 0.25: 219,506  0.5:
 ↪  74,383  0.75: 116,811
```

The second and third lines settle two open questions: the filter leaves no nulls, and no applicable value is above 80. The bounded mean's error bar is the sum's error bar divided by the exact length. The last line takes the place of an error bar for the quantiles. Each nearest other candidate scores at least 74,383 above the true quartile, against a noise scale of at most 3,240. So a wrong candidate is far too unlikely to see, and the test requires the exact quartiles.

```
released (one run):
  responses 3,811,961  (true 3,811,932, within 539.7 with probability 0.95)
  sum of hours 56,308,788  (true 56,315,594, within 43,139.0)
  joint: sum 56,335,491  count 1,495,681  mean 37.67  (true 37.64)
  bounded mean 37.6411  (true 37.6393, within 0.0144)
  median 37  quartiles 35, 37, 40  (true 37; 35, 37, 40)
  sum of the whole column 240,726,542  (true with nulls as 40: 240,742,034)
```

The count and the sum are inside their error bars, as they must be about 19 times in 20. A count error of a few hundred is small on 3.8 million responses. The joint mean is off in the second decimal, because it divides two noisy numbers. The bounded mean is inside a bar of about a hundredth of an hour, which is what a public length buys. The median and the quartiles are exact. The sum of the whole column is near the total that counts each null as 40 hours, far from the total without them.

## 3.8. Summary

Table 3.1.: The calls of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.unit_of(contributions=36)` | The privacy unit: neighbours differ by one person's rows, up to 36 | Count the most rows one person can own, not the typical number |
| `dp.polars.Margin(max_length=...)` | A public bound on the number of rows | Sums, means and quantiles refuse to run without it; take it from public facts |
| `dp.len(signed=True)` | The number of rows, with Laplace noise of scale sensitivity over $\varepsilon$ | Without `signed=True` a negative count is clamped to zero |
| `.dp.len`, `.dp.count`, `.dp.null_count`, `.dp.n_unique` | A column's rows, non-nulls, nulls and distinct values | Each is as sensitive as the frame length; several in one query share its $\varepsilon$ |
| `query.summarize(alpha=0.05)` | The column, aggregate, distribution, scale and error bar of each statistic | Read it before `release`; a smaller `alpha` widens the bar |
| `.dp.sum(bounds=(0, 80))` | Fills nulls with the midpoint, clips, sums, adds noise | Cast to an integer; the error bar excludes the bias of clipping |
| Sum and `dp.len` in one query | A mean under add/remove, by dividing afterwards | Each statistic gets twice the scale; the division is free post-processing |
| `invariant="lengths"` and `.dp.mean` | A bounded-DP mean: noise on the sum, the exact length | Weaker guarantee; filter outside the context only if the row count is truly public |
| `.dp.median(candidates)`, `.dp.quantile(alpha, candidates)` | The candidate nearest the quantile, by the exponential mechanism | No error bar; candidates from domain knowledge; several quantiles split the share evenly |

## 3.9. References

- OpenDP documentation: "Essential Statistics" in the tutorial on tabular data, and "Aggregation" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the counts and sums.
- McSherry and Talwar, "Mechanism Design via Differential Privacy" (2007): the exponential mechanism behind the median and the quantiles.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

# 4. Grouping

Most private releases are tables with one row per group, such as counts by age and employment status or the mean hours worked by sex. Grouping adds a danger that a single count does not have. The keys of the table are data too, because a group that one person fills alone shows that person just by its presence in the table. This chapter gives the three ways in which OpenDP releases a grouped table, and what each one costs.

First the library selects the keys, and only the groups whose noisy count passes a threshold appear. Then a public key list gives the keys, and then margins that declare the keys or the group sizes public. At the end, you can choose between the three and say what each gives up.

The script opendp_grouping.py runs every query of this chapter on OpenDP 0.16.0 against the full survey sample. The scales, error bars, thresholds, error messages and true group counts that it prints are the same on every run. The released counts and means are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_grouping.py
```

## 4.1. Why grouping keys are dangerous

The data are the France Labour Force Survey (LFS) sample of The data and the context. The privacy unit is the one of that chapter's context: up to 36 rows per person, one per quarter.

A grouping of the responses by `AGE` and by `ILOSTAT`, the labour status code, gives 16 groups in truth. The smallest, the responses with no recorded age, holds 255 rows. A noisy count with an error bar of a few hundred hides how many rows it has, but not that it exists. In a finer grouping, say by age, sex and town, one person can fill a group alone. Then the noise hides the count, but the row's presence says that someone with those keys answered the survey. So the keys of a grouped table need protection as much as the counts do.

OpenDP enforces this. On a context with a pure budget, `summarize` refuses a count by `AGE` and `ILOSTAT` before it computes anything.

```
without delta: The key-set of {col("AGE"), col("ILOSTAT")} is private and cannot be
 ↪  released without a filter or join. Please pass a filtering threshold into
 ↪  make_private_lazyframe or conduct a join against a public key-set.
```

The message names the two ways out. A filter keeps only the groups whose noisy count passes a threshold, and it needs a $\delta$ in the budget. A join uses a public key list. The snippet shows the summary of each on a toy frame.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
data = pl.LazyFrame({"region": ["A", "A", "B"], "hours": [35, 40, 20]})
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=data, privacy_unit=unit,
 ↪  privacy_loss=dp.loss_of(epsilon=1.0), split_evenly_over=1)
counts = pure.query().group_by("region").agg(dp.len(signed=True))
try:
    counts.summarize()
except dp.OpenDPException as err:
    print("cannot be released without a filter or join" in str(err))  # True
loss = dp.loss_of(epsilon=1.0, delta=1e-6)
approx = dp.Context.compositor(data=data, privacy_unit=unit, privacy_loss=loss,
 ↪  split_evenly_over=1)
summary =
 ↪  approx.query().group_by("region").agg(dp.len(signed=True)).summarize(alpha=0.05)
print(summary.columns)  # ['column', 'aggregate', 'distribution', 'scale', 'accuracy',
 ↪  'threshold']
print(int(summary["threshold"].item()))  # 14
keyed = counts.with_keys(pl.LazyFrame({"region": ["A", "B", "C"]}))
print(keyed.summarize().columns)  # ['column', 'aggregate', 'distribution', 'scale']
```

## 4.2. Stable keys

The first way is to let the library decide which groups to show. The documentation calls the keys that it keeps **stable keys**, because a group is stable when it stays in the data after the removal of any one person. The release keeps a group only when its noisy count is above a threshold. The library sets the threshold so that a group that one person fills alone appears with probability at most $\delta$. The chapter uses the documentation's budget, `dp.loss_of(epsilon=0.25, delta=1e-7)`. This is **partition selection**, and the full edition's chapter on sensitivity derives why a count with a threshold satisfies approximate differential privacy.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at
 ↪  most 36 quarters
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # the delta pays for the
 ↪  threshold
    split_evenly_over=1,
)
stable = context.query().group_by("AGE", "ILOSTAT").agg(dp.len(signed=True))
stable_summary = stable.summarize(alpha=0.05)
stable_hat = stable.release().collect()
```

```
stable keys: privacy loss (Approximate(MaxDivergence), (0.25, 1e-07))
stable keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
 ↪  threshold=2773
```

Read the summary before you release. One person's 36 rows can all be in one group, so the count of any group has sensitivity 36, as the ungrouped count had. That sensitivity over $\varepsilon = 0.25$ gives the scale of 144. With probability 0.95, a released count is within 431.9 of the truth. The new column is the threshold, 2773: the release drops a group whose noisy count is not above it.

The library calibrates the threshold to the worst case for one person. Their 36 rows could all go into one otherwise empty group, with a true count of 36. Discrete Laplace noise of scale 144 lifts that group above 2773 with probability $2.77 \times 10^{-9}$. But the same person's rows could instead spread over 36 groups of one row each, and any of them might pass. So the library adds the chances over 36 groups, which gives $9.98 \times 10^{-8}$, just within $\delta = 10^{-7}$. One step lower, the total is above $\delta$, so 2773 is the smallest threshold that keeps the promise.

```
one person's 36 rows clear 2773 with probability 2.77e-09; over 36 groups 9.98e-08 <=
 ↪  delta 1e-07
at threshold 2772: over 36 groups 1.01e-07 > delta
```

The threshold falls with a looser $\delta$ and with a smaller scale, as the script shows. A larger privacy unit raises both scale and threshold, because a person who owns more rows can make a larger group alone.

```
threshold with delta=1e-05: scale=144.00 threshold=2110
threshold with epsilon=1: scale=36.00 threshold=720
```

On this public data, 14 of the 16 groups have a true count above 2773, so the release must have 14 rows. A dropped group does not show itself, and the number of rows is itself noisy information. If you see fewer groups than you expected, do not look at the data to find which are missing, because that look is outside the guarantee.

## 4.3. Explicit keys

If you know the keys in advance, you can give them to the library and keep a pure budget. **Explicit keys** are a key list that you supply, and the release has every key in it, with or without rows in the data. The list must be fixed without a look at the data, because a key list read off the data is a release in itself. Here it is every survey year crossed with every status code of the codebook, including codes that never occur in this sample.

```
explicit = (
    context.query()
    .group_by("YEAR", "ILOSTAT")
    .agg(dp.len(signed=True))
    .with_keys(public_keys())  # the years crossed with the codebook's codes, fixed
 ↪  without the data
)
explicit_summary = explicit.summarize(alpha=0.05)
explicit_hat = explicit.release().collect()
```

```
explicit keys: 54 keys, 36 with rows in the data, 18 without
explicit keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The scale and the accuracy do not change, and the threshold column is gone, because no group is dropped. The 18 keys without rows come back as signed noise around zero, so a reader cannot tell an empty group from a small one. A released key set is public, so the documentation reuses the keys of a stable release as the explicit keys of a later query.

`with_keys` stands for a join: the library rewrites the query as a right join of the aggregation onto the key list. If you write that join yourself, you get the same summary.

```
right join len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

Only a left or a right join against a key list releases a key set. An inner join keeps only the keys that have rows, which shows which keys the data holds, so the library refuses it.

```
inner join: only left or right joins can be used to release key-sets
```

## 4.4. Invariant keys

The third way is to declare the keys of a grouping public through a **margin**, the set of public facts about a grouping that the data and the context introduced. The margin `dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")` says that the set of quarters in the data is public. The library then releases every group's key in the clear, with no threshold and no $\delta$.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")],  # the
 ↪  quarters are public
)
quarterly = context.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
quarterly_summary = quarterly.summarize(alpha=0.05)
quarterly_hat = quarterly.release().collect()
```

```
invariant keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true quarterly counts: 36 quarters, smallest 82,504 (2006 Q3), largest 131,656 (2011
 ↪  Q4)
error bar as a share of the smallest quarter: 0.52%, of the largest: 0.33%
```

An **invariant** is a fact about the data that the guarantee no longer covers, so the guarantee is weaker. With invariant keys, a quarter with a single respondent would appear in the table and point at that person. Here nothing is lost, because the quarters are the survey's design and were known before anyone answered. Elsewhere, be reluctant to use invariants, as the documentation advises. The error bar is the same as before.

### 4.4.1. Invariant lengths

A margin can go one step further and declare the group sizes public, with `invariant="lengths"`. Public sizes imply public keys, so this gives up more. The documentation's example is the mean of `HWUSUAL`, the hours usually worked, by sex. The code 99 means not applicable and would bias the mean, so those rows must go first. But a filter inside the query changes the group sizes and voids the margin, and then the library treats the keys as private again.

```
filter inside the query: The key-set of {col("SEX")} is private and cannot be released
 ↪  without a filter or join. Please pass a filtering threshold into
 ↪  make_private_lazyframe or conduct a join against a public key-set.
```

So filter the LazyFrame before you build the context, and the margin then describes the filtered data, 1,496,192 rows.

```
context = dp.Context.compositor(
    data=working,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=MEAN_EPSILON),
    split_evenly_over=1,
    margins=[
        dp.polars.Margin(
            by=["SEX"],
            invariant="lengths",  # the number of responses per sex is public, keys
 ↪  included
            max_length=MAX_LENGTH,  # needed by any sum: a public bound on the rows in
 ↪  a group
            max_groups=3,  # male, female or null
        )
    ],
)
mean_hours = context.query().group_by("SEX").agg(hours().dp.mean(HOURS_BOUNDS))
mean_summary = mean_hours.summarize(alpha=0.05)
mean_hat = mean_hours.release().collect()
```

```
invariant lengths HWUSUAL: Sum, Integer Laplace, scale=1,764.00 95% accuracy=5,285.0
invariant lengths HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
true lengths: SEX=1 779,714  SEX=2 716,478
true mean hours: SEX=1 40.778  SEX=2 34.224
mean 95% accuracy: SEX=1 0.0068  SEX=2 0.0074
```

The library releases the mean as a sum divided by a length, with a summary row for each. The length has scale 0, because it is public and exact. The sum has scale 1764 at $\varepsilon = 1$, which is its sensitivity. The group sizes cannot change, so one person's rows can only be swapped for other rows. That is at most 18 swaps, each of which moves the sum by at most 98 hours. The documentation's text gives 7056 for this scale, but its own table and the installed version give 1764.

The sum's error bar of 5285 hours is divided by a public length of several hundred thousand. So on the mean, the error bar is below a hundredth of an hour for either sex. In version 0.16.0 the released means are floats, while the documentation shows them rounded to whole hours.

## 4.5. Margins as public descriptors

A margin describes one grouping, named by its `by` columns, and a context takes a list of margins. So one context can protect most groupings fully and relax a few. Besides `invariant`, a margin has two bounds. The first, `max_groups`, is a public upper bound on the number of groups. It lowers the threshold of a release with stable keys. A person's rows can then make at most that many groups, so the library spreads $\delta$ over fewer of them. With a bound of 16, the threshold falls from 2773 to 2656.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["AGE", "ILOSTAT"], max_groups=GROUPS_IN_TRUTH)],  #
 ↪  at most 16 groups
)
bounded_summary = context.query().group_by("AGE",
 ↪  "ILOSTAT").agg(dp.len(signed=True)).summarize()
```

```
max_groups=16: threshold=2656
```

The second, `max_length`, is a public upper bound on the number of rows in any one group. A sum needs it, because the guarantee covers overflow, and a sum of unbounded length could overflow. A count needs neither bound. A sum of hours per sex, with a margin that has no `max_length`, is refused.

```
sum per SEX without max_length: must specify 'max_length' in a margin with
 ↪  by=[col("SEX")]
```

The guarantee does not check either bound. The bound of 16 above comes from the public data, which is acceptable for a tutorial but wrong for a release. Set both bounds conservatively from the design of the survey. A bound that the data violate voids the guarantee without an error.

## 4.6. The noise per group and the ungrouped count

Every count in this chapter has the same scale, 144, from a group of the 16 to the ungrouped count of all rows.

```
ungrouped count len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
error bar as a share of the true count: all rows 0.011%, largest group 0.063%, smallest
 ↪  group above the threshold 3.3%
```

All 36 rows of a person can be in one group, so a group's count is as sensitive as the total. What changes is the relative error, which is 3.3% for the smallest group above the threshold. So finer

groupings cost utility twice: more groups fall below the threshold, and the others have a larger relative error. Group as finely as the question needs and no finer, and ask for several statistics in one query. Assessing utility and choosing the noise returns to this grouping with Gaussian noise, whose scale depends on how a person's rows spread over the groups. Identifier truncation and bounds shows how to bound that spread.

## 4.7. Checking the release against the truth

The LFS sample is public, so the script compares each release with the true table, a check to run on public or historical data during development. For the stable keys, the smallest group above the threshold has 13,157 rows, and the largest below it has 847. So no group is near the threshold, and every run must return 14 groups. A group within one or two error bars of the threshold would appear on some runs only.

```
smallest true count above the threshold: 13,157  largest below it: 847
test bound for a table: 4 x accuracy = 1,727.5
released (one run):
  stable groups released: 14  smallest released count: 13,321  smallest true count
 ↪  among them: 13,157
  stable AGE=47 ILOSTAT=1: 652,892  (true 652,801)
  stable AGE=65 ILOSTAT=2: 13,321  (true 13,157)
  stable largest |released - true|: 349
  explicit YEAR=2005 ILOSTAT=4: 312  (true 0)
  explicit YEAR=2013 ILOSTAT=1: 188,396  (true 188,349)
  explicit keys released: 54  largest |released - true|: 530
  quarter 2005 Q1: 87,307  (true 87,372)
  quarters released: 36  within their 95% accuracy: 36 of 36  largest |released -
 ↪  true|: 307
  mean hours: SEX=1 40.786 (true 40.778)  SEX=2 34.229 (true 34.224)
```

Read the run against the error bar of 431.9. The stable release has its 14 groups, all within the bar. The explicit release has all 54 keys, including the conscripts of 2005, whose true count is zero and whose released count is pure noise. Its largest error is a little over the bar. That is expected in a table of 54 counts, because each count misses a 95% bar about once in twenty. All 36 quarters are within their bars, and the means are within a hundredth of an hour of the truth.

The test checks the deterministic lines exactly, and every named count and mean within three error bars. For the largest error of a whole table it allows four bars, because a table has up to 54 counts. Release statistics with a library in the practitioner edition, in the practitioner edition, does the same kind of check on a shop's customers.

## 4.8. Summary

Table 4.1.: The calls and concepts of a grouped release, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `group_by(...).agg(dp.len(...))` without keys or margins | Releases the key set is private | Choose stable keys, explicit keys or a margin before you query |
| Stable keys, `dp.loss_of(epsilon, delta)` | Releases only groups whose noisy count exceeds a threshold | The threshold costs $\delta$; read it in `summarize` |
| The `threshold` column | The smallest value such that a one-person group clears it with probability at most $\delta$, over every group that person could create | Grows with the scale, with the privacy unit and with a smaller $\delta$ |
| `with_keys(keys)` | Joins a public key list; every key is released, zeros as noise | The list is fixed without the data; a released key set counts as public |
| `join(keys, how="right")` | The join that `with_keys` stands for | Only left or right joins release a key set |
| `Margin(by, invariant="keys")` | The keys of that grouping are released in the clear | Presence and absence of a key are unprotected; use for a design the public knows |
| `Margin(by, invariant="lengths")` | The group sizes are public, keys included | Gives up more than keys; filter before the context, not inside the query |
| `max_groups`, `max_length` | Public bounds on the number of groups and the rows per group | `max_groups` lowers the threshold; a sum needs `max_length`; set both from the design |
| One scale for every group | A person's rows can all sit in one group | Finer groups keep the noise and lose relative accuracy |

## 4.9. References

- OpenDP documentation: "Grouping" in the tabular-data tutorial, and "Aggregation" and "Polars" in the API user guide.
- Korolova, Kenthapadi, Mishra and Ntoulas, "Releasing Search Queries and Clicks Privately" (2009): thresholded noisy counts as a way to release keys privately.
- Desfontaines, Voss, Gipson and Mandayam, "Differentially Private Partition Selection" (2022): the threshold, its $\delta$, and why one person's groups add up.
- The full edition's chapters on sensitivity in the full edition, whose stability-based release is the thresholded count, and on post-processing and composition in the full edition, which is why a released key set can be reused for free.

# 5. Identifier truncation and bounds

A person in the survey owns several rows, one per quarter answered. The chapters so far used the fixed unit of the data and the context, up to 36 rows per person, so every noise scale grew 36-fold. But hardly anyone appears that often. This chapter uses the column that says which rows belong together, `PIDENT`, and lets each query decide how many of a person's rows it keeps. The guarantee still protects a person's whole record, but the noise fits what the query lets one person contribute.

The chapter follows the documentation's page on identifier truncation and its user guide on truncation and bounds. At the end, you know how to bound a person's rows and groups in a query, and which bounds the library enforces and which it takes on trust.

The script opendp_identifier_truncation.py runs every query on the Labour Force Survey (LFS) sample with OpenDP 0.16.0. The scales, error bars, thresholds, error messages and facts about the data that it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_identifier_truncation.py
```

## 5.1. A privacy unit with an identifier

Until now the privacy unit was a number of rows. With an identifier, it is a value of a column. Two datasets are neighbours, written $D \sim D'$, when one is the other with every row of one `PIDENT` value added or removed. That is **user-level privacy**, which hides a person's whole record and not only one row of it. The full edition's chapter on user-level privacy defines and analyses it. OpenDP writes the unit as a metric on identifiers and a distance of one.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1, identifier="PIDENT"))  # (SymmetricIdDistance {
 ↪  identifier: col("PIDENT") }, 1)
```

The argument `contributions=1` now counts identifiers, not rows: a person appears under at most one value of `PIDENT`, a public fact about the survey. The unit does not say how many rows a person owns. The old 36 had to hold for the whole dataset before any query ran. Now each query states its own bound as a truncation that the library applies, so the bound holds because the query makes it hold.

The script first looks at what an identifier owns in this file. This look is harmless only because the data are public. On real data, reason from the design of the survey, because a bound read off the data leaks it.

```
identifiers: 1,682,306  rows: 3,811,932
rows per identifier: p50=2 p90=4 p99=7 max=15
identifiers with more than 10 rows: 195
quarters per identifier: max 4  rows per identifier and quarter: max 8
```

So the worst case of 36 rows is far from this file. An identifier spans at most four quarters, but it can own up to eight rows in one quarter.

The context is the one that the documentation builds, with the public bound on the number of rows of the data and the context. Its budget, split evenly over four queries, is a privacy budget $\varepsilon$ of 1 and a slack $\delta$ of $10^{-8}$. The slack is the small probability with which the guarantee can fail. The table per quarter needs the slack, because the library selects its quarters from the data.

```
def make_context(frame: pl.LazyFrame, privacy_unit: tuple[dp.Metric, Any]) ->
↪  dp.Context:
    """The documentation's context: approximate differential privacy, split evenly over
    ↪  four queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=privacy_unit,  # dp.unit_of(contributions=1, identifier="PIDENT")
↪  below
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],
    )
```

```
budget per query: epsilon=0.25 delta=2.5e-09
```

The library refuses a query on this context that does not truncate, because it cannot bound how far one identifier's rows move a statistic. In version 0.16.0, the error message is not helpful.

```
try:
    context.query().filter(pl.col("HWUSUAL") !=
↪  99).select(hours().dp.mean(HOURS)).summarize()
    untruncated = "accepted"
except dp.OpenDPException as err:
    untruncated = reason(err)
```

```
mean without truncation: A step in your query is not recognized at this time: "DF
↪  ["PIDENT", "YEAR", "QUARTER", "SEX", ...]; PROJECT */8 COLUMNS". ""If you would
↪  like to see this supported, please file an issue.
```

The message blames the scan of the data, not the missing truncation. If you see it on a context with an identifier, first make sure that every statistic comes after a truncation.

## 5.2. Truncating each person's rows

The documentation's first query is the mean of `HWUSUAL`, the hours usually worked, without the code 99 for "not applicable". The mean needs public bounds on the hours, 0 to 80, and a bound on the rows per person. `truncate_per_group(10)` gives that bound: it keeps at most ten random rows per identifier before the aggregation.

```
mean_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)  # 99 means "not applicable"
    .truncate_per_group(ROWS_PER_IDENTIFIER)  # at most 10 rows per identifier, chosen
 ↪  at random
    .select(hours().dp.mean(HOURS))
)
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
mean, 10 rows per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0 95%
 ↪  accuracy=19,173.2
mean, 10 rows per identifier: HWUSUAL Length, Integer Laplace, scale=80.0 95%
 ↪  accuracy=240.2
```

A mean is released as a sum and a count, as in Essential statistics. The two split the query's share, so each gets $\varepsilon/8$. One person now moves the count by at most ten rows, so the count's sensitivity is 10, and its scale is 10 over $\varepsilon/8$. Each row adds at most 80 hours to the sum, so the sum's sensitivity and scale are 80 times larger. Truncation and clipping are the two ways to make a sensitivity finite, and the full edition's chapter on sensitivity derives what each does to the answer.

Under the unit of 36 contributions, the same mean gets scales of 23,040 and 288. The price of the truncation is bias, because the mean leaves out the rows after the tenth. On this file the price is nothing: the truncation drops 20 rows of about 1.5 million.

```
rows with HWUSUAL not 99: 1,496,192 by 809,710 identifiers
identifiers with more than 10 such rows: 18  rows dropped: 20
unit of 36 contributions, no identifier: scales 23,040.0, 288.0
```

The `keep` argument says which rows remain. The default, `"sample"`, keeps a random ten: its plan numbers each identifier's rows in a shuffled order and keeps the numbers below ten. With `"first"` or `"last"` the numbering follows the file's order, which is cheaper but biased when the order means something. A `SortBy` keeps the rows that sort first by an expression, such as `ILOSTAT`, the employment status. No choice changes the scales, because the bound of ten rows holds whichever rows remain.

```
truncate_per_group(10) plan: FILTER
 ↪  [(0.int_range([len().cast(Int64)]).shuffle().over([col("PIDENT")])) < (10)]
keep=first: scales 6,400.0, 80.0
keep=last: scales 6,400.0, 80.0
```

```
keep=SortBy(ILOSTAT): scales 6,400.0, 80.0
ILOSTAT values among them: [1]
```

The last line shows that every row after the filter has `ILOSTAT` equal to 1, so here the sort changes nothing.

## 5.3. Truncating the groups a person touches

The second query is a table with one row per quarter, grouped by `QUARTER` and `YEAR`, with the number of responses and the sum of their hours. Now the noise depends on two bounds: the rows that a person adds to one group, and the number of groups that a person touches. The counts change by at most the first bound in each of at most the second number of groups. So the total sensitivity is the product of the two. The full edition derives this in its section on contribution bounding in the full edition.

So the query truncates twice. `truncate_per_group(1, by=quarterly)` keeps one row per identifier and quarter, and `truncate_num_groups(10, by=quarterly)` keeps at most ten quarters per identifier. Without the second truncation the library refuses the query, and this time the message says what is missing. (The script cuts the message's suggestion, whose column order changes from run to run.)

```
per quarter without truncate_num_groups: num_groups (None), total contributions (None),
 ↪  and per_group (Some(1)) are not sufficiently well-defined. This is likely due to a
 ↪  missing truncation earlier in the data pipeline.
```

The query below is the documentation's, except for `keep="first"` in the group truncation, for a reason given below.

```
quarter_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .truncate_per_group(1, by=QUARTERLY)  # one row per identifier and quarter
    .truncate_num_groups(GROUPS_PER_IDENTIFIER, by=QUARTERLY, keep="first")  # at most
 ↪  10 quarters each
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
quarter_summary = quarter_query.summarize(alpha=0.05)
quarters_hat = quarter_query.release().collect()
```

```
per quarter, 10 quarters per identifier: len Frame Length, Integer Laplace, scale=80.0
 ↪  95% accuracy=240.2 threshold=1,714
per quarter, 10 quarters per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0
 ↪  95% accuracy=19,173.2
```

The scales are the ones of the mean, because one row in each of ten quarters is again ten rows per person. The query gives no public keys, so the library releases a quarter only if its noisy count is above the threshold, as in Grouping. Every quarter has at least 27,228 responses, so all 36 appear.

Version 0.16.0 has two problems with `truncate_num_groups`. With the default, `keep="sample"`, the query passes `summarize` and `release` spends its share, but `collect` fails inside Polars.

```
sample_query = grouped_query(context, keep="sample")  # the same table, with the
 ↪  default keep
sample_summary = sample_query.summarize(alpha=0.05)
released = sample_query.release()  # spends the query's share of the budget
try:
    released.collect()
    sample_outcome = "collected"
except dp.OpenDPException as err:
    sample_outcome = reason(err)
```

```
keep=sample: summarize scales 80.0, 6,400.0; collect: multiple fields with name
 ↪  'QUARTER' found
shares spent after the failed collect: 4 of 4
```

The share is gone, because the budget is spent when the measurement is built. But Polars checks a plan without the data, so `explain()` before `release` shows the same error at no cost, as on this toy frame.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
toy = pl.LazyFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3],
 ↪  "HWUSUAL": [35.0, 36.0, 37.0]})
context = dp.Context.compositor(
    data=toy,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),
    privacy_loss=dp.loss_of(epsilon=1.0, delta=1e-8),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=100)],
)
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
query = context.query().truncate_per_group(1, by=quarterly).truncate_num_groups(10,
 ↪  by=quarterly)
query = query.group_by(quarterly).agg(dp.len(signed=True))
print(f"{query.summarize()['scale'].item():.1f}")  # 10.0
try:
    query.explain()
except pl.exceptions.DuplicateError as err:
    print(str(err).splitlines()[0])  # multiple fields with name 'QUARTER' found
```

The second problem is with `keep="first"` and `keep="last"`, which run. Their plan keeps the ranks of an identifier's quarters below the number that you ask for. But Polars counts ranks from one, so the truncation keeps one group fewer than asked, as the two numberings show.

```
import polars as pl

toy = pl.DataFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3]})
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
numbered = toy.with_columns(
    index=pl.int_range(pl.len()).over("PIDENT"),  # what truncate_per_group compares
 ↪  with k
    rank=pl.struct(quarterly).rank("dense").over("PIDENT"),  # what truncate_num_groups
 ↪  compares with k
)
print(numbered["index"].to_list(), numbered["rank"].to_list())  # [0, 1, 2] [1, 2, 3]
```

No identifier spans more than four quarters, so the script measures the effect at a smaller `k`. A request for four keeps at most three quarters, so the 13,948 identifiers with four quarters lose one. A request for five keeps all four, at the scale and threshold of five.

```
truncate_num_groups(10, keep=first) plan: FILTER
 ↪  [(col("QUARTER").as_struct([col("YEAR")]).rank().over([col("PIDENT")])) < (10)]
identifier-quarters after the filter: 1,241,896  identifiers with 4 quarters: 13,948
truncate_num_groups(4, keep=first): keeps at most 3 quarters, 1,227,948
 ↪  identifier-quarters; scale 32.0 threshold 657
truncate_num_groups(5, keep=first): keeps at most 4 quarters, 1,241,896
 ↪  identifier-quarters; scale 40.0 threshold 830
```

So in version 0.16.0, ask for one group more than you want to keep, and accept the noise of that larger number. The guarantee is safe either way, because the library calibrates to the number asked for. The estimate is not, because the dropped group is always the last in sort order. The practitioner edition's release workflow avoids the call for these reasons.

Put the truncations of a query next to each other, with the group truncation last before the aggregation. With a filter between them, the library calls the truncation's own expression unrecognized.

```
a filter between the two truncations: Expr is not recognized at this time: dyn int:
 ↪  0.int_range([len()]).shuffle().over([col("PIDENT"), col("QUARTER"), col("YEAR")]).
 ↪  If you would like to see this supported, please file an issue.
```

## 5.4. One row per person

For a mean, there is a better method than ten rows per person. Group by the identifier first, aggregate each person's rows to one row with any Polars expression, and take the private mean of those rows. After the group-by, no identifier owns more than one row.

```
person_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .group_by("PIDENT")  # one row per identifier: the truncation
```

```
    .agg(pl.col("HWUSUAL").mean())  # any expression may summarize the person's rows
    .select(hours().dp.mean(HOURS))
)
person_summary = person_query.summarize(alpha=0.05)
dp_person_mean = person_query.release().collect().item()
```

```
mean of one row per identifier: HWUSUAL Sum, Integer Laplace, scale=640.0 95%
 ↪  accuracy=1,917.8
mean of one row per identifier: HWUSUAL Length, Integer Laplace, scale=8.0 95%
 ↪  accuracy=24.5
```

The count's sensitivity is now 1 and the sum's is 80, so both scales are a tenth of the scales with ten rows. And no row is dropped.

But the question changes. The first query averaged over responses, so a person with four responses counted four times. This one averages each person's own average, so every person counts once, which gives 37.24 hours against 37.64. The cast to an integer then truncates each person's mean towards zero, which lowers the true value to 37.10. With an error bar of about 0.0035 hours, the choice of what to average matters far more than the noise.

```
true mean of one row per identifier: 37.0970  (before the cast to int: 37.2359; all
 ↪  rows: 37.6393)
per-identifier mean error bar at the true size: 0.0035
```

## 5.5. Bounds in the privacy unit

The unit can also hold finer knowledge, as a list of `dp.polars.Bound` objects. Each names its grouping, a bound on identifiers per group and a bound on the groups that one person touches. The documentation's example is data merged from two sources that assigned identifiers separately. A person can then appear under two identifiers, which would double the noise, but never under two in the same quarter.

```
two_sources = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=2),  # a person may appear under two identifiers
        dp.polars.Bound(by=QUARTERLY, per_group=1),  # but under one identifier in any
 ↪  quarter
    ],
    identifier="PIDENT",
)
bounded = make_context(frame, two_sources)
bounded_query = grouped_query(bounded, keep="first", k=5)  # each identifier may affect
 ↪  up to 5 quarters
bounded_summary = bounded_query.summarize(alpha=0.05)
bounded_hat = bounded_query.release().collect()
```

```
two identifiers, one per quarter, 5 quarters: len Frame Length, Integer Laplace,
 ↪  scale=80.0 95% accuracy=240.2 threshold=1,714
two identifiers, one per quarter, 5 quarters: HWUSUAL Sum, Integer Laplace,
 ↪  scale=6,400.0 95% accuracy=19,173.2
one identifier as a bound, 5 quarters: scales 40.0, 3,200.0
```

The query keeps one row per identifier and quarter, and five quarters per identifier. Two identifiers times five quarters is ten rows per person, so the scale is 80 again. The second bound tells the library that the two identifiers never meet in one quarter, so one person adds at most one row to any quarter. It does not lower the count's scale, because the ten rows can still fall in ten different quarters. With a single identifier as the bound, the same query gets half the scale.

A bound in the unit can also have `num_groups`, the number of groups that a person touches. Then the query needs no `truncate_num_groups`, and a promise of ten quarters gives the same scale as the truncation at ten. But the library enforces a truncation, while it takes a bound on trust. The script makes a promise that this file breaks, that no identifier spans more than three quarters, and the release goes through with no complaint.

```
promised = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=1),
        dp.polars.Bound(by=QUARTERLY, per_group=1, num_groups=3),  # a promise: at most
 ↪  3 quarters each
    ],
    identifier="PIDENT",
)
promise_query = (
    make_context(frame, promised)
    .query()
    .filter(pl.col("HWUSUAL") != 99)
    .truncate_per_group(1, by=QUARTERLY)  # no truncate_num_groups: the unit already
 ↪  bounds the quarters
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
promise_summary = promise_query.summarize(alpha=0.05)
promise_hat = promise_query.release().collect()
```

```
num_groups=10 promised in the unit: len Frame Length, Integer Laplace, scale=80.0 95%
 ↪  accuracy=240.2 threshold=1,714
num_groups=3 promised in the unit: len Frame Length, Integer Laplace, scale=24.0 95%
 ↪  accuracy=72.4 threshold=486
num_groups=3 promised in the unit: HWUSUAL Sum, Integer Laplace, scale=1,920.0 95%
 ↪  accuracy=5,752.3
identifiers that break the promise of 3 quarters: 13,948
```

The noise is calibrated for three quarters, but 13,948 identifiers have four. For those people the privacy loss is above the declared $\varepsilon$, and nothing in the output says so. So truncate in the query, where the library enforces the bound. Declare a bound in the unit only when no truncation can express it, as with the two identifiers above, which no filter can recognize as one person. The

same advice applies to values: clip them in the query, and do not declare their range in the input domain. Assessing utility and choosing the noise returns to bounds per group with Gaussian noise, whose scale depends on them differently.

## 5.6. The check against the truth

The data are public, so the script computes what each query would return without noise, with its own seeded random ten rows for the first mean. The quarters' true values use the first row of each identifier in each quarter, while the release keeps a random row. So the check is exact for the counts and approximate for the hours.

```
true mean, 10 rows per identifier: 37.6393  (1,496,172 rows)
true per quarter (first row per identifier and quarter): 36 quarters, rows from 27,228
 ↪  to 41,868
true 2005Q1: rows 29,561  hours 1,110,610
true 2005Q2: rows 29,406  hours 1,106,170
true 2005Q3: rows 28,364  hours 1,070,826
true 2005Q4: rows 28,681  hours 1,080,009
```

Here are the releases of one run. Each mean's error bar comes from the bars of its sum and its count, and holds whenever both hold.

```
released (one run):
  mean, 10 rows per identifier: 37.6366  (true 37.6393, within 0.0189 with probability
 ↪  at least 0.9)
  per quarter: 36 quarters released; rows within their 95% accuracy: 36 of 36; hours:
 ↪  35 of 36
  2005Q1: rows 29,644  hours 1,105,518
  2005Q2: rows 29,460  hours 1,098,965
  2005Q3: rows 28,244  hours 1,063,846
  2005Q4: rows 28,645  hours 1,087,028
  mean of one row per identifier: 37.0970  (true 37.0970, within 0.0035 with
 ↪  probability at least 0.9)
  two bounds, 5 quarters: 36 quarters released; rows within four times their 95%
 ↪  accuracy: 36 of 36
  num_groups=3 promised: 36 quarters released; rows within four times their 95%
 ↪  accuracy: 36 of 36
```

The means are within their bars, and so are the quarters' counts and all but one of their sums. A bar of 95% predicts about that many misses for 36 values. The test checks the two means within three times their bars and the 36 quarters within four times theirs, so a correct run almost never fails it.

## 5.7. Summary

Table 5.1.: The calls of identifier truncation and bounds, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.unit_of(contributions=1, identifier="PIDENT")` | The privacy unit is every row of one identifier | `contributions` now counts identifiers; the rows per person are bounded by the query |
| A query with no truncation | Refused, with a message that blames the data scan | On a context with an identifier, every statistic sits behind a truncation |
| `truncate_per_group(k, keep=...)` | Keeps at most `k` rows per identifier, and per group with `by` | `"sample"` is unbiased; `"first"`, `"last"` and `SortBy` are cheaper or deliberate, and biased when order matters |
| `truncate_num_groups(k, by=...)` | Keeps at most `k` groups per identifier | In 0.16.0, `"sample"` fails at `collect` after the budget is spent, and `"first"` keeps `k - 1` groups; call `explain()` before `release()` |
| Truncations in a query | Must sit next to each other, the group truncation last | A filter between them makes the query unrecognized |
| `group_by("PIDENT").agg(...)` before a statistic | One row per person, so the sensitivity is that of one row | The best choice for a mean, if the mean over people is the one you want |
| `dp.polars.Bound(by, per_group, num_groups)` in `unit_of` | Finer bounds on identifiers per group and groups per person | A promise the library trusts; a person who breaks it loses more than $\varepsilon$ |
| Truncate or declare | The query enforces a truncation; the unit declares a bound | Truncate, and declare only what no truncation can express |

## 5.8. References

- OpenDP documentation: "Identifier Truncation" in the tabular-data tutorial, and "Identifier Truncation and Bounds" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): sensitivity and the Laplace mechanism.
- Amin, Kulesza, Muñoz Medina and Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (2019): why truncating a person's rows trades bias for variance.
- Wilson, Zhang, Lam, Desfontaines, Simmons-Marengo and Gipson, "Differentially Private SQL with Bounded User Contribution" (2020): bounding rows per group and groups per user in grouped queries.
- The full edition's chapters on sensitivity in the full edition and user-level privacy in the full edition derive the results this chapter states.

# 6. Preparing microdata

Survey data rarely arrive in the form that a statistic needs. Hours of work come as floats, with 99 for "not applicable" and a null for a missing answer. The date of a response is in two integer columns. So before you release anything, you cast, fill, clip, filter and recode. This chapter is about where that work happens. OpenDP reads the whole lazy query before it runs, preparation included, and accounts for every step. So the preparation belongs inside the query, where the guarantee covers it.

The chapter follows the documentation's tutorial on preparing microdata, its page on data types and its pages on expressions. You see which Polars contexts and expressions OpenDP allows, what it changes about them, and how it tracks nulls, undefined floats and bounds. The chapter ends with a worked preparation of the Labour Force Survey (LFS), for the mean difference between actual and usual hours of work.

The script opendp_preparing_microdata.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. The noise scales, error bars, thresholds, error messages and true values that it prints are the same on every run. The released counts and mean are not, because OpenDP's noise cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_preparing_microdata.py
```

## 6.1. Prepare inside the query

A context on a Polars LazyFrame reads only the frame's schema, which the library treats as public, and touches the data only at a release. OpenDP parses each query, a lazy plan, into a chain of transformations before anything runs, as Transformations explains. For each step, the library finds how much one person's rows can change the output. So a filter or a fill in the middle of the plan is part of the accounting.

Preparation before the context is different, because the library then sees only the prepared frame, and its guarantee is about that frame. A fixed function of each row, such as a renamed column or a date parsed with a known format, leaves every person with the same rows. Then the guarantee still holds, but the claim is yours, because the library did not check it. A step that reads the data is worse. An imputation by the column's own mean, a bound at its maximum or a filter at a quantile carries information about everyone into the release, outside the accounting. Before the context, use only public facts, such as the schema, the code book and the published bound on the sample's size.

The context has the privacy unit of the data and the context, up to 36 rows per person, and that chapter's bound on the rows. The budget is approximate differential privacy, with a privacy budget $\varepsilon$ of 1 and a slack $\delta$ of $10^{-7}$, split evenly over four queries.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The documentation's context: 36 rows per person, (epsilon = 1, delta = 1e-7)
     ↪  split evenly over 4 queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],  # public: a bound on the
 ↪  number of rows
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(MaxDivergence), (1.0, 1e-07))
budget per query: epsilon=0.25 delta=2.5e-08
schema: PIDENT=Int64  YEAR=Int64  QUARTER=Int64  SEX=Int64  AGE=Float64
 ↪  HWUSUAL=Float64  HWACTUAL=Float64  ILOSTAT=Int64
```

The schema line is all that the context knows about the data at this point, and it assumes that any column can hold nulls.

## 6.2. Columns, filters and groups

Polars applies an expression in one of five contexts, and OpenDP puts them in two groups. Expressions in `.select` and `.agg` can change the number and the order of rows, because an aggregation does exactly that. Expressions in `.with_columns`, `.filter` and `.group_by` must be **row-by-row**: a function applied to each row on its own. The reason is the privacy unit. A row-by-row expression keeps each person's rows their own, so 36 rows in gives 36 rows out. An expression that sorted or shifted the column would mix people's rows, and the library could not say what one person changes.

The documentation's first example adds a column with `.with_columns` and groups on it. The row-by-row expression `cut` puts the usual hours into bands at fixed breaks, and replaces `HWUSUAL` with its bin. The labels of the bins depend only on the breaks, so they are public and can be the keys.

```
def binned_counts(context: dp.Context) -> LazyFrameQuery:
    """How many responses fall in each band of usual hours: a row-by-row `cut`, then a
     ↪  count per bin."""
    labels = pl.Series("HWUSUAL", cut_labels(BREAKS, left_closed=True),
 ↪  dtype=pl.Categorical)
    return (
        lazy_query(context)
        .with_columns(pl.col("HWUSUAL").cut(breaks=BREAKS, left_closed=True))  #
 ↪  shadows HWUSUAL with its bin
        .group_by("HWUSUAL")
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame([labels]))  # the bin labels are public, so no
 ↪  threshold is needed
```

```
    )
```

```
binned counts: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
without keys: len Frame Length, Integer Laplace, scale=144.00 threshold=2,973
true bins: null=20,819  [-inf, 0)=0  [0, 20)=89,495  [20, 40)=908,061  [40, 60)=406,947
 ↪  [60, 80)=78,469  [80, 98)=13,220  [98, inf)=2,294,921
```

The scale is the 36 rows that one person can add to a bin, divided by the query's share of the budget. With the keys, every bin is released, including the empty one below zero, whose signed count can be negative. Without the keys, the library must learn from the data which bins exist, so the query has a threshold, as in Grouping. The rows with a null `HWUSUAL` fall in a null bin that is not among the keys, so no release has them. Also, a margin that you declared about `HWUSUAL` applies to the original column, and the new column has none.

A `.filter` removes rows with a boolean row-by-row expression, such as the documentation's filter on the code 99 before a sum. A filter keeps most of what the margins say. But it discards any invariant about the group keys and sizes, which were facts about the data before the filter. A filter also tells the library nothing about what remains: a column filtered to non-null values is still nullable in its domain.

```
filtered sum: HWUSUAL Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
```

The last context of the tutorial is a `.group_by` and `.agg` *before* the private aggregation. In the documentation's version of sample and aggregate, the rows go into 1,000 groups by `PIDENT`, and the query releases the mean of each group's minimum usual hours. Any expression can go in that inner `.agg`, but the price is noise. The script prints the scales of the mean on the context above, and with a `Bound` from Identifier truncation and bounds that puts each person's rows in one group.

```
sample and aggregate: HWUSUAL Sum, Integer Laplace, scale=17,280.00
sample and aggregate: HWUSUAL Length, Integer Laplace, scale=576.00
one group per person: HWUSUAL Sum, Integer Laplace, scale=480.00
one group per person: HWUSUAL Length, Integer Laplace, scale=16.00
```

The first context does not know that a person's rows share a `PIDENT`, so it allows for 36 changed groups. Even with the bound, the inner aggregation doubles the noise, because a changed group counts as one value removed and another added.

## 6.3. The types OpenDP tracks

OpenDP assumes that every column can hold nulls until an expression proves otherwise. It also keeps **domain descriptors** for each column: facts about its values that later aggregations need. For a number it tracks lower and upper bounds, which a `.clip` sets and some later steps drop. The supported integers are the signed ones and the unsigned ones of 32 and 64 bits, so a `UInt8` column is refused. A float column also records whether its values can be NaN, the float for an

undefined number, so a float sum needs both `.fill_null` and `.fill_nan`. Booleans and strings have no descriptors.

A categorical column stores an integer code per value, with a table of labels, and two things about that table depend on the data. An expression that could add or remove a category, such as `.fill_null`, makes Polars rewrite the codes and warn, a side effect that depends on the data. And the codes follow the order in which the values first appear, so a group-by on them would show the order of the rows. So the library refuses both. An `Enum` column fixes its categories in the schema, so neither problem occurs, and neither does it with `cut`, whose encoding depends only on the breaks.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
status = ["working", "absent", "other"] * 100
unit, loss = dp.unit_of(contributions=1), dp.loss_of(epsilon=1.0, delta=1e-7)


def context_for(column: pl.Series) -> dp.Context:
    return dp.Context.compositor(data=pl.LazyFrame([column]), privacy_unit=unit,
     ↪  privacy_loss=loss, split_evenly_over=1)


categorical = context_for(pl.Series("status", status, dtype=pl.Categorical))
try:
    categorical.query().group_by("status").agg(dp.len(signed=True)).summarize()
except dp.OpenDPException as err:
    print(err.message)  # Categories are data-dependent, which may reveal sensitive
     ↪  record ordering. Cast status to string before grouping.
enum = context_for(pl.Series("status", status, dtype=pl.Enum(["working", "absent",
 ↪  "other"])))
print(f"{enum.query().group_by('status').agg(dp.len(signed=True)).summarize()['scale']
 ↪  .item():.4f}")  # 1.0000
```

The error names the fix, a cast to string, and the enum query goes through. Dates, datetimes and times are also supported, with one time zone per datetime column.

## 6.4. Casting, filling and clipping

The documentation's reason to cast is the float sum. OpenDP accounts for the rounding error of float addition, and on many rows with wide bounds that term shows in the noise scale. The script sums the usual hours both ways.

```
def hours_sum(context: dp.Context, as_integers: bool) -> pl.DataFrame:
    """The noise scale of a sum of usual hours, on the float column or after casting it
     ↪  to integers."""
    usual: Any = pl.col("HWUSUAL")  # typed Any: OpenDP adds the `.dp` namespace to
 ↪  expressions at run time
```

```
    column = usual.cast(int) if as_integers else usual
    return lazy_query(context).select(column.dp.sum(HOUR_BOUNDS)).summarize(alpha=0.05)
```

```
float sum: HWUSUAL Sum, Float Laplace, scale=14,405.18 95% accuracy=43,154.1
integer sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
```

On integers the scale is exactly 36 times the upper bound of 100 over the share of the budget, and on floats it is a little more. A cast has one more effect. In Polars, a value that cannot be converted raises an error, which depends on the data, so OpenDP makes a failed cast return a null. So every cast column is nullable, and a statistic on it needs a `.fill_null` first.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": ["40", "35", "n/a"] * 100}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=1000)],
)
hours = pl.col("hours").cast(int)  # OpenDP turns "n/a" into a null instead of raising
 ↪  an error
try:
    context.query().select(hours.clip(0, 80).sum().dp.noise()).summarize()
except dp.OpenDPException as err:
    print(err.message)  # input data (hours) might contain nulls. Preprocess your data
     ↪  with `.fill_null`.
query = context.query().select(hours.fill_null(0).clip(0, 80).sum().dp.noise())
print(query.summarize()["scale"].item())  # 80.0
```

The `.dp.sum` of the earlier chapters adds the imputation and the clip for you. The snippet instead prepares the column by hand and adds noise to a plain `.sum`, to show what the library checks. The sum needs the bounds that a `.clip` sets, because without bounds one person could move the sum by any amount. But `.fill_null` and `.fill_nan` drop the bounds, so fill first, then clip. The script runs the same preparation on the survey in `.with_columns`, and tries the wrong orders.

```
def prepared_sum(context: dp.Context, steps: pl.Expr) -> pl.DataFrame:
    """A sum with noise on a column prepared in with_columns: OpenDP checks the steps
     ↪  left it non-null and bounded."""
    total: Any = pl.col("HWUSUAL").sum()
    return lazy_query(context).with_columns(steps).select(total.dp.noise()).summarize()
```

```
prepared sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00
cast without fill_null: input data (HWUSUAL) might contain nulls. Preprocess your data
 ↪  with `.fill_null`.
clip before fill_null: input domain must consist of bounded data. Either specify bounds
 ↪  in the input domain or use make_clamp.
```

```
fill_null before fill_nan: input data (HWUSUAL) might contain nans. Preprocess your
 ↪  data with `.fill_nan`.
float prepared sum: HWUSUAL Sum, Float Laplace, scale=14,405.18
```

The fill value can also come from another column, such as the actual hours for missing usual hours. Then the column and the fill must both be row-by-row, so that their rows line up, and the fill must have no nulls.

```
filled from a column: HWUSUAL Sum, Integer Laplace, scale=14,400.00
filled from a nullable column: fill expression must not be nullable
```

The expressions `.drop_nulls` and `.drop_nans` remove rows and change the column's length, so the library forbids them in a row-by-row context.

```
drop_nulls in with_columns: drop_nulls is not allowed in a row-by-row context
```

Recoding has two forms. The expression `.replace` swaps listed values and keeps the type. The expression `.replace_strict` maps every value and can change the type, so an unmapped value takes the `default`. The documentation says that OpenDP sets a missing `default` to null, but version 0.16.0 does not. On a toy column of three codes with a mapping for two, `summarize` accepts the query, but `release` fails. So give `replace_strict` a `default` yourself.

```
replace_strict without default: summarize scale=2.0; release:
 ↪  InvalidOperation(ErrString("incomplete mapping specified for
 ↪  `replace_strict`\n\nHint: Pass a `default` value to set unmapped values."))
replace_strict with default: released keys one other two
```

## 6.5. Predicates and arithmetic

OpenDP accepts the comparison, conjunction and arithmetic operators of Polars, as row-by-row expressions. Both sides of a binary operator must be row-by-row even inside `.select` or `.agg`, because a length mismatch would raise an error that depends on the data. Tests such as `is_null` and `is_nan` are row-by-row too, and they build filters and grouping columns. The documentation counts the respondents over retirement age with a group-by on a comparison.

```
def over_retirement_age(context: dp.Context) -> LazyFrameQuery:
    """How many responses come from people over 64: a boolean expression as the
     ↪  grouping column."""
    return (
        lazy_query(context)
        .group_by(pl.col("AGE") > RETIREMENT_AGE)
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"AGE": [True, False]}))  # a null age is in neither
 ↪  group and is dropped
    )
```

```
over 64: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true over 64: true=1,190,845  false=2,620,832  null=255
```

A comparison with a null is null, so the 255 rows without an age form a third group that the keys leave out. The grouped form answers both halves of the question for the price of one.

Version 0.16.0 does not support two operators that you might want. It refuses `is_in`, membership in a list, so write the test as a disjunction of equalities. The script counts the employed in this way.

```
is_in: IsIn { nulls_equal: false } is not currently supported
employed: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The other is the absolute value, in Section 6.8.

## 6.6. Strings and dates

Of the string expressions, OpenDP supports only parsing into a temporal type, with `.str.strptime` and its variants `to_date`, `to_datetime` and `to_time`. The `format` argument is mandatory, because a format inferred from the data could change with one person's rows. Nanosecond datetimes are refused, because the parser can panic on some inputs, an error that depends on the data.

```
to_date without format: format must be specified; otherwise Polars will attempt to
 ↪  infer the format from the data, resulting in an unstable transformation
nanosecond datetime: Nanoseconds are not currently supported due to potential panics
 ↪  when parsing inputs. Please open an issue on the OpenDP repository if you would
 ↪  find this functionality useful. Otherwise, consider parsing into micro- or
 ↪  millisecond datetimes instead.
```

The survey has no date column, only `YEAR` and `QUARTER`. The documentation builds one with `concat_str` before the context, because the library refuses concatenation inside a query.

```
concat_str: Expr is not recognized at this time: ConcatHorizontal { delimiter: "-",
 ↪  ignore_nulls: false }. If you would like to see this supported, please file an
 ↪  issue.
```

But arithmetic is supported, so you can build the date inside the query. The year times 100 plus three times the quarter gives an integer such as 200503. As a string, it parses with the format `%Y%m` into the first day of the quarter's last month. The `.dt` expressions then read the components of the date, from the year to the day.

```
def quarter_end() -> pl.Expr:
    """The last month of the quarter as a date, built from YEAR and QUARTER and parsed
     ↪  with a fixed format."""
    year_month = pl.col("YEAR") * 100 + pl.col("QUARTER") * 3  # 2005 and 1 become
 ↪  200503
    return year_month.cast(str).str.to_date(format="%Y%m")


def responses_per_month(context: dp.Context) -> LazyFrameQuery:
    """How many responses carry each quarter-end month, read off the parsed date."""
    return (
        lazy_query(context)
        .with_columns(DATE=quarter_end())
        .group_by(pl.col("DATE").dt.month().alias("MONTH"))
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"MONTH": pl.Series(QUARTER_END_MONTHS,
 ↪  dtype=pl.Int8)}))
    )
```

```
first dates: 2005 Q1 -> 2005-03-01  2005 Q2 -> 2005-06-01
per month: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true per month: 3=953,695  6=958,396  9=933,257  12=966,584
```

Every quarter ends in one of four public months, so they are the keys, with no threshold. The keys have the type `Int8`, because they must match the type that `.dt.month` returns.

## 6.7. A worked preparation: hours lost to absence

The statistic is the mean difference between the hours that a person with a job actually worked in the survey week and the hours that they usually work. A negative mean means that holidays, illness and short weeks outweigh overtime. Each step of the preparation is a decision about the data, inside the query. First the query keeps `ILOSTAT` equal to 1, working for pay or profit. It casts both hour columns to integers, for the integer noise of Section 6.4. It fills the usual hours with 0. Then it keeps the rows whose usual hours are known, without 0 for hours that vary and 99 for not applicable.

The fill comes before the filter because a filter proves nothing to the library (Section 6.2). After the filter, no null usual hours remain, but the column is still nullable in its domain. And the next step uses it as a fill value, which must not be nullable. That step gives a row without actual hours its usual hours, so its difference is zero, the least informative choice. Last, the query clips the difference to public bounds of plus or minus 40 hours, because a sum needs bounds. The full range of plus or minus 80 would double the noise.

```
def prepare[Frame: (LazyFrameQuery, pl.LazyFrame)](frame: Frame) -> Frame:
    """The rows of people who work, with their usual hours known, and the hours they
     ↪  lost that week, clipped."""
    low, high = LOST_BOUNDS
    return (
```

```
        frame.filter(pl.col("ILOSTAT") == 1)  # working for pay or profit
        .with_columns(pl.col("HWUSUAL").cast(int).fill_null(0),
↪  pl.col("HWACTUAL").cast(int))
        .filter((pl.col("HWUSUAL") > 0) & (pl.col("HWUSUAL") < 99))  # 0 is "varies",
↪  99 "not applicable"
        .with_columns(pl.col("HWACTUAL").fill_null(pl.col("HWUSUAL")))  # unknown
↪  actual hours: assume the usual
        .with_columns(LOST=(pl.col("HWACTUAL") - pl.col("HWUSUAL")).clip(low, high))
    )
```

The function takes a query or a plain LazyFrame, so the release and the check use one preparation. The query ends in `pl.col("LOST").dp.mean(LOST_BOUNDS)`, and `summarize` shows what the mean costs.

```
hours lost: LOST Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
hours lost: LOST Length, Integer Laplace, scale=288.00 95% accuracy=863.3
hours lost, bounds (-80, 80): LOST Sum, Integer Laplace, scale=23,040.00 95%
↪  accuracy=69,022.2
```

A mean is a noisy sum over a noisy count, which split the query's share of the budget. So the sum's scale is 36 rows times the bound of 40, over an eighth of the budget. With bounds of plus or minus 80, the sum's scale doubles, so the clip is a trade: a narrower bound gives less noise and more bias. The full edition's chapter on sensitivity derives how the bounds of a sum set its sensitivity.

The survey is public, so the script measures the bias, which on real data you can do only with a public or historical copy.

```
true prepared rows: 1,490,113  true sum of hours lost: -9,289,648  true mean: -6.2342
rows the clip changes: 42,830 (2.874%)  mean without the clip: -6.5319
mean error bar at 95%: 0.0268
```

The clip changes under three percent of the rows. But these rows move the true mean from -6.53 hours to -6.23, a bias of about a third of an hour. The noise is far smaller. The script turns the two accuracies into an error bar for the mean: the sum's bar plus the mean times the count's bar, all over the count. It comes to 0.027 hours. So on this many rows, the clip limits the result, not the noise.

```
released (one run):
  bins: [-inf, 0)=-129  [0, 20)=89,491  [20, 40)=907,895  [40, 60)=406,594  [60,
↪  80)=78,436  [80, 98)=13,452  [98, inf)=2,294,708
  over 64: true=1,190,894  false=2,620,771
  per month: 3=953,684  6=958,313  9=933,077  12=966,807
  hours lost: mean -6.2147  (true -6.2342, within 0.0268 with probability 0.95)
budget spent: 4 of 4 queries
```

Every count is within its bar of about 432, and the empty bin is negative, as a signed count can be. The mean is within its bar of the clipped truth. The four releases used the four shares. Assessing utility and choosing the noise takes the error bars further.

## 6.8. What a refused expression looks like

An expression that OpenDP cannot parse raises an `OpenDPException`, with a variant and a message, when you build the query and before it reads any data. The absolute value of the difference in hours is an example.

```
abs: Expr is not recognized at this time: [(col("HWACTUAL")) - (col("HWUSUAL"))].abs().
↪  If you would like to see this supported, please file an issue.
```

The messages on this page are of two kinds. "Not recognized at this time" means that nobody has implemented the expression yet, as with `concat_str` and `abs`. The other messages state a rule, such as a mandatory format or a fill that must not be nullable, and each names its fix inside the query. For an expression without an implementation, rewrite it with supported operators if you can, as `is_in` became a disjunction. If not, apply it before the context, but only if it is a fixed function of each row. Then state that step with the release, because the library did not check it.

## 6.9. Summary

Table 6.1.: The contexts and expressions of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| Preparation inside the query | OpenDP parses every step and accounts for it | Prepare before the context only with fixed row-by-row steps and public facts, and say so |
| `.with_columns`, `.filter`, `.group_by` | Row-by-row contexts: each person's rows stay their own | A new column has no margins; a filter drops key and length invariants and proves nothing about what remains |
| `.select`, `.agg` | May change the number and order of rows | Binary operators and fills inside them still need row-by-row inputs |
| `.cut(breaks)` | Bins a number into categories with a data-independent encoding | Pass the labels as keys to avoid a threshold |
| Domain descriptors | Nullable, may be NaN, bounds | Every column starts nullable; floats need `.fill_null` and `.fill_nan`; `.clip` sets bounds |
| `.cast` | A failed cast returns null instead of raising | Follow it with `.fill_null`; cast to integers for a cheaper sum |
| `.fill_null`, `.fill_nan` | Impute; a column fill must be row-by-row and non-nullable | Fill first, then clip, because filling drops the bounds |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `pl.Categorical` vs `pl.Enum` | Categories from the data vs fixed in the schema | Group by enums; cast categoricals to string |
| `.replace_strict(mapping)` | Recodes every value | Pass a `default`: in 0.16.0 a missing one fails at release |
| Comparisons, `&`, `\|`, arithmetic | Row-by-row predicates and new columns | `is_in` and `abs` are not supported in 0.16.0 |
| `.str.to_date(format=...)` | Parses a string into a date | The format is mandatory; nanosecond datetimes are refused |
| `.dt.month()` and the other components | Reads a part of a date or time | Keys for a component must match its type (`Int8` for a month) |
| `OpenDPException` | Raised when the plan is parsed, before any data are read | "Not recognized" is unimplemented; the other messages state a rule and its fix |

## 6.10. References


- OpenDP documentation: "Preparing Microdata" in the tabular data tutorial; "Data Types" and the "Expressions" pages "Boolean", "Columns", "Manipulation", "Operators", "String" and "Temporal" in the Polars user guide.
- Nissim, Raskhodnikova and Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (2007): the sample and aggregate framework behind the stable `group_by/agg`.
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the stable transformations that each expression is parsed into.
- The full edition's chapter on sensitivity in the full edition derives how the bounds of a clipped column fix the noise of its sum and mean.

# 7. Assessing utility and choosing the noise

Every query in the earlier chapters printed a scale and an accuracy before its release. This chapter is about how to read those two numbers and change them. An **accuracy** is the half-width of an error bar: with a stated probability, the released value is within that distance of the value that the noise was added to. OpenDP converts a noise scale to an accuracy and back, so you can find what a target error bar costs before you spend anything.

The second half of the chapter is about which noise to add. Under pure differential privacy (DP) the context adds Laplace noise, and under zero-concentrated differential privacy (zCDP) it adds Gaussian noise. One argument of `dp.loss_of` selects between them. The choice matters most when one person's records fall in many groups. Then Gaussian noise is calibrated to a sensitivity that grows with the square root of the number of groups, not with the number itself. On the survey, this cuts the noise on a quarterly count six-fold, but only under zCDP.

The script opendp_utility_and_noise.py runs every step on OpenDP 0.16.0 against the Labour Force Survey (LFS) sample of the earlier chapters. The conversions, scales, accuracies, thresholds and budgets that it prints are the same on every run. The released counts and the means of a small clamping experiment are not, because OpenDP's noise cannot be seeded, so this page shows them in blocks marked as one run.

```
uv run python code/part02-tabular-data/opendp_utility_and_noise.py
```

## 7.1. What an accuracy statement means

The documentation gives a template to read the accuracy functions. At a given scale, the released value is within the accuracy of the true value at a significance level $\alpha$, that is, with confidence $(1 - \alpha) \times 100\%$. Every accuracy in this book has $\alpha = 0.05$, so about one release in twenty is outside its error bar. A smaller $\alpha$ gives a wider bar, as Essential statistics showed at $\alpha = 0.01$.

The module `opendp.accuracy` has eight conversion functions, which the prelude gives under `dp`. Four convert a Laplace or Gaussian scale to an accuracy and back, and the other four, with `discrete_` in their names, do the same for integer noise. The documentation's example asks for the Gaussian scale at which a release is within 2 of its input with 95% confidence.

```
import opendp.prelude as dp

print(dp.accuracy_to_gaussian_scale(accuracy=2.0, alpha=0.05))  # 1.020426913849308
print(dp.gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 352.7935172172097
print(dp.discrete_gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 354.0
print(dp.laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 539.2318092397184
print(dp.discrete_laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  #
 ↪  539.7311147961669
```

The continuous functions are closed forms. For Laplace noise, the accuracy is the scale times $\ln(1/\alpha)$. For Gaussian noise, it is the scale times the two-sided normal quantile, about 1.96 at the usual level. The discrete functions sum the probabilities of the integer noise, so a discrete Gaussian accuracy is a whole number. Use the discrete ones for counts, because OpenDP adds integer noise to an integer, and the continuous ones for sums and means of floats. At a scale of 180 the difference is small, and `summarize` always uses the correct one.

The error bar is about the noise and nothing else, and this is the most frequent mistake in how people read one. The accuracy functions say how far the release can be from the input to the noise mechanism: the statistic of the data after every preparation step. A clip into public bounds, an imputed constant, a truncation of a person's rows and a resize all change that input. The error bar says nothing about the gap between the input and the quantity that you want. Those gaps are biases, which no number of repeated releases removes. The practitioner edition's release workflow in the practitioner edition measures one, where truncation and clipping keep about half of the revenue.

## 7.2. A target error bar on the survey

Suppose you want the count of survey responses within 100 of the truth with 95% confidence. Start from the count that Essential statistics released, with $\varepsilon = 1$ split evenly over five queries. The excerpt summarizes it again, and then asks the discrete Laplace conversion for the scale that the target needs.

```
laplace = laplace_context(frame, EPSILON)
count_laplace = laplace.query().select(dp.len(signed=True))
laplace_summary = count_laplace.summarize(alpha=ALPHA)
scale_needed = float(dp.accuracy_to_discrete_laplacian_scale(accuracy=TARGET,
 ↪  alpha=ALPHA))
epsilon_needed = CONTRIBUTIONS / scale_needed  # the Laplace scale is the sensitivity
 ↪  over epsilon
```

```
laplace count: Integer Laplace scale=180 95% accuracy=539.731
budget per query: epsilon=0.2000
error bar relative to the true count: 0.014%
discrete laplace scale for accuracy 100: 33.2152
epsilon that scale needs at sensitivity 36: 1.0838  (5.42 times the share)
```

The scale of 180 is the sensitivity over the share of 0.2, with a sensitivity of 36 from the privacy unit. The target needs a scale of 33.2, which costs more than the whole budget on this one query. So a bar of 100 is not available at this budget. But a bar of 0.014% of the true count is already small for a headline figure.

The conversion also gives the exchange rate. The Laplace scale is the sensitivity over $\varepsilon$, and the accuracy is proportional to the scale, so half the error bar costs twice the budget. At $\varepsilon = 2$ the same query gets half the scale.

```
laplace count at epsilon 2: Integer Laplace scale=90 95% accuracy=270.115
```

The bar is not exactly half, because the discrete conversion rounds the integer noise, but the rule of thumb holds. The full edition's chapter on the Laplace mechanism derives the scale and the tail bound behind the conversion.

## 7.3. Laplace under pure DP, Gaussian under zCDP

OpenDP chooses the noise from the kind of budget. With `dp.loss_of(epsilon=...)`, the guarantee is pure DP and the context adds Laplace noise. With `dp.loss_of(rho=...)`, the guarantee is zCDP and the context adds Gaussian noise. Nothing else in a query changes, unless the query names a distribution. So the script's two context functions differ only in the budget line.

```
def laplace_context(frame: pl.LazyFrame, epsilon: float) -> dp.Context:
    """Pure differential privacy: the context adds Laplace noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=epsilon),
        split_evenly_over=QUERIES,
    )


def gaussian_context(frame: pl.LazyFrame, rho: float, delta: float | None = None) ->
↪   dp.Context:
    """Zero-concentrated differential privacy: the context adds Gaussian noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(rho=rho, delta=delta),
        split_evenly_over=QUERIES,
    )
```

The privacy loss is the log of how much more likely an output is with a person in the data than without. Pure DP bounds it by $\varepsilon$ in the worst case. Zero-concentrated DP bounds it on average. A mechanism is $\rho$-zCDP when its privacy loss has mean at most $\rho$ and tails no heavier than those of a normal variable with variance $2\rho$. The Gaussian mechanism fits this definition exactly, and budgets in $\rho$ add up across queries as budgets in $\varepsilon$ do. The full edition's chapter on concentrated DP derives all of this. The documentation's Gaussian page repeats the count under $\rho = 0.1$, split over the same five queries.

```
gaussian = gaussian_context(frame, RHO)
count_gaussian = gaussian.query().select(dp.len(signed=True))
gaussian_summary = count_gaussian.summarize(alpha=ALPHA)
```

```
privacy loss: (ZeroConcentratedDivergence, 0.1)
gaussian count: Integer Gaussian scale=180 95% accuracy=354
budget per query: rho=0.0200  sqrt(2 rho)=0.2000
continuous gaussian accuracy at scale 180: 352.7935
```

The scale is again 180, on purpose. Write $\sigma$ for the standard deviation of the Gaussian noise, which OpenDP calls its scale, and $\Delta$ for the sensitivity. The mechanism is $\Delta^2/(2\sigma^2)$-zCDP, so the context sets the scale to the sensitivity over $\sqrt{2\rho}$. With a share of $\rho = 0.02$, that root is 0.2, the Laplace share. At equal scale the Gaussian bar is narrower, because Gaussian tails fall faster. But equal scale is not equal privacy: Section 7.5 converts this share of $\rho$ to an $\varepsilon$ of about 1, five times the Laplace share.

The exchange rate also differs. The Gaussian scale is the sensitivity over $\sqrt{2\rho}$, so half the error bar costs four times $\rho$, as the script confirms at $\rho = 0.4$.

```
gaussian count at rho 0.4: Integer Gaussian scale=90 95% accuracy=177
```

So on a single count, Gaussian noise under zCDP gives no gain. Its advantage is on a query that answers many things at once.

## 7.4. One person in many groups: L1 against L2

A grouped count returns a vector, one number per group, and the noise is calibrated to how far one person can move the whole vector. Laplace noise measures that distance in the $L_1$ norm, the sum of the changes across groups. Gaussian noise measures it in the $L_2$ norm, the root of the sum of their squares. Between opposite corners of a unit square, the $L_1$ distance is 2 and the $L_2$ distance is $\sqrt{2}$. In $d$ dimensions they become $d$ and $\sqrt{d}$.

On the survey, a person answers at most once per quarter, so their 36 records fall in 36 different groups of year and quarter, each changed by one. The $L_1$ distance between the two count vectors is 36, but the $L_2$ distance is $\sqrt{36} = 6$. If you do not tell the library, it must assume that all 36 records are in one group, and then both distances are 36. The `dp.polars.Bound` of Identifier truncation and bounds tells it.

```
def bounded_context(frame: pl.LazyFrame, privacy_loss: tuple[Measure, Any]) ->
↪  dp.Context:
    """A budget of either kind, with the privacy unit also bounded to one record per
    ↪  year and quarter."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(
            contributions=[
                dp.polars.Bound(by=[], per_group=CONTRIBUTIONS),  # at most 36 records
↪  overall
                dp.polars.Bound(by=["YEAR", "QUARTER"], per_group=1),  # and one in
↪  each year and quarter
            ]
        ),
```

```
        privacy_loss=privacy_loss,
        split_evenly_over=QUERIES,
    )
```

The budget now has a $\delta$, because the query has no public keys. So the context releases only the groups whose noisy count passes a threshold, as in Grouping. The script summarizes the quarterly count three ways: under zCDP with 36 contributions only, with the bound per group, and with that bound under pure DP.

```
bounded = bounded_context(frame, dp.loss_of(rho=RHO, delta=DELTA))
count_by_quarter = bounded.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
grouped_summary = count_by_quarter.summarize(alpha=ALPHA)
```

```
privacy loss with a threshold: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
grouped count, 36 contributions: Integer Gaussian scale=180 95% accuracy=354
 ↪  threshold=1133
grouped count, 1 per year and quarter: Integer Gaussian scale=30 95% accuracy=60
 ↪  threshold=184
grouped count, 1 per year and quarter, pure DP: Integer Laplace scale=180 95%
 ↪  accuracy=539.731 threshold=3712
true groups: 36  smallest 82,504  largest 131,656
```

The bound per group cuts the Gaussian scale from 180 to 30, the factor of six above, and the threshold falls with it. Under pure DP the bound changes nothing, because the $L_1$ distance is 36 either way. The thresholds are far below the smallest quarterly count, so every group is released, with an error bar of 60. The full edition's chapter on the two mechanisms proves why Gaussian noise scales with the $L_2$ sensitivity.

In general, when a person's records spread thinly over many groups, the $L_2$ sensitivity is far below the $L_1$ one. Then Gaussian noise under zCDP is the correct tool. But the gain needs a bound per group. If public knowledge cannot justify the bound, enforce it with a truncation first.

## 7.5. Stating the guarantee in epsilon

A budget in $\rho$ is convenient for accounting, but most readers of a release do not know it. OpenDP converts it to the usual pair $(\varepsilon, \delta)$ with two combinators, which Combinators covers. The first, `make_zCDP_to_approxDP`, turns a zCDP guarantee into a curve of $\varepsilon$ against $\delta$. The second, `make_fix_delta`, picks a $\delta$ on that curve. The script applies them to a Gaussian measurement whose scale makes a sensitivity of 1 cost exactly $\rho$.

```
def epsilon_of_rho(rho: float, delta: float) -> float:
    """The epsilon that OpenDP's conversion gives for rho-zCDP at delta, read off a
     ↪  Gaussian measurement."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    gaussian = dp.m.make_gaussian(*space, scale=1 / math.sqrt(2 * rho))  # sensitivity
 ↪  1 at this scale is rho-zCDP
    approx = dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta)
```

```
    epsilon, _ = approx.map(1.0)
    return float(epsilon)
```

```
rho=0.1 at delta=1e-07: epsilon=2.3484
closed form rho + 2 sqrt(rho ln(1/delta)): 2.6391
rho=0.02 (one query) at delta=1e-07: epsilon=0.9951
the whole context converted: epsilon=2.3484 delta=1e-07
```

At $\delta = 10^{-7}$, the whole budget of $\rho = 0.1$ is worth $\varepsilon = 2.35$, and one query's share is worth about $\varepsilon = 1$. The textbook conversion is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{7.1}$$

which gives 2.64 here. OpenDP's smaller number comes from a tighter conversion over the orders of the Rényi divergence, which the full edition's chapter on conversions derives. You can also convert a whole context: its `accountant` attribute is the measurement that composes every query. The two combinators on it give the same 2.35, the number to write in the release's statement.

You choose this $\delta$. Keep it far below one over the number of people, because a $\delta$ of that size allows a mechanism that publishes one person's record. With 1.7 million people, $10^{-7}$ is defensible. This $\delta$ is a conversion cost, separate from the $\delta$ of the threshold in the grouped query, so a release that needs both must budget both.

The Gaussian count's summary said 354, while the continuous conversion at the same scale gives 352.79. The context adds a discrete Gaussian to an integer count, so it reports the discrete accuracy, as Measurements explains. So to recompute an accuracy from a scale, use the discrete function for counts.

## 7.6. Signed and unsigned counts

Every count on this page has `signed=True`, because of bias. A noisy count can be negative when the true count is small, and OpenDP used to raise such a value to zero. In version 0.16.0, `signed=True` casts the count to a signed integer before the noise, so a negative result stays. With `signed=False` or no argument, the result is clamped at zero, and without the argument the library warns that the default will change.

For a single small cell, clamping even reduces the error. The problem is a sum of counts. A clamped count can be too high but never too low, so each small cell is biased upwards. A total over hundreds of cells adds up those biases. The OpenDP cookbook shows this on a table of 400 mostly empty cells. The clamped total is too high on every repetition, while the signed total is correct on average.

You can clamp a signed release afterwards for display, because that is free post-processing. So release signed counts whenever anyone might sum, average or model them. The script shows the bias on a small scale. It counts an empty frame a thousand times at a fixed Gaussian scale of 30, through the framework API without a context, which suits toy data only.

```
def mean_noisy_count_of_nobody(signed: bool, scale: float, repeats: int) -> float:
    """The mean of `repeats` noisy counts of an empty frame, at a fixed Gaussian scale,
     ↪  signed or clamped at zero."""
    empty = pl.LazyFrame(schema={"A": pl.Int32})
    domain = dp.with_margin(
        dp.lazyframe_domain([dp.series_domain("A", dp.atom_domain(T="i32"))]),
        dp.polars.Margin(by=[], max_length=1000),
    )
    count = dp.m.make_private_lazyframe(
        domain, dp.symmetric_distance(), dp.zero_concentrated_divergence(),
 ↪  empty.select(dp.len(signed=signed)), scale
    )
    return sum(int(count(empty).collect().item()) for _ in range(repeats)) / repeats
```

```
dp.len() without signed: warns=True clamped=True
expected clamped mean at scale 30: scale/sqrt(2 pi)=11.9683
standard error of the signed mean: scale/sqrt(1000)=0.9487
clamping experiment (one run): 1000 noisy counts of nobody at gaussian scale 30
  unsigned mean 11.48  signed mean 1.02  (true count 0)
```

The true count is zero. The signed mean is near zero, within the standard error of a thousand draws. The unsigned mean is near 12, the mean of the positive half of a Gaussian of scale 30, and no number of repetitions brings it down. A table of empty cells released unsigned has that 12 in every cell.

## 7.7. Which noise to choose

The choice depends on what you release and to whom you must explain it, as Table 7.1 shows.

Table 7.1.: Which noise to choose, by what is released.

| Situation | Noise and budget | Why |
|---|---|---|
| One count, sum or mean, and a guarantee in pure $\varepsilon$ | Laplace under `dp.loss_of(epsilon=...)` | The guarantee has no $\delta$ and is the easiest to state |
| A grouped table where a person touches many groups | Gaussian under `dp.loss_of(rho=..., delta=...)` with a `Bound` per group | The $L_2$ sensitivity grows with the root of the number of groups |
| Many queries composed into one release | Gaussian under zCDP | Budgets in $\rho$ add up tightly and convert once at the end |
| A release that others will analyse assuming normal errors | Gaussian | The noise is normal, so standard intervals apply |
| A guarantee that must be reported as $(\varepsilon, \delta)$ | Either; convert $\rho$ with `make_zCDP_to_approxDP` and `make_fix_delta` | OpenDP's conversion is tighter than the closed form |

| Situation | Noise and budget | Why |
|---|---|---|
| Counts that anyone might add up | Either, with `signed=True` | Clamping at zero biases totals upward |

Two cautions go with the table. Gaussian noise cannot satisfy pure DP, so a policy that demands $\delta = 0$ rules it out. And the gain from the $L_2$ sensitivity exists only when the library knows how a person's records spread across groups. So the bound per group is part of the choice.

## 7.8. The releases of one run

The script ends with the count under both budgets and the quarterly table under the bounded zCDP context. The Laplace count repeats the count of Essential statistics, with the same error bar.

```
released (one run):
  laplace count 3,811,867  (true 3,811,932, within 539.7 with probability 0.95)
  gaussian count 3,811,994  (true 3,811,932, within 354.0 with probability 0.95)
  2005 Q1 count 87,442  (true 87,372, within 60)
  2005 Q2 count 86,457  (true 86,469, within 60)
  2005 Q3 count 83,536  (true 83,518, within 60)
  2005 Q4 count 84,746  (true 84,739, within 60)
  groups released 36 of 36; within 60 of the truth: 32
```

Both counts are inside their bars. Of the 36 quarterly counts, 32 are within their bar, and the first quarter of 2005 is one of the misses. That is what an error bar at $\alpha = 0.05$ means across a table: about one cell in twenty is outside it, and you cannot tell which. On your own data, make these choices from the scales and accuracies, which cost nothing to read, and release once.

## 7.9. Summary

Table 7.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `summarize(alpha=0.05)` | The scale and the 95% accuracy of each statistic, before release | Read it before `release()`; it costs nothing |
| `laplacian_scale_to_accuracy`, `gaussian_scale_to_accuracy` | A continuous scale to a 95% error bar | For sums and means of floats |
| `discrete_laplacian_scale_to_accuracy`, `discrete_gaussian_scale_to_accuracy` | The same for integer noise | For counts, to match `summarize` |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `accuracy_to_laplacian_scale` and the three others | A target error bar to the scale it needs | Divide the sensitivity by the scale to see the $\varepsilon$ it costs |
| The error bar | Covers the noise only | Clipping, imputation, truncation and resizing bias the input, and the bar does not see it |
| `dp.loss_of(epsilon=...)` | Pure DP; the context adds Laplace noise | Halving the bar doubles $\varepsilon$ |
| `dp.loss_of(rho=...)` | zCDP; the context adds Gaussian noise | Halving the bar quadruples $\rho$ |
| `dp.polars.Bound(by=[...], per_group=1)` | One record per person per group | Cuts the $L_2$ sensitivity to the root of the number of groups; does nothing for Laplace |
| `make_zCDP_to_approxDP` then `make_fix_delta` | $\rho$ to $(\varepsilon, \delta)$, tighter than the closed form | Apply to `context.accountant` for the whole release |
| `signed=True` | Keeps negative noisy counts | Always, if the counts may be summed; clamp for display afterwards |

## 7.10. References

- OpenDP documentation: "Assessing Utility" in Getting Started, "Gaussian Noise" in the Polars user guide, and "Aggregation" in the Polars expressions guide for `signed`. The OpenDP Cookbook's "Signed Counts: Why Clipping at Zero Biases Totals" is the experiment the signed-counts section summarizes.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, the Gaussian mechanism under it, and the conversion to $(\varepsilon, \delta)$.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the integer Gaussian that the context adds to counts and its tail bounds.
- Kamath, Mouzakis, Regehr, Singhal and Steinke, "A Bias-Variance-Privacy Trilemma for Statistical Estimation" (2023): why the bias of clipping cannot be removed in general, cited by the documentation's note on accuracy.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, concentrated DP in the full edition and conversions to (ε, δ) in the full edition derive the results this chapter states.

# Part III.

# The framework

Every query that the Context API ran in the previous parts was built from a few pieces, and this part shows those pieces. OpenDP's framework describes a private computation as a chain of two kinds of object. A transformation maps data to data, and its stability bounds how much its output can change when its input changes. A measurement maps data to a random release, and its privacy bounds how much the distribution of the release can change. In a chain, these promises pass from one step to the next, and when you compose measurements, their privacy losses add up. So when you can read these objects, you can see exactly which guarantee a query carries. You can also build queries that the Context API does not offer, and check a claim that someone else makes.

The first chapter gives the vocabulary: domains, metrics and measures, and the two maps that carry the promises. The next three build the pieces in order: the data-to-data steps, the mechanisms that add noise or select, and the combinators that chain and compose them. The last chapter covers the tools around the pieces and shows how the Context API maps onto them.

Table 7.3.: The chapters of this part and the questions they answer.

| Chapter | Question it answers |
|---|---|
| A framework to understand differential privacy | What are domains, metrics, measures, transformations and measurements, and what does each promise? |
| Transformations | How do I build and chain the data-to-data steps, and how does stability flow through them? |
| Measurements | Which mechanisms does OpenDP offer, and how do I calibrate each one? |
| Combinators | How do I compose measurements, convert between privacy measures, and account adaptively? |
| Utilities and the Context API inside | How do types, parameter search and serialization work, and what does a context do under the hood? |

At the end of this part, you can build a private query from its pieces and read the guarantee of any chain with `map` and `check`. You can also explain what the Context API does when it resolves a query.

# 8. A framework to understand differential privacy

Every query of the first two parts used the Context API, which chose a noise scale, spent a share of the budget and refused a query above it. Under that API, OpenDP describes a private computation with two kinds of object. A transformation maps data to data, and it promises how far apart its outputs can be when its inputs are close. A measurement maps data to a random release, and it promises how far apart the distributions of its releases can be. Each promise is a map from an input distance to an output distance, and differential privacy (DP) in OpenDP is the bookkeeping of such maps. At the end of the chapter, you can read the guarantee of any OpenDP object from its fields and its map.

The script opendp_framework.py runs every step on OpenDP 0.16.0. Its sum example uses the `HWUSUAL` column of the Labour Force Survey (LFS) sample as a list of integers, and its workflow uses the first chapter's hundred toy scores. The domains, metrics, measures, maps, scales and error bars are the same on every run. The released values change, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part03-framework/opendp_framework.py
```

## 8.1. Two kinds of object

The first chapter built a Laplace measurement without saying what came back. The documentation's example is the same mechanism with a scale of 5. It uses `make_laplace`, which takes the domain and the metric as its first two arguments instead of through `>>`.

```
def laplace_measurement(scale: float) -> dp.Measurement:
    """Laplace noise of the given scale on one finite float, compared by absolute
     ↪ distance, under pure DP."""
    return dp.m.make_laplace(
        dp.atom_domain(T=float, nan=False),  # input domain: any float that is not NaN
        dp.absolute_distance(T=float),  # input metric: |a - b|, the units of the
 ↪ sensitivity
        scale=scale,
    )
```

What comes back is a `Measurement`, and the script reads its fields.

```
laplace scale 5: type=Measurement
laplace scale 5: input domain=AtomDomain(T=f64)
laplace scale 5: input metric=AbsoluteDistance(f64)
laplace scale 5: output measure=MaxDivergence
laplace scale 5: function=Function
```

A transformation and a measurement share four fields. The `function` is the computation, it accepts the `input_domain` and produces the `output_domain`, and the `input_metric` gives the distance between two inputs. The last two fields differ, because a **transformation** is deterministic while a **measurement** is random. A transformation's output is data, so it has an `output_metric` and a **stability map**, which bounds the distance between two outputs from a bound on the distance between the inputs. A measurement's output is a random draw, so it has an `output_measure` and a **privacy map**, which bounds the distance between two output distributions.

So the Laplace measurement accepts one finite float, compares two inputs by their absolute difference, and compares two output distributions by the max divergence, which $\varepsilon$ bounds. The documentation states the two promises in one shape. A transformation is (`d_in`, `d_out`)-stable when inputs at most `d_in` apart give outputs at most `d_out` apart. A measurement is (`d_in`, `d_out`)-differentially private when the same inputs give output distributions at most `d_out` apart. The sections below take the parts of these sentences in turn.

## 8.2. Functions

The function is the part you would write anyway. You call a transformation or a measurement like a function, and `invoke` does the same. The clamp below works on vectors of finite floats under the symmetric distance, and it clips every element into the bounds 0 to 5.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0.0, 5.0))
print(type(clamp).__name__)  # Transformation
print(clamp([1.3, 7.8, -2.5, 7.0]))  # [1.3, 5.0, 0.0, 5.0]
print(clamp.invoke([10.0]))  # [5.0]
print(type(clamp.function).__name__)  # Function
```

The `function` field holds the computation as an object, which the plugin chapter of the next part uses to wrap a Python function. A transformation adds no noise, so its result is not private. This is by design: a transformation prepares the data inside the guarantee, and only a measurement may release anything. If you print the output of a transformation, you are outside the guarantee.

## 8.3. Domains

A domain is a set of values, and the library uses it in two ways. The proof behind a map holds only for inputs in the domain, so the domain states what the map assumes. And when you chain two objects, the library checks that the output domain of the first equals the input domain of the second. The most common domain is `atom_domain`, one value of a type `T`, and `vector_domain` lifts it to vectors.

```
import opendp.prelude as dp

floats = dp.atom_domain(T=float, nan=False)
print(floats)  # AtomDomain(T=f64)
print(floats.member(1.0), floats.member(float("nan")))  # True False
print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
small = dp.atom_domain(bounds=(-2, 2))
print(small)  # AtomDomain(bounds=[-2, 2], T=i32)
print(small.member(-2), small.member(3))  # True False
pairs = dp.vector_domain(dp.atom_domain(T=bool), size=2)
print(pairs)  # VectorDomain(AtomDomain(T=bool), size=2)
print(pairs.member([True, True]), pairs.member([True, True, True]))  # True False
```

The `member` method tests whether a value is in the domain, so you can check an assumption by hand. The type `T` is written in Python terms, so `float` prints as `f64` and `int` as `i32`. The `nan` flag says whether a float may be NaN, and the installed version prints it only when it is true. So `atom_domain(T=float)` allows NaN and `atom_domain(T=float, nan=False)` does not. A measurement that adds noise needs the second, because the absolute distance to a NaN is not defined.

The `bounds` restrict the values to an interval, and the sum needs them, because a sum of unbounded values has no finite sensitivity. A vector domain can also have a `size`, which the mean needs, as the first chapter's resize showed.

## 8.4. Metrics

A metric is a rule for the distance between two members of a domain. The framework never computes it, and only uses it to give `d_in` its meaning. Under the **symmetric distance**, the distance between two datasets is the number of additions and removals that turn one into the other, counted with multiplicity. The function below computes it for two small vectors from the documentation's notebook, where the second lacks the first person's record.

```
from collections import Counter

import opendp.prelude as dp


def symmetric_distance(u: list[int], v: list[int]) -> int:
    """The number of additions and removals that turn the multiset u into v."""
    cu, cv = Counter(u), Counter(v)
    return sum(((cu - cv) + (cv - cu)).values())


print(symmetric_distance([12, 10, 8, 7], [10, 8, 7]))  # 1
print(symmetric_distance([12, 10, 8, 7], [10, 10, 8, 7]))  # 2
print(dp.symmetric_distance(), dp.absolute_distance(T=float))  # SymmetricDistance()
 ↪  AbsoluteDistance(f64)
print(dp.unit_of(contributions=1, ordered=True))  # (InsertDeleteDistance(), 1)
print(dp.unit_of(changes=1))  # (ChangeOneDistance(), 1)
```

If you remove one record, the distance is 1. If you change a 12 into a 10, the distance is 2, because a change is one removal and one addition. So under the symmetric distance, `d_in` is the largest number of records that one person can add or remove, which is the privacy unit. Two datasets are **neighbouring**, written $D \sim D'$, when they are at most `d_in` apart under the input metric, here add/remove. This is the metric behind `dp.unit_of(contributions=k)`, so on the LFS, where a person has at most 36 rows (the privacy unit), `d_in` is 36.

The **absolute distance** is the absolute value of the difference of two numbers. Under it, `d_in` is a **sensitivity**, the most an aggregate can change when one person's records change. So you rarely give a sensitivity yourself, because it appears in the middle of a chain, as the output distance of an aggregate. The full edition's chapter on sensitivity in the full edition works out the common ones.

The other dataset metrics change the symmetric distance in one way each. The insert-delete distance also protects the order of the records. The change-one distance counts replaced records in datasets of a fixed size, the replace-one relation of bounded DP. The Hamming distance is the ordered version of change-one.

## 8.5. Measures

A measure is a rule for the distance between two probability distributions, and it is what makes a measurement's `d_out` a privacy guarantee. The **max divergence** is the largest log ratio of the two probabilities of a set of outputs, over every such set. A bound of $\varepsilon$ on it is pure differential privacy, which the full edition defines in its chapter on the definition in the full edition. The library has four measures, and `d_out` has a different form under each.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.max_divergence(), dp.zero_concentrated_divergence())  # MaxDivergence
 ↪  ZeroConcentratedDivergence
print(dp.smoothed_max_divergence(), dp.fixed_smoothed_max_divergence())  #
 ↪  SmoothedMaxDivergence Approximate(MaxDivergence)
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
gaussian = space >> dp.m.then_gaussian(scale=1.0)
print(gaussian.output_measure, gaussian.map(1.0))  # ZeroConcentratedDivergence 0.5
profile = dp.c.make_zCDP_to_approxDP(gaussian).map(1.0)
print(f"{profile.epsilon(delta=1e-7):.4f}")  # 5.6709
print(dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta=1e-7).map(1.0))
 ↪  # (5.6708588354687715, 1e-07)
```

Under the max divergence, `d_out` is one number, $\varepsilon$. Under the **zero-concentrated divergence**, `d_out` is the parameter $\rho$ of zero-concentrated differential privacy (zCDP), which bounds a family of Rényi divergences at once and is the native measure of the Gaussian mechanism. A Gaussian of scale 1 at sensitivity 1 costs $\rho = 0.5$, the sensitivity squared over twice the scale squared.

The two smoothed measures carry approximate DP, whose guarantee holds up to a slack $\delta$. Under the smoothed max divergence, `d_out` is a **privacy profile**, a curve that gives the smallest $\varepsilon$ for any $\delta$, here $10^{-7}$. Under the fixed smoothed max divergence, printed `Approximate(MaxDivergence)`,

`d_out` is one pair of $\varepsilon$ and $\delta$, like the first chapter's `dp.loss_of(epsilon=1.0, delta=1e-7)`. The combinator `make_fix_delta` turns the curve into the pair. Combinators covers the conversions. Here, remember that a measurement's `output_measure` tells you how to read its `d_out`.

## 8.6. Stability maps and privacy maps

A map turns an input distance into an output distance, and it holds the guarantee. For a measurement, the call `map(d_in)` returns the smallest `d_out` that the library can prove, so the measurement is (`d_in`, `d_out`)-differentially private. For a transformation, it returns the smallest `d_out` for which it is (`d_in`, `d_out`)-stable. The `check` method says whether the guarantee fits inside a `d_out` you propose. The script reads the map of the Laplace measurement at two sensitivities.

```
epsilon_at_1 = laplace.map(d_in=1.0)  # the sensitivity over the scale: 1 / 5
epsilon_at_10 = laplace.map(d_in=10.0)
fits = laplace.check(d_in=1.0, d_out=0.2)  # is map(1.0) <= 0.2?
too_tight = laplace.check(d_in=1.0, d_out=0.19)
```

```
laplace scale 5: map(1.0)=0.2 map(10.0)=2.0
laplace scale 5: check(1.0, 0.2)=True check(1.0, 0.19)=False
```

The Laplace privacy map is the sensitivity divided by the scale. So a sensitivity of 1 at scale 5 costs $\varepsilon = 0.2$, and ten times the sensitivity costs ten times more. So the check at 0.2 passes and the check at 0.19 fails. The Context API uses only this comparison when it finds a scale: it tries scales until `check` passes at the query's share of the budget. The full edition derives the map in its chapter on the Laplace and Gaussian mechanisms in the full edition.

A map returns an upper bound that the library can prove, which is not always the smallest true one, and the sum shows why this matters.

```
import opendp.prelude as dp

dp.enable_features("contrib")
float_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 5.0))),
 ↪  dp.symmetric_distance())
int_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 5))),
 ↪  dp.symmetric_distance())
print(float_sum([1.3, 3.8, 0.0, 5.0]))  # 10.1
print(float_sum.map(d_in=2))  # 10.00000004656613
print(int_sum.map(d_in=2))  # 10
print(int_sum.check(d_in=2, d_out=10), float_sum.check(d_in=2, d_out=10.0))  # True
 ↪  False
```

A person who adds two records of at most 5 can move the sum by 10, and the integer sum's map says exactly 10. The float sum's map says a little more, because adding floats rounds, and the proof adds a margin for the rounding. The margin is tiny, but `check` is a strict comparison, so the float sum fails at exactly 10. So let the maps give you the distances, and do not type in the number you know from theory.

## 8.7. Chaining a transformation and a measurement

When a transformation's output domain and metric equal a measurement's input domain and metric, the `>>` operator joins them into a new measurement. The new measurement applies the transformation and then the measurement. Its privacy map applies the measurement's map to the output of the transformation's map. The script builds the documentation's chain of a clamp, a sum and Laplace noise on `HWUSUAL`, the hours a respondent usually works per week. Without the nulls and the code 99 for "not applicable", that is 1,496,192 whole numbers of hours from 0 to 80.

```
clamp = dp.t.make_clamp(dp.vector_domain(dp.atom_domain(T=int)),
 ↪  dp.symmetric_distance(), bounds=HOURS_BOUNDS)
bounded_sum = dp.t.make_sum(clamp.output_domain, clamp.output_metric)  # the clamp's
 ↪  output space is its input space
laplace_int = dp.m.make_laplace(bounded_sum.output_domain, bounded_sum.output_metric,
 ↪  scale=LAPLACE_SCALE)
noisy_sum = clamp >> bounded_sum >> laplace_int  # the chain: its input space is the
 ↪  clamp's, its measure Laplace's
sum_sensitivity_1 = bounded_sum.map(d_in=1)  # one row of at most 80 hours
sum_sensitivity_36 = bounded_sum.map(d_in=CONTRIBUTIONS)  # 36 rows
epsilon_at_scale_5 = noisy_sum.map(d_in=CONTRIBUTIONS)
```

Each constructor takes the output space of the object before it as its input space, so the domains match by construction: the clamp outputs integers in 0 to 80, which the sum accepts. The script prints the fields and the maps along the chain.

```
clamp: input domain=VectorDomain(AtomDomain(T=i32))
clamp: output domain=VectorDomain(AtomDomain(bounds=[0, 80], T=i32))  map(36)=36
sum: output domain=AtomDomain(T=i32)  output metric=AbsoluteDistance(i32)
sum: map(1)=80 map(36)=2880
chain scale 5: input domain=VectorDomain(AtomDomain(T=i32))
chain scale 5: input metric=SymmetricDistance()  output measure=MaxDivergence
chain scale 5: map(36)=576.0
```

Read the distances from left to right. The clamp is 1-stable, because clipping adds or removes no record, so 36 records in give 36 out. The sum's map turns that into a sensitivity: one record moves the sum by at most 80 hours, so 36 records move it by at most 2880. The Laplace map at a scale of 5 then gives $\varepsilon = 576$, which is no privacy at all. That scale was chosen for one float, and the map shows the problem before anything is released.

```
sum_scale = dp.binary_search_param(
    lambda scale: clamp >> bounded_sum >> dp.m.then_laplace(scale), d_in=CONTRIBUTIONS,
     ↪  d_out=EPSILON
)
noisy_sum = clamp >> bounded_sum >> dp.m.then_laplace(sum_scale)
sum_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=sum_scale, alpha=0.05)
released_sum = noisy_sum(hours)
```

```
chain for epsilon 1 at d_in 36: scale=2880.0 map(36)=1.0
chain for epsilon 1 at d_in 36: 95% accuracy=8,628.2 (0.01532% of the sum)
```

The utility `binary_search_param` gets the right scale from the map. It takes a function from a scale to a chain and finds the smallest scale whose `check` passes, here 2880, the sensitivity over $\varepsilon = 1$. The chain uses `then_laplace`, because a `then_` constructor takes its input space from the object before it. The sum is an integer, so the noise is discrete Laplace, with the discrete accuracy function. An error of about 8,600 hours on 56 million is tiny, because a sum over a large dataset is the easy case. Transformations and Measurements give the maps of the other pieces.

## 8.8. The typical workflow in both APIs

The Context API is a layer over the framework, so you can write the first chapter's five steps in either API. The script does both on the hundred toy scores of A first release, from the same seed, so the scales are comparable. The framework states the privacy unit as a distance and a metric, and the budget as a distance and a measure.

```
d_in = 1  # neighbouring datasets differ by at most one row...
input_metric = dp.symmetric_distance()  # ...counted as additions and removals
input_domain = dp.vector_domain(dp.atom_domain(T=float))  # a vector of floats, NaN
 ↪  allowed
d_out = EPSILON  # the distributions of two releases differ by at most epsilon...
privacy_measure = dp.max_divergence()  # ...measured by the max divergence, which is
 ↪  pure DP
```

```
unit: d_in=1 input metric=SymmetricDistance()  (Context API: (SymmetricDistance(), 1))
loss: d_out=1.0 output measure=MaxDivergence  (Context API: (MaxDivergence, 1.0))
input domain: VectorDomain(AtomDomain(nan=true, T=f64))
```

These are the same pairs that `dp.unit_of` and `dp.loss_of` returned in the first chapter. The framework also asks for the input domain, which the context found from your list, and it allows NaN because a list of floats can hold one. The public bounds and the imputed value are the same in both APIs. The fourth step mediates access, and it shows what a context is made of.

```
compositor = dp.c.make_adaptive_composition(
    input_domain=input_domain,
    input_metric=input_metric,
    output_measure=privacy_measure,
    d_in=d_in,
    d_mids=[d_out / QUERIES] * QUERIES,  # the shares of the budget, one per query
)
queryable = compositor(data)  # a measurement; invoking it on the data gives the
 ↪  queryable that answers queries
```

```
compositor: type=Measurement map(1)=1.0
queryable: type=Queryable
context accountant: type=Measurement d_mids=[0.3333, 0.3333, 0.3333]
```

An **adaptive composition** is itself a measurement, whose privacy map is the sum of the shares, because the budgets of sequential releases add up (the full edition's chapter on properties). When you invoke it on the data, it releases nothing. It returns a **queryable**, which holds the data and answers the measurements you submit, one share each. It is adaptive because each measurement may depend on the earlier answers. A context's `accountant` field is this same object, with the same shares. The fifth step submits the queries, each built by hand.

```
count = dp.t.make_count(input_domain, input_metric)
count_sensitivity = count.map(d_in)  # adding or removing one row changes a count by 1
count_scale = dp.binary_search_param(
    lambda scale: count >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
count_measurement = count >> dp.m.then_laplace(count_scale)
dp_count = queryable(count_measurement)
```

```
count: sensitivity=1  output metric=AbsoluteDistance(i32)
count at scale 3.0: map(1)=0.33333333333333337 check(1, 1/3)=False
count by search: scale=3.0000000000000004 check(1, 1/3)=True
count scale: framework=3.0000 context=3.0000 equal=True
count: 95% accuracy=9.4457
```

The count's stability map gives the sensitivity 1, so from theory the Laplace scale is 1 over a third of $\varepsilon$, which is 3. But a typed 3.0 fails the check, because one over three in floating point is one rounding step above the share. The search returns a scale one rounding step above 3, which passes. It is bit for bit the scale that `context.query().count().laplace().param()` gave in the first chapter, because the context runs the same search. The queryable spends the first share on the count. The mean uses the released count as its size, as in the first chapter.

```
mean = (
    dp.t.make_impute_constant(input_domain, input_metric, IMPUTED)  # NaN becomes the
 ↪  public midpoint
    >> dp.t.then_clamp(BOUNDS)
    >> dp.t.then_resize(size=dp_count, constant=IMPUTED)  # the released count fixes
     ↪  the size
    >> dp.t.then_mean()
)
mean_scale = dp.binary_search_param(
    lambda scale: mean >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
dp_mean = queryable(mean >> dp.m.then_laplace(mean_scale))
```

```
mean: output domain=AtomDomain(T=f64)  output metric=AbsoluteDistance(f64)
mean at size 100: sensitivity=1.0000
mean scale at size 100: framework=3.0000 context=3.0000 equal=True
mean at size 100: 95% accuracy=8.9872
mean scale at the released count: equal=True
```

The four transformations chain into one, and its stability is the maps of the parts composed. At the true size of 100, one row moves the mean by at most 1, so the scale is again 3, the same as the context's. The page quotes the scale at the true size, because the scale at the released count changes from run to run. A third query takes the last share, and the queryable refuses a fourth.

```
try:
    queryable(count_measurement)
    refusal = "released"
except dp.OpenDPException as err:
    refusal = f"{err.variant}: {err.message}"
```

```
fourth query: FailedFunction: out of queries
```

The queryable raises the library's own exception, which the context wrapped in a `ValueError`. It also refuses a query whose map is more than its share, before it touches the data. So a context is three objects you can build yourself: a space, an adaptive composition over it, and a queryable. Utilities and the Context API inside shows where a query uses each. Here are the releases of one run.

```
released (one run):
  laplace scale 5 on 0.0: 12.84  (within 15.0 with probability 0.95)
  hours sum 56,316,146  (true 56,315,594, within 8,628 with probability 0.95)
  count 99  (true 100, within 9.4 with probability 0.95)
  mean scale 3.0303  mean 50.43  (true 50.93)
  third query, a second count 104
```

Each release landed within its error bar. You can compare them with the truth only because the LFS sample and the toy scores are public. On real data, read the error bars, which cost nothing, and release once.

## 8.9. Summary

Table 8.1.: The objects and elements of the framework, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `Transformation` | A deterministic map from data to data, with an `output_metric` and a stability map | Its output is not private; chain it with a measurement before anything leaves |
| `Measurement` | A randomized map from data to a release, with an `output_measure` and a privacy map | Its `map(d_in)` is the guarantee; read it before releasing |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `trans(x)`, `trans.invoke(x)` | Evaluate the `function` field | Printing a transformation's output steps outside the guarantee |
| `atom_domain(T, bounds, nan)` | One value of type `T`, optionally bounded, with or without NaN | Noise needs `nan=False`; a sum needs `bounds`; the printed form shows `nan` only when true |
| `vector_domain(domain, size)` | Vectors of a domain, optionally of fixed length | A mean needs the `size`; `member` tests a value by hand |
| `symmetric_distance()` | Additions and removals, with multiplicity | `d_in` is the records one person can add or remove |
| `absolute_distance(T)` | The difference of two numbers | `d_in` is a sensitivity; it appears midway through a chain |
| `insert_delete`, `change_one`, `hamming` | Ordered add/remove, replace-one, ordered replace-one | Pick the one that matches how your data can change |
| `max_divergence()` | Pure differential privacy; `d_out` is $\varepsilon$ | The measure behind `loss_of(epsilon=...)` |
| `zero_concentrated_divergence()` | zCDP; `d_out` is $\rho$ | The Gaussian mechanism's native measure |
| `smoothed_max_divergence()`, fixed | Approximate DP; `d_out` is a privacy profile, or one $(\varepsilon, \delta)$ pair | `make_fix_delta` turns the curve into the pair |
| `obj.map(d_in)`, `obj.check(d_in, d_out)` | The smallest provable `d_out`; whether a proposed one fits | Let the map produce the distance; a hand-typed 3.0 fails by one rounding step |
| `trans >> meas` | A measurement whose map composes the two maps | Domains and metrics must match; `then_` constructors match them for you |
| `binary_search_param` | The smallest scale whose chain passes `check` | What the Context API runs for `param()` |
| `make_adaptive_composition` | A measurement whose invocation returns a queryable | A context is this, over a space, plus the queryable |

## 8.10. References

- OpenDP documentation: "A Framework to Understand DP", "Differential Privacy with OpenDP", "Core Structures" and "Supporting Elements" in the Framework API user guide, and "API Comparison".

- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the framework of domains, metrics, measures, stability maps and privacy maps that the library implements.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): sensitivity and the Laplace mechanism.
- The full edition's chapters on the definition in the full edition, post-processing and composition in the full edition, the Laplace and Gaussian mechanisms in the full edition and sensitivity in the full edition derive the results this chapter states.

# 9. Transformations

Every query of the previous parts started with the data and ended with noise, with a chain of steps between them that cast, imputed, clamped, resized and aggregated. In OpenDP each step is a **transformation**, a function from data to data. Its stability map bounds how much its output can change when its input changes (A framework to understand differential privacy). This chapter builds the transformations, from preprocessing to the sum, the mean and the quantile scores, and shows how stability passes along a chain.

A chain works only when the output space of each step is exactly the input space of the next. Then the chain's stability map applies the maps of its steps in turn. At the end of the chapter, you can build a sum, a mean or a median's scores from the framework alone. You can also read the sensitivity for your privacy unit, and recognize the error that OpenDP raises when two spaces do not match.

The script opendp_transformations.py runs every step on OpenDP 0.16.0 with the Labour Force Survey (LFS) sample. Its vector is the `HWUSUAL` column, the hours a respondent usually works, as integers, without its nulls and the code 99 for "not applicable". The sensitivities, scales, accuracies, scores and true values are the same on every run. The noisy releases change, because OpenDP's noise cannot be seeded, and so does a float sum that keeps a random sample of its input. They are in one block marked as one run.

```
uv run python code/part03-framework/opendp_transformations.py
```

## 9.1. Constructors and chains

Every constructor has two forms. The `make_` form takes the input space first, as a domain and a metric, and then its own arguments, as in `make_clamp(input_domain, input_metric, bounds)`. The `then_` form leaves the space out. It returns a partial constructor, and `>>` fills in the space from the object to its left, which is a space written as a tuple or a transformation. The two forms build the same object.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = dp.t.make_clamp(*space, bounds=(0, 10))
print(clamp([-5, 3, 50]))  # [0, 3, 10]
print(clamp.output_space)  # (VectorDomain(AtomDomain(bounds=[0, 10], T=i32)),
 ↪  SymmetricDistance())
total = dp.t.make_sum(*clamp.output_space)
chain = space >> dp.t.then_clamp(bounds=(0, 10)) >> dp.t.then_sum()
```

```
print(chain([-5, 3, 50]), (clamp >> total)([-5, 3, 50]))  # 13 13
print(chain.output_space)  # (AtomDomain(T=i32), AbsoluteDistance(i32))
print(clamp.map(1), total.map(1), chain.map(1))  # 1 10 10
```

The clamp changes the domain and not the metric. Its output is vectors of integers between 0 and 10, still under the symmetric distance, the number of rows to add or remove to turn one vector into the other. The sum takes that output space as its input space, as `make_sum(*clamp.output_space)` says, and outputs one integer under the absolute distance. So read `output_space` after each step, because the next constructor must accept exactly that space (Section 9.6).

The number that `map` returns is the **stability**, the largest change in the output for inputs at most `d_in` apart. For the clamp it is 1, because one added row in gives one added row out. For the sum it is 10, the larger bound, and for the chain it is the sum's map applied to the clamp's. When a chain ends with an aggregator, its stability is the sensitivity of the query, the number a measurement needs to calibrate its noise. The full edition's chapter on sensitivity derives it for sums, means and histograms.

## 9.2. Preprocessing

Preprocessing turns the data into the domain that an aggregator accepts: the sum needs bounded values, and the mean needs bounded values and a known size. The documentation lists the preprocessing constructors in the usual order of a chain, which is cast, impute, clamp and resize, and the subsections keep that order. All of them except the resize work row by row, so one added row in is one added row out, and the stability is 1.

### 9.2.1. Casting and imputing

Data from a file are strings, and `make_cast` turns a vector of strings into a vector of another type. A string that does not parse becomes a missing value, so the output is a vector of options, which hold a value or nothing. No aggregator accepts options. If it skipped the missing values, it would understate the sensitivity, and if it kept them, it would show whether a value was missing. So remove them first. The step `make_impute_constant` puts in a value you choose, and `make_drop_null` drops the rows. The step `make_cast_default` casts and imputes the type's default, 0 for numbers.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
cast = strings >> dp.t.then_cast(TOA=int)
print(cast.output_domain)  # VectorDomain(OptionDomain(AtomDomain(T=i32)))
imputed = cast >> dp.t.then_impute_constant(0)
print(imputed(["1", "2", "x"]))  # [1, 2, 0]
print(imputed.output_domain)  # VectorDomain(AtomDomain(T=i32))
by_default = strings >> dp.t.then_cast_default(TOA=int)
print(by_default(["1", "2", "x"]))  # [1, 2, 0]
floats = dp.vector_domain(dp.atom_domain(T=float)), dp.symmetric_distance()
```

```
print(floats[0])  # VectorDomain(AtomDomain(nan=true, T=f64))
uniform = floats >> dp.t.then_impute_uniform_float(bounds=(0.0, 10.0))
print(uniform.output_domain)  # VectorDomain(AtomDomain(T=f64))
print(cast.map(1), imputed.map(1), uniform.map(1))  # 1 1 1
```

Floats have their own missing value, NaN, which the plain `atom_domain(T=float)` allows, as the printed `nan=true` shows. The constructor `make_impute_constant` accepts that domain too. And `make_impute_uniform_float` replaces each NaN by a uniform draw from the bounds you give, so that imputed values do not collect at one point. Both output the domain without NaN that the aggregators need, with a stability of 1. Preparing microdata does the same on a whole table through Polars.

### 9.2.2. Clamping and resizing

Clamping moves every value into public bounds: a value below the lower bound becomes the lower bound, and a value above the upper becomes the upper. So it costs no accuracy on data that are already in the bounds, and it makes the sum's sensitivity finite. Take the bounds from public knowledge, such as a codebook. Do not take them from the data, because a bound read from the data leaks it.

Resizing has a stability of 2. It pads a vector shorter than the target size with a constant, and cuts a longer one down by a random sample. The mean and the variance need it, because they must know the number of rows they divide by, and that number is private. So the resize writes a size into the domain, which OpenDP then treats as public: a guess, a released count or public knowledge. For the stability, add one row to an input shorter than the target. The output gains that row and loses one padded row, so two rows change for one.

```
import opendp.prelude as dp

dp.enable_features("contrib")
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = floats >> dp.t.then_clamp(bounds=(0.0, 10.0))
print(clamp([-5.0, 3.0, 50.0]), clamp.map(1))  # [0.0, 3.0, 10.0] 1
resize = clamp >> dp.t.then_resize(size=5, constant=5.0)
print(resize([1.0, 2.0, 3.0]))  # [1.0, 2.0, 3.0, 5.0, 5.0]
print(resize.output_domain)  # VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64),
 ↪  size=5)
print(resize.map(1))  # 2
```

A wrong target biases the result, as the documentation's resize page shows. A target below the true size throws rows away, which widens the spread of the mean, and a target above it pulls the mean toward the constant. No error bar shows either error, because the size is treated as true, and the resize cannot warn you without leaking the true size. So spend a little budget on a count first, which lands close to the truth because its sensitivity is 1.

### 9.2.3. Categories and bins

Three constructors map between categorical values and indices. The first, `make_find`, replaces each value by its position in a list of categories. A value outside the list becomes a missing index, so an imputation follows and gives it the next index. The step `make_find_bin` turns a number into the index of its bin, from the bin edges, and `make_index` turns an index back into a label.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
find = strings >> dp.t.then_find(categories=["A", "B", "C"]) >>
 ↪  dp.t.then_impute_constant(3)
print(find(["A", "B", "C", "A", "D"]))  # [0, 1, 2, 0, 3]
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
bins = floats >> dp.t.then_find_bin(edges=[1.0, 2.0, 10.0])
print(bins([0.0, 1.0, 3.0, 15.0]))  # [0, 1, 2, 3]
label = bins >> dp.t.then_index(categories=["low", "mid", "high"], null="other")
print(label([0.0, 1.0, 3.0, 15.0]))  # ['low', 'mid', 'high', 'other']
counts = strings >> dp.t.then_count_by_categories(categories=["A", "B", "C"])
print(counts(["A", "B", "C", "A", "D"]))  # [2, 1, 1, 1]
print(counts.output_metric, counts.map(1))  # L1Distance(i32) 1
```

The aggregator for categories is `make_count_by_categories`, which counts each category and adds a count of all other values unless you set `null_category=False`. Its output is a vector of counts, so its output metric is a vector norm. The default $L_1$ distance goes with Laplace noise, and the $L_2$ distance, through the `MO` argument, goes with Gaussian noise. One added row changes one count by one, so the stability is 1 in either norm.

## 9.3. The sum

The sum is the base of the other aggregators, and its documentation page shows where sensitivity comes from. Start with integers in public bounds and no known size.

```
import opendp.prelude as dp

dp.enable_features("contrib")
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 10))), dp.symmetric_distance()
total = bounded >> dp.t.then_sum()
print(total([1, 2, 4]), total.map(1))  # 7 10
sized = dp.vector_domain(dp.atom_domain(bounds=(-10, 10)), size=3),
 ↪  dp.symmetric_distance()
sized_total = sized >> dp.t.then_sum()
print([sized_total.map(d) for d in range(5)])  # [0, 0, 20, 20, 40]
```

One added row changes the sum by at most the larger bound in absolute value, here 10, which is `map(1)`. When the size is public, you write it into the domain, and the bound is tighter. Two datasets of the same size differ at least by one replaced row, which is one removal and one addition,

a symmetric distance of 2. A replacement changes the sum by at most the width of the bounds, which is `map(2)`. Odd distances round down, so `map(1)` is 0 and the last line is a staircase.

On the survey, the script builds the vector once with Polars, and the chain clamps the hours into the codebook's bounds, 0 to 98, and sums them.

```
def hours_usually_worked(frame: pl.LazyFrame) -> list[int]:
    """The `HWUSUAL` column without nulls and without the code 99, as a list of
     ↪ ints."""
    kept = frame.filter(pl.col("HWUSUAL").is_not_null() & (pl.col("HWUSUAL") != 99))
    return kept.select(pl.col("HWUSUAL").cast(pl.Int64)).collect()["HWUSUAL"].to_list()
```

```
def int_sum(size: int | None = None) -> dp.Transformation:
    """Clamp to the public bounds, then sum; with a size the domain says how many rows
     ↪ there are."""
    space = dp.vector_domain(dp.atom_domain(T=int), size=size), dp.symmetric_distance()
    return space >> dp.t.then_clamp(BOUNDS) >> dp.t.then_sum()
```

```
hours vector: 1,496,192 values
true sum: 56,315,594  true mean: 37.6393  true median: 37
int sum: value 56,315,594  map(1)=98  map(36)=3528
sized int sum: map(36)=1764  map(35)=1666
```

Every map on this page is read at an input distance of 36, the most rows one person has (the privacy unit). Without a size, the sensitivity is 36 rows of 98 hours, which is 3,528 hours. With the size, it is half that, 18 replacements, and a distance of 35 rounds down to 17. But the Polars filter runs outside OpenDP, so the guarantee does not cover it. Preparing microdata runs it inside the guarantee, and Essential statistics releases this sum through the Context API.

Floats are different, for two reasons from the documentation's sum page. The first is rounding: each partial sum of floats rounds to the nearest float. So the sums of two datasets at distance 2 can differ by a little more than the width of the bounds, and the stability map adds a term for that.

```
import opendp.prelude as dp

dp.enable_features("contrib")
metric = dp.symmetric_distance()
sized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0)), size=1000), metric
print((sized >> dp.t.then_sum()).map(2))  # 20.00000000004426
unsized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0))), metric
print((unsized >> dp.t.then_sum()).map(1))  # 20.00000009313226
negative = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 0.0))), metric
print((negative >> dp.t.then_sum()).map(1))  # 10.00000009313226
print(dp.t.make_bounded_float_checked_sum(size_limit=100, bounds=(-10.0, 0.0)).map(1))
 ↪ # 10.00000000000295
```

With a known size of a thousand rows, the term is in the eleventh decimal place. The second reason applies when the size is unknown. Then the rounding error has no bound, so the float sum

first keeps at most $2^{20}$ rows, a little over a million, by a random sample. A neighbour with one more row may then drop a different row, so the sensitivity becomes the largest of the magnitudes of the bounds and their width. That doubles it for bounds symmetric about zero (the second line) but not for bounds of one sign (the third). With `make_bounded_float_checked_sum` you set the size limit, and a limit of 100 shrinks the rounding term to the twelfth decimal place.

This limit matters on the survey, because the hours vector has 1,496,192 values, more than $2^{20}$. So a plain `then_sum` on them as floats sums a random million of them. The result is about 70% of the true sum, as the one-run block at the end shows, and nothing warns you. The guarantee holds for the truncated sum, but the number answers a different question. So set a size limit from public knowledge above any size the data can have, here four million rows. Or put the exact size in the domain when it is public.

```
def float_sum(size_limit: int | None = None) -> dp.Transformation:
    """The float sum: `then_sum` truncates to 2^20 rows; a size limit of your own lifts
     ↪  that."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    clamped = space >> dp.t.then_clamp(FLOAT_BOUNDS)
    if size_limit is None:
        return clamped >> dp.t.then_sum()
    return clamped >> dp.t.make_bounded_float_checked_sum(size_limit=size_limit,
     ↪  bounds=FLOAT_BOUNDS)
```

```
float sum, size limit 2^20=1,048,576: map(36)=3528.0000009127
float sum, size limit 4,000,000: value 56,315,594  map(36)=3528.0000038179
```

Both sensitivities are 3,528 hours plus a rounding term, and the larger limit gives a larger term, in the sixth decimal place instead of the seventh. The term also depends on the summation algorithm, which the specialized constructors take as the `S` argument.

```
import opendp.prelude as dp

dp.enable_features("contrib")
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0),
 ↪  S="Pairwise<f64>")
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0),
 ↪  S="Sequential<f64>")
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 4.43e-11 4.44e-09
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0),
 ↪  S="Pairwise<f64>")
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0),
 ↪  S="Sequential<f64>")
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 8.85e-08 4.44e-03
```

The map at distance 0 isolates the rounding term, because the same rows in a different order can sum to different floats. The default pairwise sum adds the values as a balanced tree, so its term grows a little faster than the number of rows. The sequential sum adds them one after another, and its term grows with the square of the number of rows. Use the sequential analysis

for a sum computed elsewhere, because database engines add floats in sequence. Integer sums have no rounding term, which is one reason to cast hours to integers.

To release the integer sum, add a Laplace measurement with a scale equal to the sensitivity over $\varepsilon$ (the full edition's chapter on the Laplace and Gaussian mechanisms). Each of the three measurements on this page uses $\varepsilon = 0.5$ on its own, and Combinators composes them. Here the scale is 7,056 hours, and the error bar is 0.04% of the total.

```
sum release: scale=7056  map(36)=0.5000  95% accuracy=21,138.4
```

## 9.4. The mean and the variance

The mean is a sum divided by a size, and `make_mean` divides by the size in its input domain, not by the length of the data. So the domain must have a size, or the constructor refuses.

```
mean without a size: dataset size must be known. Either specify size in the input
 ↪  domain or use make_resize
```

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0)), size=10),
 ↪  dp.symmetric_distance()
mean = space >> dp.t.then_mean()
print(mean([5.0] * 10), mean.map(2))  # 5.0 1.0000000000000169
print(mean([5.0]))  # 0.5
print(mean.input_domain.member([5.0]))  # False
```

With ten values between 0 and 10, one replaced value moves the mean by at most the width over the size. So `map(2)` is 1, plus a rounding term. The second print shows the documentation's warning: the mean of a one-element list is 0.5, because the division uses the declared size and nothing checks the data. Each transformation assumes that its input is in its input domain, and `member` checks that.

On the survey the size is not public, so the chain resizes first, with the true median of 37 hours as the padding constant. A real release would resize to a released count, as in A first release, or to a public total. The script resizes to the true size, which it can do because the data are public, so nothing is padded.

```
def mean_chain(size: int, constant: float) -> dp.Transformation:
    """Clamp, resize to a known size (padding with the constant), then the mean over
     ↪  that size."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    return space >> dp.t.then_clamp(FLOAT_BOUNDS) >> dp.t.then_resize(size=size,
     ↪  constant=constant) >> dp.t.then_mean()
```

```
mean chain: value 37.6393  map(36)=0.0023580
mean release: scale=0.0047160  map(36)=0.5000  95% accuracy=0.0141
```

Compare this map with the sized sum's. That map was 18 replacements of 98 hours, and divided by the size it is 0.0012. The mean's map is twice that, because the resize in front of it is 2-stable and turns the input distance of 36 into 72. That is the cost of an unknown size, paid once per chain. At $\varepsilon = 0.5$, the Laplace scale is 0.0047 hours and the error bar 0.014 hours.

The documentation's mean page also shows the plug-in approach, which releases a sum and a count and then divides. The division is post-processing, so it costs no more budget (the full edition's chapter on properties). But at the same budget the resized mean is tighter, because its numerator and denominator use the same number of rows. The variance uses the same preprocessing. On the resized chain, `then_variance` gives 137.63 hours squared, and its stability at distance 36 is 0.2311.

```
variance chain: value 137.6290  map(36)=0.2311
```

## 9.5. Quantile scores

Noise alone cannot release a quantile, and the documentation's quantile page shows why. Take a dataset with half its values at the lower bound and half at the upper. One added value moves its median from the middle of the range to the upper bound, so the sensitivity is half the range. Laplace noise of that size hides the median. So OpenDP scores every candidate by how far it is from the quantile and lets a selection mechanism pick one. This is the exponential mechanism of the full edition's chapter on selection, and `make_quantile_score_candidates` computes the scores.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
median = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75, 100],
 ↪  alpha=0.5)
print(median([10, 20, 30, 40, 50, 60, 70, 80, 90]))  # [9, 5, 0, 5, 9]
print(median.output_metric, median.map(1))  # LInfDistance(T=u64) 1
quartile = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75,
 ↪  100], alpha=0.25)
print(quartile([10, 20, 30, 40, 50, 60, 70, 80, 90]), quartile.map(1))  # [9, 1, 8, 19,
 ↪  27] 3
```

A candidate's score for the median is the absolute difference between the number of values below it and the number above it. In the snippet, the middle candidate has four values on each side and scores zero, while the lowest has all nine above it. So the lowest score is the best, and the selection picks the minimum. For another quantile, the two counts get weights. The quarter quantile weights the values below by three, so the score is zero where a quarter of the data are below, and its stability is 3. The output metric is the $L_\infty$ distance, the largest change of any score, because one row moves every score at once.

On the survey, the candidates are the whole numbers of hours from 0 to 98.

```
def median_scores() -> dp.Transformation:
    """One score per candidate: how far it is from being the median; the lowest score
     ↪  is the best candidate."""
    space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
    return space >> dp.t.then_quantile_score_candidates(candidates=CANDIDATES,
     ↪  alpha=0.5)


def median_measurement(scale: float) -> dp.Measurement:
    """Noisy max over the scores releases an index; `negate` makes the lowest score the
     ↪  one most likely to win."""
    return median_scores() >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale,
     ↪  negate=True)
```

```
median scores: lowest 15,190 at 37 hours, next 89,573 at 36  map(36)=36
median release: scale=144  map(36)=0.5000  score gap=74,383  gap/scale=517
```

The lowest score is at 37 hours, the true median, and the next candidate scores worse by 74,383. To release it, the noisy max of Measurements adds noise to every score and reports the index of the best one. Its privacy map under the max divergence is twice the stability over the scale. So the scores' stability of 36 at a scale of 144 gives $\varepsilon = 0.5$. The gap between the two best scores is over 500 times the scale, so the released median is 37 on almost every run. The `make_private_quantile` of the next chapter chains the same steps but doubles its scale for the median, so its scale 72 is this chain's 144.

## 9.6. Reading a chain

Two methods read the promise of a chain. The `map` method gives the stability, or the privacy loss when a measurement ends the chain. And the `check` method asks whether the promise at an input distance is within a bound. The sum chain's map at 36 is 3,528, so `check(36, 3528)` is true and `check(36, 3527)` is false. Neither method touches the data, and the context runs `check` in a loop to find a scale (Utilities and the Context API inside).

```
int sum chain: check(36, 3528)=True  check(36, 3527)=False
```

You must also read refusals. The `>>` operator checks, when it builds the chain, that the output space of each step is the input space of the next. Below, a sum built on vectors bounded by 0 and 100 follows a clamp to 0 and 98.

```
try:
    _ = clamped >> dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 100))),
 ↪  dp.symmetric_distance())
    mismatch = "built"
except dp.OpenDPException as err:
    mismatch = f"{err.variant}: {err.message}"
```

```
mismatched chain: DomainMismatch: Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0, 98], T=i32))
    input_domain:  VectorDomain(AtomDomain(bounds=[0, 100], T=i32))
```

The message names both spaces. To fix the chain, build the sum from the clamp's output space, as `make_sum(*clamped.output_space)`. The `then_` form does that for you. A metric mismatch is similar. An ordered sum needs the insert-delete distance, so it refuses to follow a clamp under the symmetric distance, and `make_ordered_random` between them converts the metric. The third refusal is a domain that lacks what the constructor needs, such as a size for the mean or bounds for the sum.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0, 10))
try:
    clamp >> dp.t.make_bounded_int_ordered_sum(bounds=(0, 10))
except dp.OpenDPException as err:
    print(err.variant)  # MetricMismatch
try:
    space >> dp.t.then_sum()
except dp.OpenDPException as err:
    print(err.message)  # `input_domain` must be bounded. Use `make_clamp` to bound
     ↪  data.
```

Every refusal happens when the chain is built, before it touches any data. Here are the releases of one run, to read against their error bars.

```
released (one run):
  sum 56,307,689  (true 56,315,594, within 21,138 with probability 0.95)
  float sum truncated to 2^20 rows 39,467,741  (true 56,315,594; about 70% of it)
  mean 37.6367  (true 37.6393, within 0.0141 with probability 0.95)
  median 37  (true 37)
```

The sum and the mean landed within their bars, as they do about 19 times in 20. The truncated float sum is not noise but a wrong computation, 30% short, and no error bar covers it. The median is exact, as the gap between its scores predicted.

## 9.7. Summary

Table 9.1.: The transformations of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `make_x(domain, metric, ...)`, `space >> then_x(...)` | Two forms of every constructor | Read `output_space` after each step |
| `transformation.map(d_in)` | The stability at input distance `d_in` | Read it at the privacy unit's distance, 36 here |
| `make_cast`, `make_cast_default` | Parse strings into a type; failures become options or the default | Impute or drop options before any aggregator |
| `make_impute_constant`, `make_impute_uniform_float` | Replace missing values by a constant or a uniform draw from bounds | Stability 1; the constant or bounds are public |
| `make_clamp(bounds)` | Pin values into public bounds | Bounds come from a codebook, never from the data |
| `make_resize(size, constant)` | Pad or sample to a known size, written into the domain | Stability 2; a wrong size biases silently |
| `make_find`, `make_find_bin`, `make_index`, `make_count_by_categories` | Values to indices, numbers to bins, indices to labels, counts per category | Counts carry an $L_1$ or $L_2$ metric |
| `make_sum` | Sum of bounded values; with a size, per replacement | Unsized float sums truncate to $2^{20}$ rows: set `size_limit` |
| `make_mean`, `make_variance` | Divide by the size in the domain | A resize before them doubles the input distance |
| `make_quantile_score_candidates` | A score per candidate, lowest best | Feed the scores to the next chapter's noisy max |

## 9.8. References

- OpenDP documentation: "Transformations", "Preprocessing: Resize", "Aggregation: Sum", "Aggregation: Mean" and "Aggregation: Quantile" in the API user guide.
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the rounding term and the truncation of the float sum.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (2011): the quantile scores and their sensitivity.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the selection behind the noisy max.
- The full edition's chapters on sensitivity in the full edition, the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition and selection in the full edition derive the results this chapter states.

# 10. Measurements

A **measurement** ends a chain. It takes data and returns a random release, and it bounds how much the distribution of that release can change when one person's data change. Transformations built the deterministic half of a query, and this chapter builds the random half, which gives the privacy guarantee.

You read every measurement in OpenDP from four parts. The input domain says what it accepts, and the input metric compares two inputs. The output measure is the currency of the privacy cost, and the privacy map turns a distance between inputs into a cost in that currency. With these four parts, you can calibrate any mechanism in the library and compare two mechanisms fairly.

The script opendp_measurements.py builds every mechanism on this page on OpenDP 0.16.0 and prints its metric, measure and map. A thresholded count of `ILOSTAT` and a private median of `HWUSUAL` run on the Labour Force Survey (LFS), and the others use the documentation's toy inputs. The maps, checks, thresholds, scores and error messages are the same on every run. The releases change, because OpenDP's noise cannot be seeded, so they are in one block marked as one run.

```
uv run python code/part03-framework/opendp_measurements.py
```

## 10.1. Additive noise

The simplest measurement adds a random draw to a number. The constructor `make_laplace` adds Laplace noise and counts privacy in the budget $\varepsilon$ of pure differential privacy (DP), under the output measure `MaxDivergence`. The constructor `make_gaussian` adds Gaussian noise and counts privacy in the budget $\rho$ of zero-concentrated differential privacy (zCDP), under `ZeroConcentratedDivergence`. Both use the same input space.

```
def scalar_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Laplace and Gaussian noise on one float; the input space is the same, the output
     ↪  measures differ."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    laplace = space >> dp.m.then_laplace(scale=scale)  # output measure: MaxDivergence,
 ↪  read as epsilon
    gaussian = space >> dp.m.then_gaussian(scale=scale)  # output measure:
 ↪  ZeroConcentratedDivergence, read as rho
    return laplace, gaussian
```

The Laplace map is the sensitivity divided by the scale, so a unit sensitivity at unit scale costs $\varepsilon = 1$. The Gaussian map is the squared sensitivity over twice the squared scale, so the same

inputs give $\rho = 0.5$. The `check` method compares the map with a budget, so a sensitivity of 2 fails the budget that a sensitivity of 1 fits.

```
laplace float: map(1.0)=1.0 measure=MaxDivergence
gaussian float: map(1.0)=0.5 measure=ZeroConcentratedDivergence
laplace float: check(1.0, 1.0)=True check(2.0, 1.0)=False
```

The two currencies promise different things. Pure DP with parameter $\varepsilon$ says that one person changes the probability of any release by a factor of at most $e^{\varepsilon}$. Zero-concentrated DP with parameter $\rho$ bounds the privacy loss on average and in its tails at once, which suits the thin tails of the Gaussian. Combinators converts $\rho$ into $\varepsilon$ with a slack $\delta$, and Assessing utility and choosing the noise compares the two. The full edition's chapter on the Laplace and Gaussian mechanisms derives both maps.

### 10.1.1. Integers, floats and vectors

The same constructors accept integers. On an integer domain, `make_laplace` draws from the discrete Laplace distribution and `make_gaussian` from the discrete Gaussian, so the release is an integer. The maps do not change.

```
laplace int: map(1)=1.0 output type=int
gaussian int: map(1)=0.5
```

A vector input changes the metric. Laplace noise needs a bound on the L1 distance, the sum of the absolute differences, because its map adds the cost of each coordinate. Gaussian noise needs a bound on the L2 distance, the square root of the summed squares, because its cost adds in squares. The constructor refuses a space with the wrong metric.

```
def vector_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """The same two mechanisms on a vector of floats: Laplace wants an L1 bound,
     ↪  Gaussian an L2 bound."""
    domain = dp.vector_domain(dp.atom_domain(T=float, nan=False))
    laplace = (domain, dp.l1_distance(T=float)) >> dp.m.then_laplace(scale=scale)
    gaussian = (domain, dp.l2_distance(T=float)) >> dp.m.then_gaussian(scale=scale)
    return laplace, gaussian
```

So doubling the L2 sensitivity multiplies $\rho$ by four, while the Laplace map stays linear. A plain float given to a vector mechanism is a type error, and the message says what the mechanism expected.

```
laplace vector: metric=L1Distance(f64) map(1.0)=1.0
gaussian vector: metric=L2Distance(f64) map(1.0)=0.5
gaussian vector: map(2.0)=2.0 map(4.0)=8.0
laplace vector on a scalar: Expected type is Vec<f64> but input data is not a list.
gaussian on an L1 space: No match for concrete type L1Distance<f64>. See
 ↪  https://github.com/opendp/opendp/discussions/451.
```

### 10.1.2. Why OpenDP rounds, and the parameter `k`

A computer cannot sample a continuous distribution, and with floating-point noise the gaps between floats can leak the input. So OpenDP turns a float input into an exact rational, adds discrete noise to its numerator, and rounds the result back to a float. This equals true continuous noise followed by rounding, which is post-processing, so the analysis stays exact.

The grid of that rational is $2^k$, where `k` is a parameter of the constructor. The default `k` is the smallest exponent of a 64-bit float, so the grid is exact and costs nothing. A coarser grid samples faster, but rounding to it can move two neighbouring inputs one grid step further apart. So the map adds that step to the sensitivity: a grid of a half adds a half to every map, even at zero sensitivity.

```
laplace k=-1074: map(0.0)=0.0 map(1.0)=1.0
laplace k=-1: map(0.0)=0.5 map(1.0)=1.5
laplace k=0: map(0.0)=1.0 map(1.0)=2.0
laplace k=1: map(0.0)=2.0 map(1.0)=3.0
laplace k on integers: k is only valid for domains over floats
```

Integers need no grid, so the constructor rejects `k` on an integer domain. Change `k` only if a vector release is too slow, and then read the map again. The full edition's chapter on discrete mechanisms explains why the discrete distributions are safe.

### 10.1.3. The geometric mechanism and `make_noise`

The constructor `make_geometric` is the Laplace mechanism on integers with one more option. With `bounds`, the sampler runs in constant time, which closes a timing side channel, and it clamps the release. It fails on floats, because the geometric distribution has no float version.

```
geometric = int_space >> dp.m.then_geometric(scale=1.0)
bounded = int_space >> dp.m.then_geometric(scale=1.0, bounds=(0, 10))  # runs in
 ↪  constant time; clamps the output
```

```
geometric: map(1)=1.0 measure=MaxDivergence
geometric bounds (0, 10): map(1)=1.0
geometric on floats: No match for concrete type f64. See
 ↪  https://github.com/opendp/opendp/discussions/451.
```

The general constructor behind the two is `make_noise`. You name the output measure, and the library picks the distribution for it: Laplace for the max divergence and Gaussian for the zero-concentrated divergence. Use it in code that lets the caller choose the currency.

```
noise_pure = space >> dp.m.then_noise(dp.max_divergence(), scale=1.0)  # the Laplace
 ↪  mechanism
noise_zcdp = space >> dp.m.then_noise(dp.zero_concentrated_divergence(), scale=1.0)  #
 ↪  the Gaussian mechanism
```

```
make_noise MaxDivergence: map(1.0)=1.0 measure=MaxDivergence
make_noise ZeroConcentratedDivergence: map(1.0)=0.5 measure=ZeroConcentratedDivergence
```

## 10.2. Canonical noise

The mechanisms above take a scale and give its cost. The canonical noise mechanism takes the sensitivity and the budget instead, and draws from the distribution that spends exactly that budget. The budget is approximate DP, which relaxes the pure guarantee with a slack $\delta$, the probability that the factor $e^{\varepsilon}$ fails. Its canonical distribution is the Tulap distribution, a discrete Laplace draw plus a uniform one, which leaves the most powerful hypothesis test possible at the budget. So use it when a release feeds a statistical test or a confidence interval.

```
def canonical_mechanism(d_in: float, epsilon: float, delta: float) -> dp.Measurement:
    """Canonical noise on one float, calibrated so that inputs at most d_in apart cost
     ↪  exactly (epsilon, delta)."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    return space >> dp.m.then_canonical_noise(d_in=d_in, d_out=(epsilon, delta))
```

The map is a step and not a line, because the calibration is fixed at construction. Up to the declared sensitivity, the map returns the declared pair, and above it the map refuses, and so does `check`.

```
canonical noise: measure=Approximate(MaxDivergence)
canonical noise: map(1.0)=(1.0, 1e-07) map(0.5)=(1.0, 1e-07)
canonical noise: check(1.0, (1.0, 1e-07))=True check(1.0, (0.9, 1e-07))=False
canonical noise map(2.0): d_in from the map (2) must be in [0, 1]
```

The installed version supports only a scalar float and an $(\varepsilon, \delta)$ budget. In the Context API, `query.canonical_noise()` with `binomial_size` gives confidence intervals and p-values for a binomial count, as the documentation's notebook shows.

## 10.3. Thresholded noise

When the keys of a histogram come from the data, noise alone does not protect them: if a category appears in the release, someone in the data belongs to it. So the thresholded noise mechanisms release only the keys whose noisy count passes a threshold. A key with many contributors passes on both of two neighbouring datasets, so its presence says nothing about one person. A key with one contributor passes only if its noise is very large, and the slack $\delta$ pays for that chance. Grouping used this mechanism through Polars.

The input domain is a map from keys to counts, and the input metric is a triple of bounds. They bound how many keys one person can touch, the total change across all keys, and the change to any one key. The total is in L1 for Laplace and in L2 for Gaussian.

```
def threshold_mechanisms(scale: float, threshold: float) -> tuple[dp.Measurement,
 ↪  dp.Measurement]:
    """Laplace and Gaussian noise on a dictionary of float counts, releasing only the
     ↪  keys whose noisy count passes."""
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=float, nan=False))
    laplace = dp.m.make_laplace_threshold(
        domain, dp.l01inf_distance(dp.absolute_distance(T=float)), scale=scale,
 ↪  threshold=threshold
    )
    gaussian = dp.m.make_gaussian_threshold(
        domain, dp.l02inf_distance(dp.absolute_distance(T=float)), scale=scale,
 ↪  threshold=threshold
    )
    return laplace, gaussian
```

In the documentation's example, a person touches one key by at most one. At scale 1, the Laplace version prices that at $\varepsilon = 1$ plus a $\delta$, the chance that a count one person can make passes the threshold by noise alone. The Gaussian version prices it in $\rho$ with a far smaller $\delta$, because its tails fall faster.

```
laplace threshold: metric=L01InfDistance(AbsoluteDistance(f64))
 ↪  measure=Approximate(MaxDivergence)
laplace threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(1.0, 2.8014e-09)
gaussian threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(0.5, 1.1102e-16)
gaussian threshold: measure=Approximate(ZeroConcentratedDivergence)
```

But a smaller $\delta$ is not a smaller cost, because the currencies differ. At the Laplace $\delta$, the Gaussian guarantee converts to an $\varepsilon$ about six times larger (Combinators). The Gaussian wins only when a person touches many keys by a little, as in the second line. Then the L2 bound is much smaller than the L1 bound.

```
gaussian threshold as approximate DP at delta=2.8014e-09: epsilon=6.3036
spread (100, 10.0, 0.001): laplace (0.1000, 1.0316e-07) gaussian epsilon=0.0500
```

In that line, the Laplace map charged a tenth, not ten. A hundred keys changed by at most a thousandth change the total by at most a tenth, so the library tightens the L1 bound to that product. Integer counts use the discrete distributions. The general `make_noise_threshold` takes the measure as an argument, and the installed version needs it wrapped as approximate, which its docstring does not say.

```
discrete laplace threshold scale 1 threshold 10: map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold Approximate(MaxDivergence): map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold MaxDivergence: output_measure must be MaxDivergence or
 ↪  ZeroConcentratedDivergence
```

### 10.3.1. Counts of ILOSTAT

The LFS codes the labour status of each response in ILOSTAT, with four codes in this sample. Suppose that you do not know which codes occur, so the keys must come from the data. The script counts the responses per code with Polars and gives the dictionary to a thresholded Laplace mechanism.

```
def ilostat_counts(frame: pl.LazyFrame) -> dict[int, int]:
    """The number of responses in each ILOSTAT category, keyed by the category: the
     ↪  input of the thresholded count."""
    counts = frame.group_by("ILOSTAT").len().sort("ILOSTAT").collect()
    return {int(key): int(count) for key, count in zip(counts["ILOSTAT"],
     ↪  counts["len"])}


def ilostat_mechanism(scale: float, threshold: int) -> dp.Measurement:
    """Discrete Laplace noise on integer counts keyed by an integer category, with a
     ↪  threshold on the noisy count."""
    domain = dp.map_domain(dp.atom_domain(T=int), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_laplace_threshold(domain, metric, scale=scale,
     ↪  threshold=threshold)
```

One person has at most 36 rows (the privacy unit). Without more public knowledge, each entry of the triple is 36, because a person could touch 36 keys once each, or one key 36 times. At a budget of 1, the scale is 36, with an error bar of about 108 responses. So the threshold is the only free choice, and it sets $\delta$. The script prices three thresholds and finds the smallest that keeps $\delta$ under one in ten million.

```
ILOSTAT counts: {1: 1517011, 2: 155661, 3: 1450283, 9: 688977}
ILOSTAT d_in: (36, 36, 36) scale: 36 95% accuracy: 108.3
ILOSTAT threshold 500: map=1.0, delta=4.4835e-05
ILOSTAT threshold 1000: map=1.0, delta=4.1667e-11
ILOSTAT threshold 1500: map=1.0, delta=3.9968e-15
ILOSTAT smallest threshold with delta <= 1e-07: 720
ILOSTAT threshold 10: threshold (10) must not be smaller than l-infinity sensitivity 36
```

Each $\delta$ is the chance that the noise lifts a count that one person can make alone, at most 36, over the threshold. It falls fast as the threshold rises. The mechanism refuses a threshold below the per-key sensitivity, because one person could then make a key with no noise. Here the smallest count is over 150,000, so a threshold of 720 drops nothing. The threshold removes only rare categories, which is where it must act. The full edition's chapter on sensitivity shows, under partition selection, why it costs a $\delta$.

## 10.4. The approximate Laplace projection

The approximate Laplace projection is another answer to unknown keys. It releases a hashed sketch of the whole key space, which you then query for any key, including keys that never

occurred. It lists no keys, so no key leaks, and queries after the release are post-processing. It has two costs: answers are rounded to multiples of `alpha`, four by default, and two frequent keys can collide in the hash unless the sketch is large enough. The size of the sketch grows with two public bounds. These are `total_limit` on the sum of all counts and `value_limit` on any one count.

```
def projection_mechanism(scale: float) -> dp.Measurement:
    """A private projection of counts keyed by strings; releases a queryable that
     ↪  answers for any key, known or not."""
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_alp_queryable(domain, metric, scale=scale, total_limit=50_000,
     ↪  value_limit=1_000)
```

Be careful: here `scale` is not a noise scale but the privacy loss per unit of sensitivity, so a larger value spends more, not less. The map at unit sensitivity equals the scale, and it grows with the L1 entry of the triple.

```
alp scale 1: map((1, 1, 1))=1.0 measure=MaxDivergence
alp scale 2: map((1, 1, 1))=2.0
alp scale 1: map((2, 2, 1))=2.0
```

The documentation finds the scale for a budget with `binary_search_chain` (Utilities and the Context API inside). On the toy table, the one-run answers are within a few units of the true counts. The name that is not in the table comes back near zero.

## 10.5. Noisy max and selection

Some releases are a choice: which category is largest, which candidate is the median. The noisy max mechanisms add noise to each score and release the index of the largest. Their input metric is the L-infinity distance, a bound on the change of any one score. That is the right bound, because the release depends on how the scores compare. The output measure chooses the noise: exponential under the max divergence, and Gumbel under the zero-concentrated divergence.

```
def noisy_max_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Select the index of the largest score: exponential noise under pure DP, Gumbel
     ↪  noise under zCDP."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.linf_distance(T=float)
    exponential = space >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale)
    gumbel = space >> dp.m.then_noisy_max(dp.zero_concentrated_divergence(),
 ↪  scale=scale)
    return exponential, gumbel
```

The exponential version's map is twice the sensitivity over the scale, because one person can raise one score and lower another at the same time. So at a scale of 2, a unit sensitivity costs $\varepsilon = 1$. If the scores can move only in one direction, declare the metric monotonic, which removes the factor of two. The Gumbel version's map is the square of the same ratio, divided by eight. Negated

scores select the smallest at no extra cost, and `make_noisy_top_k` releases the best $k$ indices for $k$ times the cost.

```
noisy max: metric=LInfDistance(T=f64) measure=MaxDivergence
noisy max exponential scale 2: map(1.0)=1.0
noisy max gumbel scale 2: map(1.0)=0.125 measure=ZeroConcentratedDivergence
noisy max monotonic scale 2: exponential map(1.0)=0.5 gumbel map(1.0)=0.03125
noisy min scale 2: map(1.0)=1.0
noisy top 2 scale 2: exponential map(1.0)=2.0 gumbel map(1.0)=0.25
```

The older `make_report_noisy_max_gumbel` still exists, but the installed version marks it deprecated. The full edition's chapter on selection shows why selection costs twice what a count costs, and why exponential noise beats Gumbel under pure DP.

```
report_noisy_max_gumbel scale 2: map(1.0)=1.0 measure=MaxDivergence
report_noisy_max_gumbel: Call to deprecated function (or staticmethod)
 ↪  make_report_noisy_max_gumbel
```

### 10.5.1. A private median of `HWUSUAL`

OpenDP releases a quantile by selection. The transformation `make_quantile_score_candidates` of the previous chapter scores each candidate, here the hours from 1 to 98, by how far it is from splitting the data in half. The best candidate has the smallest score, so a noisy max over negated scores selects it. A lookup in the candidate list then turns the index into an answer, as post-processing. The constructor `make_private_quantile` does the three steps in one call.

```
def hours_of_the_working(frame: pl.LazyFrame) -> list[float]:
    """HWUSUAL for the people at work (ILOSTAT 1), keeping 1 to 98 hours: 0 means
     ↪  varying, 99 not applicable."""
    at_work = (pl.col("ILOSTAT") == 1) & pl.col("HWUSUAL").is_between(1, 98)
    return frame.filter(at_work).select("HWUSUAL").collect()["HWUSUAL"].to_list()


def median_by_hand(scale: float) -> tuple[dp.Transformation, dp.Measurement]:
    """The private median from its pieces: score each candidate, then pick an index by
     ↪  noisy max."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    scores = space >> dp.t.then_quantile_score_candidates(candidates=HOURS, alpha=0.5)
    select = scores >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale,
 ↪  negate=True)  # the smallest score wins
    return scores, select  # the released index is looked up in HOURS afterwards, which
     ↪  is post-processing


def median_mechanism(scale: float) -> dp.Measurement:
    """The same median in one constructor."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    return space >> dp.m.then_private_quantile(dp.max_divergence(), candidates=HOURS,
     ↪  alpha=0.5, scale=scale)
```

The data are the usual hours of the people at work, without the codes for “varies” and “not applicable”. A row filter adds no rows, so the input distance is still 36. Each integer score moves by at most the input distance, so the scores’ L-infinity sensitivity is 36.

```
HWUSUAL at work, 1 to 98 hours: n=1,490,113 true median=37
quantile scores: metric=LInfDistance(T=u64) map(36)=36
best candidate: 37 hours, score 9,111
second candidate: 36 hours, score 95,652
gap in noise scales: 1,202
scores >> noisy max scale 72: map(36)=1.0
make_private_quantile scale 36: map(36)=1.0
make_private_quantile zCDP scale 36: map(36)=0.125
```

Two scales appear because `make_private_quantile` doubles the scale you give it. The scores of a median are integers multiplied by two, the denominator of one half. So the constructor multiplies the scale by the same two, and the two maps agree at $\varepsilon = 1$. Read the scores before you trust the release. The best candidate beats the next by over a thousand noise scales, so the release is 37 hours on almost every run. On a smaller dataset the gaps are smaller, and the release moves among nearby candidates, which is the error of a private quantile.

## 10.6. Randomized response

All the mechanisms so far run on a trusted server that holds the data. Randomized response runs on the respondent’s device before the answer is sent, so no server sees the truth. This is the local model of DP, where the privacy unit is one answer. Its metric is the discrete distance, which is 1 between two different inputs and 0 otherwise. OpenDP gives only the randomizers, not the transport or the aggregation.

```
def randomized_response_mechanisms(prob: float) -> tuple[dp.Measurement,
 ↪  dp.Measurement]:
    """Randomized response on one yes/no answer and on one ILOSTAT category, each kept
     ↪  truthful with probability prob."""
    boolean = dp.m.make_randomized_response_bool(prob=prob)
    categorical = dp.m.make_randomized_response(categories=[1, 2, 3, 9], prob=prob)
    return boolean, categorical
```

The boolean version keeps the true answer with the probability you give and flips it otherwise. Its map is the logarithm of the odds of the truth, so a probability of three quarters costs about 1.1. It refuses a probability below one half. The categorical version keeps the truth with the same probability and otherwise picks one of the other categories at random. Its map grows with the logarithm of the number of alternatives, so four `ILOSTAT` codes cost twice what a yes/no answer costs.

```
randomized response bool: domain=AtomDomain(T=bool) metric=DiscreteDistance()
 ↪  measure=MaxDivergence
randomized response bool prob 0.75: map(1)=1.0986 ln(0.75/0.25)=1.0986
randomized response bool prob 0.9: map(1)=2.1972
randomized response bool prob 0.4: probability must be within [0.5, 1]
randomized response 4 categories prob 0.75: map(1)=2.1972 ln(3*0.75/0.25)=2.1972
```

One randomized answer is almost useless by design. The answers become useful in aggregate, after a correction for the known flip rate. The third randomizer, for a bit vector, is the primitive behind RAPPOR, the first large deployment of local DP. Its domain bounds how many bits can be set, and it rerandomizes each bit with probability `f`. Its map is twice that weight times the log odds that a bit survives.

```
def bitvector_mechanism(f: float, max_weight: int) -> dp.Measurement:
    """Randomized response on a bit vector with at most max_weight ones, each bit
     ↪  rerandomized with probability f."""
    return dp.m.make_randomized_response_bitvec(dp.bitvector_domain(max_weight=
     ↪  max_weight), dp.discrete_distance(), f=f)
```

```
bit vector f=0.8 max_weight=3: domain=BitVectorDomain(weight=3) map(1)=2.4328
 ↪  formula=2.4328
```

The script sends the documentation's 24-bit vector through the randomizer ten thousand times and runs `debias_randomized_response_bitvec` on the answers. The three set bits come back near ten thousand and the others near zero, with a spread of a few hundred. So local DP costs much accuracy (the full edition's chapter on trust models).

Here are the releases of one run. The additive releases are within a few units of 100, the thresholded count kept all four codes, the projection answered the unknown name with zero, and both medians are 37.

```
released (one run):
  laplace on 100.0: 104.05
  gaussian on 100.0: 100.26
  discrete laplace on 100: 100
  laplace vector on [0.0, 2.0, 2.0]: [-1.4, 1.17, 4.28]
  laplace k=-1 on 100.0, five draws: [99.5, 99.5, 102.5, 101.5, 100.0]
  bounded geometric on 10, five draws: [10, 10, 10, 9, 7]
  canonical noise on 100.0: 99.74
  laplace threshold on {a: 0, b: 20, c: 40}: {'c': 37.74439496773442}
  ILOSTAT counts: {1: 1517058, 2: 155697, 3: 1450247, 9: 689013}
  alp answers for Michael, James, Sharon, Lancelot: [748.0, 500.0, 48.0, 0.0]
  noisy max on [0, 2, 4, 6]: 3
  noisy min on [0, 2, 4, 6]: 0
  noisy top 2 on [0, 2, 4, 6]: [3, 1]
  median by hand: 37 hours
  median by make_private_quantile: 37 hours
  randomized response on True, ten draws: [True, True, False, True, True, False, False,
 ↪  False, True, True]
```

```
randomized response on category 1, ten draws: [1, 1, 1, 1, 1, 1, 1, 2, 1, 3]
bit vector release: 10011011100000010001110
bit vector frequencies over 10,000 copies, bits 12, 20, 21: [10050, 10235, 10115]
bit vector frequencies, largest other bit: 565
```

## 10.7. Summary

Table 10.1.: The measurements of the chapter: their input metric, their output measure, the map the script printed at scale 1 for inputs at distance 1, and what each is for.

| Constructor | Input metric | Output measure | Map at scale 1 for inputs at distance 1 | What it is for |
|---|---|---|---|---|
| `make_laplace` | absolute or L1 distance | `MaxDivergence` | $\varepsilon = 1$ | One number or a vector under pure DP; integers get the discrete Laplace |
| `make_gaussian` | absolute or L2 distance | `ZeroConcentratedDivergence` | $\rho = 0.5$ | The same under zCDP; vectors cost by the L2 bound |
| `make_geometric` | absolute distance on integers | `MaxDivergence` | $\varepsilon = 1$ | Integer Laplace with `bounds` for constant time |
| `make_noise` | as above | the one you pass | as above | Lets the caller choose the currency |
| `make_canonical_noise` | absolute distance | `Approximate(MaxDivergence)` | the one you declared, up to your `d_in` | Exact $(\varepsilon, \delta)$ calibration; inference on counts |
| `make_laplace_threshold` | L0, L1, L-infinity triple | `Approximate(MaxDivergence)` | $(1.2801, 10^{-9})$ at threshold 20 | Counts over keys from the data; $\delta$ pays for the threshold |
| `make_gaussian_threshold` | L0, L2, L-infinity triple | `Approximate(ZeroConcentratedDivergence)` | $(0.5, 1.1102 \times 10^{-16})$ at threshold 20 | The same when a person touches many keys by a little |
| `make_alp_queryable` | L0, L1, L-infinity triple | `MaxDivergence` | $\varepsilon = 1$ (the scale is a loss, not a noise) | Queries for any key after the release |

| Constructor | Input metric | Output measure | Map at scale 1 for inputs at distance 1 | What it is for |
|---|---|---|---|---|
| `make_noisy_max` | L-infinity distance | `MaxDivergence` or `ZeroConcentratedDivergence` | $\varepsilon = 2$ or $\rho = 0.5$ | Select an index by score; monotonic halves the cost |
| `make_noisy_top_k` | L-infinity distance | either | $\varepsilon = 4$ for $k = 2$ | The best $k$ indices, at $k$ times the cost |
| `make_private_quantile` | sym metric distance | either | $\varepsilon = 1$ | Scores plus noisy max plus lookup in one call |
| `make_randomized_response_bool` | discrete distance | `MaxDivergence` | 1.099 at probability 0.75 | One yes/no answer on the respondent's device |
| `make_randomized_response` | discrete distance | `MaxDivergence` | 2.197 for four categories at 0.75 | One category; the cost grows with the alternatives |
| `make_randomized_response_bitvec` | discrete distance | `MaxDivergence` | 2.433 for weight 3 at `f` = 0.8 | The RAPPOR primitive; debias the aggregate |

## 10.8. References


- OpenDP documentation: "Measurements", "Additive Noise Mechanisms", "Canonical Noise Mechanism", "Thresholded Noise Mechanisms", "Approximate Laplace Projection", "Report Noisy Max Mechanisms" and "Randomized Response" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the discrete Gaussian and the discrete Laplace samplers OpenDP uses.
- Awan and Vadhan, "Canonical Noise Distributions and Private Hypothesis Tests" (2023): the Tulap distribution.
- Rogers, "A Unifying Privacy Analysis Framework for Unknown Domain Algorithms in Differential Privacy" (2023): the thresholded mechanisms' maps.
- Aumüller, Lebeda and Pagh, "Differentially Private Sparse Vectors with Low Error, Optimal Space, and Fast Access" (2021): the approximate Laplace projection.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the exponential-noise noisy max.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (2011): the quantile by selection.

- Warner, “Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias” (1965), and Erlingsson, Pihur and Korolova, “RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response” (2014).
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, selection in the full edition, sensitivity and partition selection in the full edition, discrete mechanisms in the full edition and trust models in the full edition derive the results this chapter states.

# 11. Combinators

A real release is a clamp, then a sum, then noise, then a second query that depends on the first. It is reported as one guarantee of differential privacy (DP). The **combinators** join such pieces into bigger ones. Each takes objects of the framework and returns a new one, with a privacy map that gives its cost. So you never add budgets by hand, and the combinator refuses to join pieces whose promises do not fit. At the end of the chapter, you can read the privacy map of any chain and say why a context refuses a query. You can also convert a budget between privacy measures.

The script opendp_combinators.py runs every step on OpenDP 0.16.0. As in the documentation, its toy data are twenty seeded integers from 0 to 10, with a Laplace count and a Laplace sum of the clamped values. One example runs on the Labour Force Survey (LFS) sample. The maps, profiles, scales and error messages are the same on every run, but the releases change, because OpenDP's noise cannot be seeded. So they are in one block marked as one run.

```
uv run python code/part03-framework/opendp_combinators.py
```

## 11.1. Chaining

The simplest way to combine two pieces is to run one after the other. The constructor `make_chain_tt` joins two transformations, `make_chain_mt` puts a measurement after a transformation, and `make_chain_pm` attaches a post-processing function to a measurement. There is no chainer for two measurements, because a computation after a measurement is post-processing, which needs no accounting. The script first builds the clamped sum and its Laplace release with these long forms.

```
def chained_by_hand() -> tuple[dp.Transformation, dp.Measurement]:
    """The clamped sum and its Laplace release, built with the long-form chainers."""
    clamp = INT_SPACE >> dp.t.then_clamp(BOUNDS)
    total = dp.t.make_sum(clamp.output_domain, clamp.output_metric)
    clamped_sum = dp.c.make_chain_tt(total, clamp)  # a transformation: clamp, then sum
    laplace = dp.m.make_laplace(clamped_sum.output_domain, clamped_sum.output_metric,
↪   scale=SUM_SCALE)
    noisy_sum = dp.c.make_chain_mt(laplace, clamped_sum)  # a measurement: the chain,
↪   then noise
    return clamped_sum, noisy_sum
```

Each call takes the outer piece first and the inner piece second, as in function composition. The chained transformation has a stability map (Transformations). One added or removed record changes the clamped sum by at most the width of the bounds, so the map at distance 1 is 10. The chained measurement has a privacy map (Measurements), which is that stability over the noise scale. So Laplace noise of scale 5 on the sum costs a privacy loss of $\varepsilon = 2$.

```
clamp >> sum: map(1)=10
clamp >> sum >> laplace: map(1)=2.0 check(1, 2.0)=True
with >>: map(1)=2.0
```

The `>>` operator chooses the right chainer for you, and with a `then_` constructor on its right you never repeat the input space. Here is the same measurement in one line.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
noisy_sum = space >> dp.t.then_clamp((0, 10)) >> dp.t.then_sum() >>
 ↪  dp.m.then_laplace(scale=5.0)
print(noisy_sum.map(1))  # 2.0
print(noisy_sum.input_domain)  # VectorDomain(AtomDomain(T=i32))
print(noisy_sum.output_measure)  # MaxDivergence
```

**Post-processing** is any computation on a release that does not touch the data again. It costs nothing, because what an adversary could learn from the processed value, they could learn from the release itself (the full edition's chapter on properties). In OpenDP you attach a Python function to a measurement with `>>`, and the result has the same privacy map.

```
def divide_by_size(total: int) -> float:
    return total / N  # the released sum over the public size: a mean, at no further
     ↪  cost

mean_of_sum = shortcut >> divide_by_size
```

```
post-processed: map(1)=2.0
```

In version 0.16.0 this needs a second feature flag, `honest-but-curious`, because the library cannot inspect a Python function. The script switches it on just before the chain and off just after. A post-processor must not read global state, such as the clock, or raise an exception that depends on the data. Either opens a channel that the privacy map does not count.

A chain forms only when the pieces fit: the output domain and metric of the inner piece must equal the input domain and metric of the outer one. The script gives a sum of floats to a Laplace mechanism built for integers, and the library refuses.

```
try:
    chain = float_sum >> int_laplace  # a float sum feeding a Laplace mechanism on
 ↪  integers
    domain_error = f"chained: {chain}"
except dp.OpenDPException as err:
    domain_error = str(err.message)
```

```
domain mismatch: Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: AtomDomain(T=f64)
    input_domain:  AtomDomain(T=i32)
metric mismatch: Intermediate metrics don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_metric: AbsoluteDistance(i32)
    input_metric:  AbsoluteDistance(f64)
```

The second message comes from a count, whose output is an integer distance, before a Laplace mechanism that measures distances in floats. So read the two lines of the error and insert the cast, clamp or impute step that makes them equal.

Here are the error bars of the sum and the count that the rest of the chapter composes (Assessing utility and choosing the noise explains them).

```
sum: scale=5 95% accuracy=15.4537
count: scale=1 95% accuracy=3.3756
```

## 11.2. Non-adaptive composition

When you run the sum and the count on the same data, you spend both budgets. **Basic composition** says that the total is the sum of the parts. So an $\varepsilon_1$-DP release and an $\varepsilon_2$-DP release on the same data are together $(\varepsilon_1 + \varepsilon_2)$-DP (the full edition's chapter on properties). The combinator `make_composition` builds that rule into one measurement, which runs every part and returns the releases as a list.

```
both = dp.c.make_composition([shortcut, count])  # one measurement that releases the
 ↪  sum and the count
print(f"composition: map(1)={both.map(1):.1f}  parts: {shortcut.map(1):.1f} +
 ↪  {count.map(1):.1f}")
```

```
composition: map(1)=3.0  parts: 2.0 + 1.0
composition output measure: MaxDivergence
```

The parts must agree on their input domain, input metric and output measure. A composition is a statement about one dataset, one notion of neighbours and one measure of privacy. It is non-adaptive, because every part is fixed before the data are seen.

The same combinator composes measurements under other measures, with each measure's own rule. **Zero-concentrated differential privacy (zCDP)** measures privacy by one parameter $\rho$, which bounds the Rényi divergence between the output distributions on neighbours at every order. Under zCDP the parameters add like $\varepsilon$ (the full edition's chapter on concentrated DP). The Gaussian sum at scale 5 has $\rho = 2$, the Gaussian count at scale 1 has $\rho = 0.5$, and the composition reports their sum.

```
zCDP composition: map(1)=2.50  parts: 2.00 + 0.50
mixed measures: All output measures must be the same
```

The second line shows that you cannot mix measures, because the maps of a Laplace count and a Gaussian count return numbers that mean different things. Section 11.7 shows how to bring them to one measure first.

## 11.3. Adaptive composition

Often you choose the second query only after you see the first answer. Composition still holds when each query depends on the earlier answers, if the budget of each is fixed in advance (the full edition's chapter on composition). The combinator for this is `make_adaptive_composition`, which needs more arguments, because it cannot read the supporting elements from parts it has not seen.

```
def adaptive_compositor(d_mids: list[float]) -> dp.Measurement:
    """An adaptive compositor over the toy space: one share of pure-DP budget per
     ↪  query, fixed in advance."""
    return dp.c.make_adaptive_composition(
        input_domain=INT_SPACE[0],
        input_metric=INT_SPACE[1],
        output_measure=dp.max_divergence(),
        d_in=1,  # neighbours add or remove one record
        d_mids=d_mids,  # the epsilon each query may spend, in the order the queries
 ↪  arrive
    )
```

The argument `d_in` is the distance between neighbouring datasets, here 1 for one added or removed record. The argument `d_mids` lists the privacy loss that each query may spend, in order, and the compositor's privacy map is their sum, here 3.

```
adaptive: map(1)=3.0
```

When you invoke the compositor on the data, it releases nothing. It returns a **queryable**, a small state machine that holds the data and the remaining shares. It accepts a measurement as a query, checks that the measurement's map at `d_in` fits within the next share, and answers with the release. The script submits the sum, then the sum again, then the count.

```
queryable = compositor(data)  # the data go in once; what comes back answers
 ↪  measurements
adaptive_sum = queryable(shortcut)  # spends the first share, 2.0
try:
    queryable(shortcut)  # asks 2.0 of the second share, 1.0
    over_share = "answered"
except dp.OpenDPException as err:
    over_share = str(err.message)
adaptive_count = queryable(count)  # spends the second share, 1.0
```

```
second query over its share: insufficient budget for query: 2.0 > 1.0
third query: out of queries
```

The shares are spent in order, so the queryable refuses a query larger than the next share, without spending it, and after the second share it refuses every query. A context wraps this object. With `split_evenly_over=3` in A first release, the library built an adaptive compositor with three equal shares, the context's `accountant`. Each `release()` submits to the queryable, and the context's refusal is the "out of queries" above.

On real data, the number of shares sets the noise of every query. The script builds two contexts over the LFS sample, one with the whole budget of $\varepsilon = 1$ on one query and one split evenly over three. Both ask for the sum of `HWUSUAL`, the hours usually worked per week, without the code 99 for "not applicable" and bounded to 0 to 80 hours. One person has at most 36 rows (the privacy unit), so the sum moves by up to 36 times the upper bound.

```
LFS context over 1: accountant map(36)=1.0 d_mids=[1.0]
LFS HWUSUAL sum over 1: scale=2,881.04 95% accuracy=8,630.8
LFS context over 3: accountant map(36)=1.0 d_mids=[0.3333, 0.3333, 0.3333]
LFS HWUSUAL sum over 3: scale=8,643.11 95% accuracy=25,892.4
true HWUSUAL sum: 56,315,594
```

Both accountants map the privacy unit's distance to 1, because the shares add up to the declared budget. But the scale is the sensitivity over the share, so three shares triple the scale and the error bar. An error bar of 26 thousand is small on 56 million hours, but not on a small group. The scale is slightly above the sensitivity over the share because of the float sum's rounding term (Transformations).

## 11.4. Odometers

Adaptive composition fixes the budget of each query before the first answer, which is sometimes too early. **Fully adaptive composition** lets you choose the privacy loss of each query as you go. An **odometer** tracks the running total of the loss spent, valid at whatever point you stop. The full edition's chapter on adaptive composition shows why the ordinary theorems fail with adaptively chosen parameters, and which totals are valid. For pure DP the sum of the losses spent is valid, and OpenDP's odometer reports it.

```
odometer = dp.c.make_fully_adaptive_composition(
    input_domain=INT_SPACE[0], input_metric=INT_SPACE[1],
 ↪  output_measure=dp.max_divergence()
)
odometer_queryable = odometer(data)
loss_before = odometer_queryable.privacy_loss(1)  # nothing spent yet
odometer_sum = odometer_queryable(shortcut)
loss_after_sum = odometer_queryable.privacy_loss(1)
odometer_count = odometer_queryable(count)
loss_after_count = odometer_queryable.privacy_loss(1)
```

The constructor takes the three supporting elements of the adaptive compositor, but no `d_in` and no `d_mids`. It returns an `Odometer`, not a `Measurement`, with no privacy map, because it promises nothing about the total. On the data it gives an **odometer queryable**, which answers measurements and reports `privacy_loss(d_in)`, the total spent so far at that input distance.

```
odometer loss: before=0.0 after sum=2.0 after count=3.0 after another count=4.0
odometer loss at d_in=2 after three queries: 8.0
```

The total starts at zero, grows by the map of each query, and never refuses anything. At distance 2, each Laplace query costs twice as much, so the total doubles. So the odometer says how much you spent, and the decision to spend more is yours. That suits open exploration, but nothing stops the total from passing what you may claim, and the next combinator adds that stop.

## 11.5. Privacy filters

A **privacy filter** is an odometer with a cap: it refuses any query that would take the running total past a bound fixed in advance. Since the total can never pass the bound, the filter is a measurement with an ordinary privacy map. For pure DP, a cap on the sum of the realized losses is a valid guarantee (the full edition's chapter on adaptive composition). The combinator `make_privacy_filter` takes the odometer, the input distance and the cap.

```
privacy_filter = dp.c.make_privacy_filter(odometer, d_in=1, d_out=2.0)
filter_queryable = privacy_filter(data)
filter_count_1 = filter_queryable(count)
filter_count_2 = filter_queryable(count)
try:
    filter_queryable(count)  # would take the loss to 3.0
    refusal = "answered"
except dp.OpenDPException as err:
    refusal = str(err.message)
```

```
filter: map(1)=2.0
filter third query: filter is now exhausted: pending privacy loss (3.0) would exceed
 ↪  privacy budget (2.0)
filter loss after refusal: filter is exhausted: no more queries can be answered
```

The filter's map at distance 1 is the cap, whatever queries arrive later, so the filter is a measurement. Two counts at a cost of 1 each fit under a cap of 2. The filter refuses the third with a message that names the pending total and the budget. After one refusal, the installed version treats the filter as finished, and even `privacy_loss` raises an error. So read the running total before the query that can exhaust it.

A context wraps a filter when you give it a budget but no split. The script builds a context on the toy data with `dp.loss_of(epsilon=2.0)` and no split. Its queryable is an odometer queryable behind a filter, and `current_privacy_loss` reads the odometer. The count query needs an explicit scale, because there is no share to fit.

```
context without a split: queryable=OdometerQueryable spent=1.0 remaining=1.0
```

## 11.6. Chaining and nesting compositors

Compositors and filters are measurements, so you can chain and compose them too. In the first common pattern, a transformation before a compositor does the preprocessing once for all queries. The script chains the clamped sum before an adaptive compositor, so that the queries are measurements on the sum, one integer.

```
sum_compositor = clamped_sum_only >> dp.c.then_adaptive_composition(
    output_measure=dp.max_divergence(),
    d_in=clamped_sum_only.map(1),  # the sum moves by at most 10 when one record
 ↪  changes
    d_mids=[2.0, 1.0],
)
sum_space = clamped_sum_only.output_domain, clamped_sum_only.output_metric
laplace_on_sum = sum_space >> dp.m.then_laplace(scale=SUM_SCALE)  # a query on the sum,
 ↪  not on the vector
```

```
transformation >> compositor: d_in=10 map(1)=3.0
laplace on the sum: map(10)=2.0
```

Set `d_in` with care. The compositor comes after the sum, so its input distance is the sum's stability map at the privacy unit's distance, here 10. The `then_` form takes the domain and metric from the transformation but cannot know the distance, so compute it with `map`. A Laplace mechanism of scale 5 on the sum then costs 2, which fits the first share.

In the second pattern, a compositor is inside a compositor. The inner compositor is a measurement whose map is the sum of its shares. So you can submit it to an outer one, which answers with the inner queryable.

```
outer = adaptive_compositor(d_mids=[3.0, 1.0])
inner = adaptive_compositor(d_mids=[2.0, 1.0])  # a measurement costing 3.0: it fits
 ↪  the outer's first share
outer_queryable = outer(data)
inner_queryable = outer_queryable(inner)  # the answer to a compositor query is a
 ↪  queryable
inner_sum = inner_queryable(shortcut)
outer_count = outer_queryable(count)  # the outer's second share, while the inner
 ↪  queryable stays open
inner_count = inner_queryable(count)  # pure DP composes concurrently, so the inner one
 ↪  still answers
```

```
nested: outer map(1)=4.0 inner map(1)=3.0
```

The outer compositor charges the inner map, 3, whatever the inner one answers later. The script asks the inner queryable for the sum, the outer one for the count, and the inner one again for the

count. This works because pure DP supports concurrent composition, where several queryables from one compositor stay open at once. For approximate zCDP and approximate Rényi differential privacy (RDP), post-processing is not proved to hold while an analyst can still interact. So there the library freezes an earlier queryable when a new query arrives. The documentation nests a zCDP compositor inside an approximate-DP one, after it casts the inner measure.

## 11.7. Measure casting

A composition needs all of its parts to have the same output measure. But a Laplace release is measured in pure DP, a Gaussian release in zCDP, and a report usually needs a privacy loss $\varepsilon$ with a slack $\delta$, the $(\varepsilon, \delta)$ of approximate DP. The casting combinators move a measurement from one measure to another, each by a conversion that the full edition derives.

The first conversion goes from pure DP to zCDP. An $\varepsilon$-DP mechanism is $\rho$-zCDP with $\rho = \varepsilon^2/2$ (the full edition's chapter on concentrated DP), and `make_pureDP_to_zCDP` applies that to the map. The combinator `make_approximate` turns a pure-DP measurement into an approximate-DP one with a slack of zero, so that it can compose with measurements that have a real $\delta$.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(dp.c.make_pureDP_to_zCDP(laplace).map(1.0))  # 0.5
print(dp.c.make_approximate(laplace).map(1.0))  # (1.0, 0.0)
```

You will use the conversion from zCDP to $(\varepsilon, \delta)$ most, because that is how a Gaussian release is reported. A $\rho$-zCDP mechanism satisfies approximate DP at every slack at once, with a loss that grows as the slack shrinks. So the conversion gives the **privacy profile**, a curve that gives the smallest privacy loss for each slack. The combinator `make_zCDP_to_approxDP` returns a measurement whose map is that curve, with the methods `epsilon(delta)` and `delta(epsilon)`. The script reads it at a slack of one in ten million, for a Gaussian with $\rho = 0.1$, which is the scale $1/\sqrt{2\rho}$ at unit sensitivity.

```
gaussian = FLOAT_SPACE >> dp.m.then_gaussian(scale=math.sqrt(1 / (2 * rho)))  # rho = 1
 ↪  / (2 scale^2)
profile_measurement = dp.c.make_zCDP_to_approxDP(gaussian)
profile = profile_measurement.map(1.0)  # a curve, not a number
epsilon_at_delta = profile.epsilon(delta=delta)
delta_at_one = profile.delta(epsilon=1.0)
fixed = dp.c.make_fix_delta(profile_measurement, delta=delta)  # the curve at one
 ↪  delta: an (epsilon, delta) pair
```

```
gaussian: rho=0.1000
profile at rho=0.1: epsilon(delta=1e-07)=2.3484 delta(epsilon=1)=0.008933
simple conversion: 2.6391  improved conversion: 2.3484
fix_delta: map(1.0)=(2.3484, 1e-07)  measure=Approximate(MaxDivergence)
```

So a budget of $\rho = 0.1$ is $(2.35, 10^{-7})$-DP. The third line compares that with the two conversions of the full edition. The simple one is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{11.1}$$

which gives 2.64 here. The improved conversion of Canonne, Kamath and Steinke minimizes a tighter expression over the Rényi order, and the script's numerical minimum agrees with OpenDP to four decimals. So OpenDP uses the improved conversion, which matters when you compare its numbers with a paper that quotes the simple one. The full edition's chapter on RDP conversions shows where the simple formula loses.

A curve cannot compose with a pair, so `make_fix_delta` evaluates the curve at one $\delta$ and returns a measurement whose map is the pair. Its output measure is the approximate max divergence, as from `make_approximate`, so the two can compose. The last casting combinator, `make_fixed_approxDP_to_approxDP`, turns a fixed pair back into a curve. For a pure-DP release cast with `make_approximate`, that curve is flat, because $\varepsilon$-DP at zero slack holds at every slack.

```
approximate: map(1.0)=(1.0, 0.0)  measure=Approximate(MaxDivergence)
fixed to profile: epsilon(delta=1e-07)=1.0 epsilon(delta=0)=1.0
laplace count + gaussian count: map(1)=(6.6709, 1e-07)
```

The last line shows the result. The Laplace count, which Section 11.2 could not compose with the Gaussian count, is cast with `make_approximate`. The Gaussian count is cast to a curve and fixed at the same slack, and then `make_composition` accepts the two. The total is large, because a Gaussian count with $\rho = 0.5$ is expensive at so small a slack: casting makes a composition legal, not cheap. When the Gaussian parts dominate, account the whole release in zCDP and convert once at the end, as the practitioner edition's release workflow does.

## 11.8. Amplification by subsampling

The last combinator changes what a privacy map promises without changing the computation. Suppose that your dataset is a simple random sample of $n$ people from a population of $N$, and nobody knows who is in the sample. Then a person may not be in the sample, which makes the release more private with respect to the population. The full edition's chapter on subsampling derives this amplification and shows why the sampling scheme must match the neighbouring relation. Under pure DP the amplified loss is

$$\varepsilon' = \ln\Big(1 + \frac{n}{N}\,(e^{\varepsilon} - 1)\Big). \tag{11.2}$$

The script prints it next to the library's map. The combinator `make_population_amplification` needs a measurement whose input domain has the sample size, because the ratio of the two sizes is part of the guarantee.

```
def sample_mean() -> dp.Measurement:
    """The Laplace mean of a sample of exactly SAMPLE values in [0, 10]: the domain
     ↪  carries the size."""
    space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0), nan=False),
 ↪  size=SAMPLE), dp.symmetric_distance()
```

```
    return space >> dp.t.then_mean() >> dp.m.then_laplace(scale=0.5)


def amplified_mean(population_size: int) -> dp.Measurement:
    """The same mean, with its guarantee read for the population the sample was drawn
     ↪ from."""
    dp.enable_features("honest-but-curious")  # the library cannot check that the data
 ↪ are such a sample
    return dp.c.make_population_amplification(sample_mean(),
     ↪ population_size=population_size)
```

```
sample mean: map(2)=2.0000
amplified, population 100: map(2)=0.4940  formula=0.4940
amplified, population 1000: map(2)=0.0619  formula=0.0619
amplifying an unsized domain: elements of the vector domain have unknown size
```

The maps are read at distance 2, because two datasets of the same size differ by a swap of one record. A swap moves the mean of ten values in a range of 10 by 1, so Laplace noise of half that costs $\varepsilon = 2$. For a population of a hundred, the release costs about 0.49, and for a thousand, about 0.06, as the formula also gives. The library refuses a vector domain without a size.

The combinator needs the `honest-but-curious` flag, for the same reason as the Python post-processor: the library cannot verify the claim. The guarantee holds only if the data are a uniform sample of a population of the stated size. It also needs that the readers of the release do not know who is in the sample. So use the map without amplification when the sample depends on the people in it, or when the sampled identities are public.

## 11.9. Summary

Table 11.1.: The combinators of this chapter, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `make_chain_tt`, `make_chain_mt`, `make_chain_pm` | Join two transformations, a transformation and a measurement, or a measurement and a post-processor | Write `>>`; it picks the right one |
| `measurement >> python_function` | Post-processing, at no cost | Needs `honest-but-curious` in 0.16.0; no global state, no data-dependent exceptions |
| `DomainMismatch`, `MetricMismatch` | A chain whose intermediate domain or metric differs | Read the two printed lines and insert the step that makes them equal |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `make_composition([m1, m2])` | One measurement that releases every part; its map is the sum of the parts' maps | Parts share domain, metric and measure; $\rho$ adds like $\varepsilon$ |
| `make_adaptive_composition(..., d_in, d_mids)` | A queryable that answers measurements one at a time, each within the next share | What a context with a split wraps; a query over its share is refused |
| `make_fully_adaptive_composition` | An odometer whose queryable reports `privacy_loss(d_in)` so far | Never refuses; for exploration, not for a release |
| `make_privacy_filter(odometer, d_in, d_out)` | An odometer with a cap, which makes it a measurement | What a context without a split wraps; after one refusal it answers nothing |
| `transformation >> then_adaptive_composition(...)` | Preprocessing shared by every query of a compositor | Set `d_in` to the transformation's map at the privacy unit's distance |
| A compositor submitted to a compositor | A sub-budget, answered by a queryable | Costs the inner compositor's map; pure DP lets both queryables stay open |
| `make_pureDP_to_zCDP`, `make_approximate` | $\varepsilon$ to $\rho = \varepsilon^2/2$; $\varepsilon$ to $(\varepsilon, 0)$ | Cast to make a composition legal, then read the total once |
| `make_zCDP_to_approxDP` | $\rho$ to a privacy profile with `epsilon(delta)` and `delta(epsilon)` | OpenDP uses the improved conversion; $\rho = 0.1$ gives 2.35 at $\delta = 10^{-7}$ |
| `make_fix_delta`, `make_fixed_approxDP_to_approxDP` | A profile at one $\delta$, and a pair back to a flat profile | Fix $\delta$ last, after composing in zCDP |
| `make_population_amplification` | A smaller map for a secret simple random sample of a larger population | Sized input domain, `honest-but-curious`, and a sampling you can vouch for |

## 11.10. References

- OpenDP documentation: "Combinators", "Chaining", "(Non-Adaptive) Composition", "Adaptive Composition", "Fully Adaptive Composition and Odometers", "Privacy Filters", "Compositor Chaining and Nesting", "Measure Casting" and "Amplification" in the API user guide.
- Rogers, Roth, Ullman and Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (2016): the odometer and the filter behind `make_fully_adaptive_composition` and `make_privacy_filter`.
- Whitehouse, Ramdas, Rogers and Wu, "Fully Adaptive Composition in Differential Privacy" (2023): the fully adaptive composition theorem the odometer implements.

- Haney, Shoemate, Tian, Vadhan, Vyrros, Xu and Zhang, "Concurrent Composition for Interactive Differential Privacy with Adaptive Privacy-Loss Parameters" (2023): why several queryables may stay open at once.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, its composition and the cast from pure DP.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the improved conversion from zCDP to $(\varepsilon, \delta)$ that the privacy profile implements.
- The full edition's chapters on post-processing and basic composition in the full edition, composition in the full edition, concentrated DP in the full edition, RDP conversions in the full edition, subsampling in the full edition and adaptive composition in the full edition derive the results this chapter states.

# 12. Utilities and the Context API inside

Every query in this book so far ended with a call such as `.laplace()` that named no scale, and the context filled the scale in. It used a few utilities that the framework also exposes on their own. Type descriptors say what kind of number a chain carries, a binary search turns a budget into a scale, and a serializer turns a measurement into text. The Context API is a thin layer over these utilities and the combinators of the previous chapter. When you can see the layer, you can predict what a query will do, and rebuild any part of it by hand when the layer refuses.

The chapter ends by taking apart the count query of the first chapter, which gives the same scale of 3. At the end, you can read a type error, run the search a context runs, and store and reload a measurement. You can also say which framework object each line of a Context API program creates.

The script opendp_utilities.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. Its plain-data queries use the first chapter's hundred integers from a seeded generator, and its one Polars query counts the rows of the France Labour Force Survey (LFS) sample. The type descriptors, error messages, scales and privacy maps are the same on every run. The released counts change, because OpenDP's noise cannot be seeded, so they are in one block marked as one run.

```
uv run python code/part03-framework/opendp_utilities.py
```

## 12.1. Types

OpenDP's core is written in Rust, which has signed and unsigned integers from 8 to 128 bits and floats of two widths. The library keeps that precision, because the constants of a privacy proof depend on it. For example, a sum of 32-bit integers can overflow where a 64-bit sum cannot. So every domain, metric and measure carries a **type descriptor**, such as the `T=` arguments of the framework chapter. The library never casts one type to another on its own.

You can write a descriptor in three ways, and the snippet shows that they agree. The canonical form is a string such as `"i32"` or `"Vec<f64>"`. A Python type such as `int`, `float`, `str` or `bool` is an alias, and `list[int]` is an alias for a vector. The objects in `opendp.typing`, such as `dp.i32`, `dp.f64` and `dp.Vec[dp.i32]`, give the same strings with completion in an editor.

```
import opendp.prelude as dp

print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
print(dp.atom_domain((0, 1), T=dp.i32))  # AtomDomain(bounds=[0, 1], T=i32)
print(dp.RuntimeType.parse(list[int]))  # Vec<i32>
print(dp.Vec[dp.i32] == dp.RuntimeType.parse(list[int]))  # True
```

The aliases hide one decision: a Python `int` means a 32-bit integer and a `float` a 64-bit float, and `dp.RuntimeType.infer` reads a value's type in the same way. But NumPy's default integer has 64 bits, so an integer from an array arrives as `i64` and does not meet a domain built with `T=int`. If all your integers have 64 bits, call `set_default_int_type` to change what the alias means.

```
import numpy as np
import opendp.prelude as dp

print(dp.RuntimeType.parse(int))  # i32
print(dp.RuntimeType.infer(np.int64(1)))  # i64
dp.set_default_int_type(dp.i64)
print(dp.RuntimeType.parse(int))  # i64
print(dp.vector_domain(dp.atom_domain(T=int)))  # VectorDomain(AtomDomain(T=i64))
```

When the types do not meet, the error reads `inferred type is XXX, expected YYY`, which the documentation's troubleshooting page lists. The script builds a Laplace mechanism on an integer domain with an integer metric, and then asks for its privacy map at a sensitivity of 1.5.

```
int_space = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace_int = int_space >> dp.m.then_laplace(scale=1.0)  # Laplace noise on a 32-bit
 ↪  integer
try:
    laplace_int.map(1.5)  # a float sensitivity, on a metric whose distances are
 ↪  integers
    message = "no error"
except TypeError as err:
    message = first_line(err)
```

```
map(1.5) on an integer metric: TypeError: inferred type is f64, expected i32
map(2) on an integer metric: 2.0
float d_in on the count: TypeError: inferred type is f64, expected u32
integer d_in on the count: 0.3333
```

The message names the type the library inferred from your argument and the type the metric expects, so pass what it expects, here the integer 2. The second pair of lines shows the most common case. The count's input metric is the symmetric distance, whose distances are unsigned integers, so `map(1.0)` fails and `map(1)` succeeds. The documentation names one more cause, a scalar $\varepsilon$ given to a measure that expects $\varepsilon$ with the slack $\delta$ of approximate differential privacy. In 0.16.0 that message is different, as the script prints.

```
scalar epsilon on an approximate measure: TypeError: Cannot coerce a non-tuple type to
 ↪  a tuple
pair on an approximate measure: True
```

The chapter on limitations and troubleshooting collects the other errors. For this one, read the expected type, then change your argument, your bounds or your data to match it. Do not change the domain to match the argument, because the library chose that type for the proof.

## 12.2. Parameter search

A measurement has three kinds of parameter. The input distance `d_in` is the privacy unit's bound, the output distance `d_out` is the budget, and the constructor's own arguments include the noise scale. With all of them fixed, `check(d_in, d_out)` says whether the measurement is within the budget, as the measurements chapter showed. The answer is monotone: if a scale passes at a budget, every larger scale passes, and so does every larger budget. So the answer changes at one boundary, which a binary search finds, and you can solve for any one parameter from the others.

The library has three forms of the search. The function `dp.binary_search` takes a predicate on one number and returns the value nearest the boundary on the passing side. The function `dp.binary_search_param` takes a function that builds a chain from one number, with `d_in` and `d_out`, and uses `check` as the predicate. The function `dp.binary_search_chain` does the same and returns the chain at the value it found.

```
import opendp.prelude as dp

print(dp.binary_search(lambda x: x >= 5.0))  # 5.0
print(dp.binary_search(lambda x: x > 5, T=int))  # 6
print(dp.binary_search(lambda x: x <= 5, bounds=(0, None)))  # 5
```

The three lines show the arguments of a search. Without bounds, the search first finds bounds by steps outwards from zero. The type `T` says whether the parameter is a float or an integer, and bounds with `None` on one side give a one-sided search. The script then runs the search of a context on the first chapter's Laplace mechanism, with a share of $\varepsilon/3$.

```
scale = dp.binary_search_param(make_laplace, d_in=1.0, d_out=share)  # the smallest
↪  scale within the share
chain = dp.binary_search_chain(make_laplace, d_in=1.0, d_out=share)  # the measurement
↪  at that scale
largest_d_in = dp.binary_search(lambda d_in: make_laplace(1.0).check(d_in, EPSILON))  #
↪  the other way round
```

```
search: scale=3.0000000000000004 chain.map(1.0)=0.3333 chain.check(1.0, share)=True
search: largest d_in at scale 1 within epsilon 1: 1.0
search: gaussian scale for rho=1 at d_in=1: 0.7071
```

The scale is the smallest one whose map at a sensitivity of 1 is within a third of $\varepsilon = 1$. It is a little above 3, because the search stops on the passing side of the boundary, and it is the scale that `param()` reported in the first chapter. The third call asks for the largest sensitivity that a scale of 1 hides within $\varepsilon = 1$, which is 1. The Gaussian mechanism is measured in zero-concentrated differential privacy (zCDP), with a loss $\rho$ of half the squared sensitivity over the squared scale. So the search for $\rho = 1$ gives a scale of 0.7071 (the full edition's chapter on the two mechanisms).

To show what the search does, the script writes it by hand. The predicate is `check`, the interval starts at 0 to 100, and each step keeps the half that contains the boundary.

```
def search_by_hand(passes: Callable[[float], bool], low: float, high: float, tolerance:
 ↪  float) -> tuple[float, int]:
    """Bisection on a predicate that fails at `low` and passes at `high`: the smallest
     ↪  passing value, and the steps."""
    steps = 0
    while high - low > tolerance:
        middle = (low + high) / 2
        if passes(middle):
            high = middle  # the boundary is at or below the middle
        else:
            low = middle  # the boundary is above it
        steps += 1
    return high, steps
```

```
search by hand: scale=3.000000 after 27 steps
search with bounds (0, 100): 3.0000000000000004
```

After twenty-seven halvings, the interval of 100 is below a millionth, and its passing end is the scale. The library's search on the same bounds lands on the float just above 3, so it bisects further than that tolerance. A search can fail in two ways, and the script shows both messages.

```
search with bounds (0, 1): ValueError: the decision boundary of the predicate is
 ↪  outside the bounds
search on a constant predicate: ValueError: unable to infer bounds
```

Bounds that do not contain the boundary give the first error, so make them wider. A predicate that never changes gives the second, here one that always passes. With a real chain, it means that no value of the parameter changes the answer, so look at the chain before the bounds.

## 12.3. Serialization

A measurement is a recipe that names its constructors and their arguments. The function `dp.serialize` writes that recipe as a JSON string, and `dp.deserialize` turns it back into a measurement. So an analyst can build and inspect a query on a machine without the data, and send the text to the machine that holds them. The script serializes the first chapter's count measurement.

```
measurement = count_laplace(3.0)
text = dp.serialize(measurement)  # JSON that names each constructor and its arguments
restored = dp.deserialize(text)
same_space = restored.input_space == measurement.input_space
```

```
serialized: 2152 characters, starting {"__function__": "make_chain_mt",
restored: map(1)=0.3333 same input space: True
```

The text starts with the outermost constructor, the chain of a transformation and a measurement, and nests the rest inside it. The restored measurement has the same input space and privacy map, so a reviewer can check the guarantee before anyone touches the data. The documentation does not promise a stable format between versions, and it asks you not to write it by hand.

Two things do not survive a round trip. The first is the `param` attribute that a context attaches to a measurement after a search (a later section). It is a Python attribute and not part of the recipe. The second is anything with Python state that the constructors did not set, such as a context, a query or a plugin built from a Python function.

```
param before a round trip: 3.0000000000000004
param after a round trip: None
serialize a context: OpenDP JSON Encoder does not handle instances of <class
 ↪  'opendp.context.Context'>
```

A Polars query has its own route, because Polars can serialize the plan of a `LazyFrame` to bytes. The method `Context.deserialize_polars_plan` wraps those bytes in a query on a context of its own. So a client can write the plan against the survey's schema, and the server can run it on the rows.

```
client = lfs_context(pl.LazyFrame(schema=frame.collect_schema()))  # the schema, but no
 ↪  rows
plan = client.query().select(dp.len(signed=True)).polars_plan.serialize()  # bytes
server = lfs_context(frame)  # the real data
query = server.deserialize_polars_plan(plan)
summary = query.summarize(alpha=0.05)
```

```
plan: 1792 bytes
plan on the server: scale=36.0 95% accuracy=108.3429
```

The client's context holds an empty frame with the survey's schema, and the server's holds the file. Both must state the budget and the privacy unit of 36 rows per person (the chapter on the data and the context). The plan carries the query but not the guarantee. The scale of 36 is the unit's distance over the budget, and the error bar is about 108 rows on a count of millions.

## 12.4. The compositor behind a context

The rest of the chapter rebuilds the Context API from the combinators of the previous chapter. The constructor `dp.Context.compositor` builds an **adaptive composition** over the data's space and calls it on the data. That composition takes a list of budgets, the `d_mids`, and accepts one measurement per share, each of which may depend on the earlier answers. Together they stay within the sum (the full edition's chapter on composition). The script reads the context's accountant and builds the same one by hand.

```
by_hand_accountant = dp.c.make_adaptive_composition(
    input_domain=SPACE[0],
    input_metric=SPACE[1],
    output_measure=dp.max_divergence(),
    d_in=1,  # the privacy unit's distance: one row added or removed
    d_mids=[share] * QUERIES,  # the shares, which add up to the budget
)
queryable = by_hand_accountant(data)  # answers measurement queries on the data, and
 ↪  keeps count
```

```
context accountant: VectorDomain(AtomDomain(nan=true, T=f64)) SymmetricDistance()
 ↪  MaxDivergence
context d_in: 1  d_mids: [0.3333, 0.3333, 0.3333]
context accountant map(1): 1.0000
by hand accountant map(1): 1.0000  queryable: Queryable(Q=AnyMeasurement)
d_mids by weights [1, 2, 1]: [0.25, 0.5, 0.25]
```

The context found the domain from the data, a list of floats, and the metric and the measure come from `unit_of` and `loss_of`. Both compositions map the unit's distance to the whole budget, 1. On the data, the accountant returns a **queryable**, which answers one measurement at a time and refuses when the shares run out. The context keeps that queryable, and the data are only inside it, which is why the first chapter told you to drop your own reference.

The shares come from `split_evenly_over` or `split_by_weights`. The context gives the weights to `binary_search_param` and scales them until the composition's map at `d_in` meets the budget. For pure differential privacy, that is the expected proportional split: weights of 1, 2 and 1 give a quarter, a half and a quarter of $\varepsilon$.

## 12.5. The query behind a release

The call `context.query()` returns a **Query**, which holds a chain and the share it must fit, and on a LazyFrame it is the `LazyFrameQuery` of the Polars chapters. The chain starts as the data's space, and each method you call, such as `count` or `laplace`, chains on the constructor of that name. So `query.count()` is `space >> dp.t.then_count()`. The script shows the type of the chain at each step.

```
query = plain_query(context)  # the chain is the data's space: a domain and a metric
counted = query.count()  # the chain is a Transformation
noisy = counted.laplace()  # no scale: the chain is a PartialChain, waiting for one
 ↪  number
is_partial = isinstance(noisy._chain, dp.PartialChain)
```

```
query chain: tuple -> Transformation -> PartialChain
partial chain: True
resolve a transformation: ValueError: Query is not yet a measurement or odometer.
```

The important step is `laplace()` without a scale. The query finds one argument missing from the constructor's signature, so it wraps the constructor in a **partial chain**, a function from the missing number to the finished chain. Only one number may be missing at a time, so a second noise step without a scale fails. A query that is still a transformation cannot resolve into a measurement, as the third line shows.

The method `resolve` finds the number. It calls the partial chain's `fix`, which runs `binary_search` with the predicate "the chain at this value checks within the query's share". That is the search of Section 12.2, and `fix` stores the value on the measurement as `param`. So `param()` reports the scale of the search, or `None` when there was no search.

```
param of .laplace(): 3.0000000000000004
param of .laplace(scale=3.0): None
two missing parameters: ValueError: gaussian is missing 1 parameter(s).
```

The script then runs the same search by hand on the same chain, and releases through the queryable built by hand.

```
by_hand_param = dp.binary_search_param(count_laplace, d_in=1, d_out=share)  # what
 ↪  resolve runs on the chain
by_hand_measurement = count_laplace(by_hand_param)
dp_count_by_hand = queryable(by_hand_measurement)  # what release does: the queryable
 ↪  runs the measurement
```

```
resolve: context param=3.0000000000000004 by hand=3.0000000000000004 equal=True
d_mids after context(measurement): [0.3333, 0.3333]
```

The two scales are equal, because they come from the same search on the same predicate. The method `release` gives the resolved measurement to the queryable and records the share as spent, so the first share leaves `d_mids`. The version by hand does the same with `queryable(measurement)`, but nothing records the spend. That record is what the context adds.

## 12.6. Nested compositors and other measures

Two features of the Context API are easier to understand when you can see the chain. The first is `Query.compositor`, which nests an adaptive composition inside one share. The query's share becomes the inner budget, split again, and the release of the query returns a new `Context` and not a number.

```
inner = plain_query(context).compositor(split_evenly_over=2).release()  # a Context
 ↪  inside one share
inner_scale = plain_query(inner).count().laplace().param()
```

```
inner context: d_in=1 d_out=0.3333 d_mids=[0.1667, 0.1667]
inner count scale: 6.0000
outer d_mids after the inner compositor: [0.3333]
```

The inner context has the outer share, a third of $\varepsilon$, as its whole budget and splits it in two, so a count inside it gets a scale of 6. The outer context counts the nested compositor as one query, whatever happens inside it, because the composition's guarantee covers everything its queryable answers.

The second feature is a budget in one measure for a mechanism proved in another. The Laplace mechanism's measure is the max divergence of pure differential privacy, and the Gaussian mechanism's is zCDP. When the context's measure is different, `resolve` casts the measurement with the converters of Combinators before it checks the budget.

```
approx = dp.Context.compositor(
    data=data,
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # approximate differential
 ↪  privacy
    split_evenly_over=QUERIES,
)
laplace_cast = plain_query(approx).count().laplace().resolve()  # pure DP, cast to
 ↪  approximate DP
gaussian_cast = plain_query(approx).count().gaussian().resolve()  # zCDP, converted and
 ↪  its delta fixed
```

```
approx share: (0.3333, 3.3e-08)
laplace under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 0.0e+00)
gaussian under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 3.3e-08)
param after a cast: None
```

Under an $(\varepsilon, \delta)$ budget, `make_approximate` wraps the Laplace mechanism and reports $(\varepsilon, 0)$, so the search fits only $\varepsilon$. The Gaussian mechanism goes through `make_zCDP_to_approxDP` and `make_fix_delta`, which spend the share's slack to convert $\rho$ into $\varepsilon$. But the cast wraps the old measurement in a new one, so the `param` attribute is on the inner measurement, and `param()` reads `None`. To see the scale, run `binary_search_param` on the chain yourself.

```
laplace under rho: ZeroConcentratedDivergence map(1)=0.0333
gaussian under rho: scale=3.8730
gaussian under epsilon: ValueError: Unable to cast measure from
 ↪  ZeroConcentratedDivergence to MaxDivergence
```

Under a $\rho$ budget, the Laplace mechanism is cast with `make_pureDP_to_zCDP`, and the Gaussian mechanism needs no cast, so its `param()` reports a scale of 3.873. A Gaussian mechanism under a pure $\varepsilon$ budget cannot be cast, because zCDP never implies pure differential privacy, and the context says so. The full edition's chapter on concentrated differential privacy derives the conversions behind each cast.

Here are the releases of one run, to read against their error bars.

```
released (one run):
  survey rows 3811875  (true 3811932, within 108.3 with probability 0.95)
  count by hand 94  (true 100, within 9.4 with probability 0.95)
  count through the context 101  (true 100)
```

All three are within their bars. The two counts of the hundred integers came from the same measurement, through the queryable built by hand and through the context. So they differ only by their noise.

## 12.7. Summary

Table 12.1.: The utilities and the pieces of the Context API, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `"i32"`, `int`, `dp.i32` | Three spellings of one type descriptor | `int` is `i32` and `float` is `f64`; NumPy's integers are `i64` |
| `dp.set_default_int_type(dp.i64)` | Changes what `int` means for the rest of the program | The clean fix when all your integers have 64 bits |
| `inferred type is XXX, expected YYY` | Your argument's type does not match the metric's or the domain's | Change the argument to the expected type, not the domain to the argument |
| `dp.binary_search(predicate, bounds, T)` | The value nearest the boundary of a monotone predicate | Pass bounds when the chain is only valid in a range, and `T=int` for an integer |
| `dp.binary_search_param(make_chain, d_in, d_out)` | The smallest scale whose chain checks within the budget | What a context runs for `.laplace()` without a scale |
| `dp.binary_search_chain(...)` | The chain built at the value the search found | It carries no `param`; use `binary_search_param` for the number |
| `dp.serialize`, `dp.deserialize` | A measurement as JSON, and back | The recipe survives; `param`, contexts and Python plugins do not |
| `Context.deserialize_polars_plan(bytes)` | A Polars plan from a client, run on the server's context | The plan carries the query; each context states the guarantee |
| `dp.Context.compositor(...)` | An adaptive composition over the data's space, called on the data | Its queryable holds the data; `d_mids` are the shares |
| `context.query()` | A `Query` whose chain grows with each constructor | A missing number makes a partial chain; one at a time |
| `query.param()` | The scale that `resolve` found | `None` when the scale was given or the measure was cast |
| `Query.compositor(...)` | A context nested inside one share | The outer context counts it as one query |
| A Gaussian query under a pure budget | `ValueError: Unable to cast measure ...` | zCDP never implies pure differential privacy; state the budget with a $\delta$ or as $\rho$ |

## 12.8. References

- OpenDP documentation: “Typing”, “Parameter Search” and “Serialization” under “Utilities” in the API user guide; “Context” and “API Comparison” in the same guide; and the entry `Inferred type is XXX, expected YYY` of “Troubleshooting” in Getting Started.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the domains, metrics, measures and maps that the type descriptors and the searches work on.
- Dwork, Rothblum and Vadhan, “Boosting and Differential Privacy” (2010): composition of adaptively chosen mechanisms, which the compositor behind a context implements.
- Bun and Steinke, “Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds” (2016): zCDP and its conversions to approximate differential privacy, which the measure casts use.
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

# Part IV.

# Extending the library

The framework is open. Anything with a stability map or a privacy map can be a transformation or a measurement, whether OpenDP wrote it or you did. The thread of this part is that a plugin is only as private as its map. So writing one means proving its map, and the library makes the rest safe, from chaining to accounting.

Plugins writes a transformation and a measurement in Python and registers constructors for the Context API. Then it follows the documentation's two larger examples, a Theil-Sen regression and a private choice of grouping columns. Statistical modelling fits the two models that come with the library, a differentially private linear regression and a differentially private principal component analysis, and reads what each guarantees. Limitations and troubleshooting says what OpenDP does not promise, from floating-point arithmetic to side channels. It also explains the feature flags and the common error messages.

Table 12.2.: The chapters of this part and the questions they answer.

| Chapter | Question it answers |
|---|---|
| Plugins | How do I add my own transformation or measurement, and what must I prove about it? |
| Statistical modelling | How do I fit a private regression or a private principal component analysis with the library's estimators? |
| Limitations and troubleshooting | What does the library not guarantee, and what does each common error mean? |

At the end of this part you can add a mechanism of your own, fit the built-in models, and say which parts of a deployment stay yours.

# 13. Plugins

Sometimes the mechanism you need is not in the library, such as a new estimator from a paper. The framework of the previous part allows for this, because a transformation is any function with a stability map and a measurement is any function with a privacy map. So you can write both in Python and give them to the library as **plugins**, which then chain, compose and run through a context like the built-in ones.

The one idea of this chapter is that a plugin is exactly as private as its map, and the library cannot check the map. Every built-in constructor comes with a written proof of its map, but for a plugin that proof is yours. So each example below says why its map is right, or where the documentation leaves that open. You will write a measurement, a transformation and your own spaces, register a constructor for the Context API, and follow the documentation's Theil-Sen regression and private choice of grouping columns.

The script opendp_plugins.py runs every step on OpenDP 0.16.0. The regression uses the documentation's toy data, a hundred points on a line with seeded normal noise, and the choice of grouping columns uses the Labour Force Survey (LFS) sample. Every map, check, error message, score and candidate range on this page is the same on every run. OpenDP's noise cannot be seeded, so the released values appear in one block marked as one run at the end.

```
uv run python code/part04-extending/opendp_plugins.py
```

## 13.1. Trust and the honest-but-curious flag

The documentation gives two reasons for plugins: the library cannot ship every mechanism, and an idea is easier to try in Python than in Rust. Either way a Python function is at the centre of a guarantee, so the library cannot read it. It cannot tell whether your privacy map is right, whether your function has side effects, or whether it raises an error only on some data.

That is why plugin constructors need a second feature flag next to `contrib`, named `honest-but-curious` after its trust model. The author of the plugin is honest, because they have proved their map, and the library guards only against curiosity downstream. Without the flag a plugin constructor refuses to build.

```
without the flag: Attempted to use function that requires "contrib",
 ↪  "honest-but-curious", but not enabled
```

Each constructor's docstring states the promise. For any two inputs at most `d_in` apart under the input metric, the output distributions must be at most `privacy_map(d_in)` apart under the output measure. Also, the function must have no side effects, and the map must be pure. A

wrong map breaks the guarantee silently, because the context charges what the map reports. The snippet shows the extreme case, a measurement that returns its input and claims a privacy loss of zero.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
leak = dp.m.make_user_measurement(*space, dp.max_divergence(), function=lambda x: x,
 ↪  privacy_map=lambda d: 0.0, TO=list[int])
print(leak.map(1))  # 0.0
print(leak([1, 2, 3]))  # [1, 2, 3]
```

The library accepts it, and a context would release the raw data at no charge. The flag records that the claim rests on your word, and the sentence that justifies each map below is what makes that word worth something.

## 13.2. A measurement plugin

The documentation's first example is a measurement that ignores its input and returns a constant, which is useless as a release but has the easiest map to justify. The constructor `make_user_measurement` takes the five parts of a measurement (A framework to understand differential privacy). They are an input domain, an input metric, an output measure, the function and the privacy map.

```
def make_base_constant(constant: str) -> dp.Measurement:
    """The documentation's example: a measurement that ignores its input and returns a
     ↪  constant."""

    def function(_arg: int) -> str:
        return constant  # the output does not depend on the data

    def privacy_map(_d_in: int) -> float:
        return 0.0  # so two neighbouring inputs give the same distribution: a privacy
         ↪  loss of zero

    return dp.m.make_user_measurement(
        input_domain=dp.atom_domain(T=int),
        input_metric=dp.absolute_distance(T=int),
        output_measure=dp.max_divergence(),
        function=function,
        privacy_map=privacy_map,
        TO=str,
    )
```

The input space is one integer under the absolute difference. The output measure is the max divergence of pure differential privacy, so the map gives an $\varepsilon$. The map returns zero at every distance, which is correct because the output does not depend on the input. The last argument,

`TO`, names the type that the function returns. The script chains the result after a sum of bounded integers.

```
constant: input=AtomDomain(T=i32) metric=AbsoluteDistance(i32) measure=MaxDivergence
sum >> constant: map(1)=0.0 release=denied
```

The chain works because the output space of the sum, an integer under the absolute distance, is the input space of the plugin. The library checks that match, because domains and metrics are its own objects, but it does not check the map.

The next constructor has a real map. It adds Laplace noise from NumPy instead of from OpenDP, and it takes its input space as its first two arguments, the convention that the Context API needs later.

```
def make_numpy_laplace(input_domain: dp.Domain, input_metric: dp.Metric, scale: float)
↪  -> dp.Measurement:
    """Laplace noise drawn by NumPy instead of OpenDP: the same map as the built-in,
     ↪  but on your word."""
    rng = np.random.default_rng()

    def function(arg: float) -> float:
        return float(arg + rng.laplace(scale=scale))

    def privacy_map(d_in: float) -> float:
        return d_in / scale  # the Laplace mechanism's loss: the sensitivity over the
         ↪  scale

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function,
↪   privacy_map=privacy_map, TO=float
    )
```

The map is the sensitivity over the scale, which is what Laplace noise costs in $\varepsilon$ (the full edition's chapter on the Laplace and Gaussian mechanisms derives it). The built-in `then_laplace` at the same scale reports the same map, and that comparison is the check to run on any plugin with a built-in equivalent.

```
python laplace scale 2: map(1.0)=0.5 built-in map(1.0)=0.5
halved map, same noise: map(1.0)=0.25 check(1.0, 0.25)=True
laplace scale 2: 95% accuracy=5.9915
```

The second line uses the same noise with a map that claims half the loss, as a test of what the library catches. The library accepts it, and `check` reports that it fits a budget of 0.25, which is false, so a context would give such a plugin a quarter and spend half. The third line is the 95% error bar of the noise at scale 2, for the release at the end of the page. In production, prefer the Rust core, because NumPy's float sampling leaks a little through rounding and OpenDP's own samplers prevent that.

## 13.3. A transformation plugin

A transformation plugin takes one more argument than a measurement, an output domain. Its map is a stability map, which bounds the distance between two outputs when the inputs are a given distance apart (Transformations). The documentation's example repeats each record a fixed number of times.

```
def make_repeat(multiplicity: int) -> dp.Transformation:
    """The documentation's example: a transformation that repeats each record
     ↪  `multiplicity` times."""

    def function(arg: list[int]) -> list[int]:
        return arg * multiplicity

    def stability_map(d_in: int) -> int:
        return d_in * multiplicity  # one added record of the input is `multiplicity`
         ↪  added records of the output

    return dp.t.make_user_transformation(
        input_domain=dp.vector_domain(dp.atom_domain(T=int)),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=int)),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

Both metrics are the symmetric distance, the number of records added or removed. The map multiplies that distance by the multiplicity, because each input record becomes `multiplicity` output records. The output domain is a second claim. Every output must belong to it, or the function must fail in a way that does not depend on the data. Here it holds, because the output is a vector of integers whenever the input is.

The documentation chains the plugin between built-in steps, a cast from strings and a clamp before a sum, and ends with Laplace noise.

```
repeat 2: [1, 2] -> [1, 2, 1, 2] map(1)=2 map(3)=6
cast >> repeat 2 >> clamp (1, 2) >> sum: map(1)=4 on ['0', '1', '2', '3'] = 12
... >> laplace scale 1: map(1)=4.0
domain mismatch: DomainMismatch("Intermediate domains don't match
```

The stability of a chain is the product of the stabilities of its links (Combinators). The cast and the clamp have stability 1 and the repeat has 2. The clamp bounds every value by 2, so one added input record, repeated twice, moves the sum by at most 4. Laplace noise of scale 1 then costs an $\varepsilon$ of 4. In the last line the cast gives floats, and the library refuses the chain, because it can compare its own spaces.

The library cannot compare the function with the output domain, and that is the second way a transformation plugin goes wrong. The snippet declares that its output lies between 0 and 1 and then returns its input unchanged.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
ints = dp.vector_domain(dp.atom_domain(T=int))
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 1)))
claim = dp.t.make_user_transformation(ints, dp.symmetric_distance(), bounded,
 ↪  dp.symmetric_distance(), function=lambda x: x, stability_map=lambda d: d)
print(claim([5, 6]))  # [5, 6]
print((claim >> dp.t.then_sum()).map(1))  # 1
```

The sum trusts the bounds and reports a sensitivity of 1, but the real sum moves by the whole value of a record. A wrong output domain becomes a wrong map one step later, so a transformation plugin needs a justification for each.

## 13.4. Your own domains, metrics and measures

A plugin may work on data that the library has no domain for, such as a data frame, or compare neighbours by a metric that the library does not define. Three constructors let you define your own, and each carries a promise.

A **user domain** has an identifier and a membership function. The library treats two user domains as equal when their identifiers are equal, and uses that to decide whether two steps chain. So the identifier must name one set only. Otherwise two different domains chain, and the chain's stability no longer holds. The membership function must return true only for members. A **user distance** is a metric that is only a name. It promises that your distance is a pseudo-metric, which is zero from a point to itself, never negative, symmetric, and obeys the triangle inequality. The snippet declares a domain of even integers and an edit distance on strings, and measures the length of a string.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
even = dp.user_domain("EvenInts", member=lambda x: x % 2 == 0)
print(even.member(4), even.member(3))  # True False
edits = dp.user_distance("EditDistance")
length = dp.t.make_user_transformation(dp.atom_domain(T=str), edits,
 ↪  dp.atom_domain(T=int), dp.absolute_distance(T=int), function=len,
 ↪  stability_map=lambda d: d)
print(length("hello"), length.map(1))  # 5 1
print((length >> dp.m.then_laplace(scale=1.0)).map(1))  # 1.0
```

The stability map of the length is the identity, because one edit changes the length by at most one. So the chain into Laplace noise closes, and a scale of 1 costs an $\varepsilon$ of 1 per edit.

A **user divergence** is a privacy measure of your own. Post-processing must not increase it, because a release must stay private under further computation. The full edition's chapter on properties proves this for the max divergence. The function `new_privacy_profile` turns a curve from $\varepsilon$ to the slack $\delta$ into a privacy profile that the library can invert. And `new_function` wraps

a Python post-processor, as Combinators did for a bare function after `>>`. The post-processor must not read global state or raise an error that depends on the data, because either opens a channel from the data that no map accounts for.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
renyi = dp.user_divergence("RenyiDivergence(alpha=2)")
rng = np.random.default_rng()
gaussian = dp.m.make_user_measurement(*space, renyi, function=lambda x: float(x +
 ↪  rng.normal()), privacy_map=lambda d: d**2, TO=float)
print(gaussian.map(1.0), gaussian.output_measure)  # 1.0 RenyiDivergence(alpha=2)
try:
    dp.c.make_composition([gaussian, gaussian])
except dp.OpenDPException as err:
    print(str(err).split(". See")[0].strip())  # Type("No match for concrete type
     ↪  ExtrinsicDivergence
profile = dp.new_privacy_profile(lambda eps: 1.0 if eps < 0.5 else 1e-8)
print(profile.delta(epsilon=0.499), profile.epsilon(delta=1e-8))  # 1.0 0.5
double = dp.new_function(lambda x: 2 * x, TO=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print((laplace >> double).map(1.0))  # 1.0
```

The measurement adds standard normal noise. Under the Rényi divergence of order 2 its map is the squared sensitivity. The reason is that this divergence between two unit Gaussians is the squared distance between their centres (the full edition's chapter on Rényi differential privacy). The composition fails, because the combinators add only the measures that the library defines, so a plugin measure gets `map` and `check` and nothing more. The privacy profile answers in both directions, the slack at a given $\varepsilon$ and the $\varepsilon$ at a given slack, and its curve must decrease. The post-processor leaves the map of the Laplace measurement unchanged.

## 13.5. Registering a constructor for the Context API

A plugin built with the framework API is a loose measurement, and A first release showed why every query should go through a context. The function `dp.register` adds a constructor to the Context API as a method of `context.query()`. The constructor's name must start with `make_`, and the method has the name without that prefix, unless you pass a `name`. If its first two parameters are `input_domain` and `input_metric`, the context fills them in with the output space of the previous step. The documentation calls that partial application, and it gives every built-in `make_` constructor its `then_` form.

```
def make_anything_constant(input_domain: dp.Domain, input_metric: dp.Metric, constant:
 ↪  str) -> dp.Measurement:
    """The documentation's constructor for the Context API: it accepts whatever space
     ↪  the previous step outputs."""

    def function(_arg: Any) -> str:
```

```
        return constant

    def privacy_map(_d_in: Any) -> float:
        return 0.0

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function,
 ↪  privacy_map=privacy_map, TO=str
    )


dp.register(make_anything_constant)  # from now on,
 ↪  context.query().anything_constant(...)
```

The script builds the documentation's context over three integers, with a budget of $\varepsilon = 1$ split over two queries. Its privacy unit is the book's 36 contributions per person (the data and the context). Then the script calls the plugin through it.

```
anything_constant: map(36)=0.0 release=denied
budget spent by the plugin query: [0.5]
clamp >> sum >> int_constant: release=denied
register twice: 'anything_constant' is already registered in the Context API. Please
 ↪  choose a different name.
register without make_: constructor.__name__ must start with 'make_', found select_cols
```

On the second line the map of the plugin is zero, but the context charged the query its full share. The reason is that a context splits the budget before it sees a query (Utilities and the Context API inside). So a cheap plugin does not give the difference back.

The third line uses the documentation's other style, `make_int_constant`, without the two leading parameters. Its input space is fixed to integers, so it works only after a clamp and a sum. The documentation recommends the first style, but a constructor that accepts any space should check that the space suits its function. The last two lines are the errors of `register` for a name already taken and for a name without the prefix.

## 13.6. A Theil-Sen regression

The documentation's first larger example fits a line through noisy points. Least squares is a poor private estimator, because one far-away point can move the line by any amount. The **Theil-Sen** estimator takes the median of the slopes through pairs of points instead, and OpenDP already releases medians privately. The documentation's variant pairs the points at random and fits a line through each pair. It records where each line crosses two vertical cuts, at a quarter and three quarters of the range of $x$, and the medians at the cuts are two points on the fitted line. The library ships a vetted version of this estimator (Statistical modelling), so this one is for learning how plugins, your own transformations and measurements, combine.

The first plugin is the pairing, a transformation from a two-column array to a two-column array of predictions.

```
def pairwise_predict(data: NDArray[np.float64], x_cuts: NDArray[np.float64]) ->
↪  NDArray[np.float64]:
    """Pair the rows at random, fit a line through each pair, and predict y at the two
    ↪  cut points."""
    data = np.array(data, copy=True)[: len(data) // 2 * 2]
    np.random.shuffle(data)
    p1, p2 = np.array_split(data, 2)
    dx, dy = (p2 - p1).T
    x_bar, y_bar = (p1 + p2).T / 2
    points = dy / dx * (x_cuts[None].T - x_bar) + y_bar
    return points.T[dx != 0]
```

The transformation wraps that function.

```
def make_pairwise_predict(x_cuts: NDArray[np.float64], runs: int = 1) ->
↪  dp.Transformation:
    """A transformation from the (x, y) rows to the predictions: one row per pair and
    ↪  run."""

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return np.vstack([pairwise_predict(arg, x_cuts) for _ in range(runs)])

    def stability_map(d_in: int) -> int:
        return d_in * runs  # each record sits in one pair, so it changes one
        ↪  prediction per run

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

The domains, from the library's NumPy extension, are arrays with two columns. The documentation states the map, the input distance times the number of runs, without its argument: each record sits in one pair, so it changes at most one prediction per run. That claim is weaker than a built-in map in two ways. The pairing is random, so the claim is about a coupling of two runs, not about a function. And the function drops the last record when the count is odd, so removing one record can change the pair of another. That is the kind of gap the flag asks you to notice.

The second plugin selects one column of the predictions, and the library's quantile does the rest.

```
def make_select_column(j: int) -> dp.Transformation:
    """Column j of the predictions as a vector; one row in is one row out, so the
    ↪  stability is the input distance."""

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return arg[:, j]
```

```
    def stability_map(d_in: int) -> int:
        return d_in

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=float)),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

The library's quantile then runs once for each cut.

```
def make_private_medians(output_measure: dp.Measure, y_bounds: tuple[float, float],
↪  scale: float) -> dp.Measurement:
    """The private median of the predictions at each cut, by the library's quantile,
    ↪  composed."""
    median = dp.m.then_private_quantile(output_measure,
↪  candidates=np.linspace(*y_bounds, 100), alpha=0.5, scale=scale)
    return dp.c.make_composition(
        [
            make_select_column(0) >> dp.t.then_drop_null() >> median,
            make_select_column(1) >> dp.t.then_drop_null() >> median,
        ]
    )
```

Selecting a column keeps one row per row, so its stability is the input distance. The private quantile scores a hundred candidates between the public bounds of $y$ and picks one by noisy max (Measurements), at a cost of the input distance over the scale. The composition adds the two costs (the full edition's chapter on composition). Solving for the line is post-processing of two released numbers and costs nothing.

```
def make_private_theil_sen(
    output_measure: dp.Measure, x_bounds: tuple[float, float], y_bounds: tuple[float,
↪  float], scale: float
) -> dp.Measurement:
    """Pair and predict, release the two medians, then solve for the slope and the
    ↪  intercept (post-processing)."""
    x_cuts = x_bounds[0] + (x_bounds[1] - x_bounds[0]) * np.array([0.25, 0.75])
    p_inv = np.linalg.inv(np.vstack([x_cuts, np.ones_like(x_cuts)]).T)  # the line
↪  through two points

    def to_coefficients(medians: list[float]) -> tuple[float, float]:
        slope, intercept = p_inv @ np.array(medians)
        return float(slope), float(intercept)

    return make_pairwise_predict(x_cuts) >> make_private_medians(output_measure,
    ↪  y_bounds, scale) >> to_coefficients
```

The script runs it with the documentation's bounds and a scale of 1, on the toy points. Their true line has slope 2 and intercept 1.

```
theil-sen: x cuts -1.5 and 1.5, 100 candidates from -10.0 to 10.0 step 0.2020
pairwise predict: map(1)=1 medians: map(1)=2.0 theil-sen: map(1)=2.0 map(2)=4.0
toy data: n=100 least-squares slope=2.0573 intercept=1.0376
pairs: 50 medians at the cuts -2.2424 and 4.2857
selection bound: 26.2 score units at noise scale 2.0
cut -1.5: best candidate -2.3232 score 2; within the bound: -2.9293 to -1.5152 (8
 ↪  candidates)
cut 1.5: best candidate 4.3434 score 8; within the bound: 3.3333 to 6.3636 (16
 ↪  candidates)
```

The pairing has stability 1 and the two medians cost 2 together, so the estimator costs an $\varepsilon$ of 2 per record. The exact medians of the fifty pairwise predictions are the values that the private medians aim at.

The last three lines give the bound for the released medians. The private quantile adds exponential noise of scale 2 to each score and picks the best. The difference of two such draws is a Laplace draw, so the chance that a worse candidate wins falls exponentially in its gap to the best. A union bound over the other 99 candidates gives the printed bound, which a losing candidate beats at most once in ten thousand runs.

## 13.7. Choosing grouping columns privately

The documentation's second larger example solves a problem that comes before any grouped release. You want to choose grouping columns without looking at the data. Some columns are too uniform and others too diverse, and too many columns leave most groups too small to pass the thresholds of Grouping. So the example scores each candidate set by how many of its groups reach a public minimum of rows. Then it picks a set with the **exponential mechanism**, which chooses a candidate with a probability that grows exponentially in its score (the full edition's chapter on selection).

The score is a transformation plugin on a user domain, because the library has no domain for a whole data frame. The documentation scores a pandas frame, and the same score runs here on a Polars frame.

```
def score(frame: pl.DataFrame, columns: tuple[str, ...], min_rows: int) -> float:
    """How many groups of these columns have at least `min_rows` rows."""
    return float(frame.group_by(list(columns)).len().filter(pl.col("len") >=
     ↪  min_rows).height)
```

The transformation applies it to every candidate set.

```
def make_grouping_cols_score(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str,
 ↪  ...]], min_rows: int
) -> dp.Transformation:
```

```
    """A transformation from the frame to one score per candidate set of grouping
     ↪ columns."""
    dp.assert_features("contrib")

    def function(frame: pl.DataFrame) -> list[float]:
        return [score(frame, columns, min_rows) for columns in candidates]

    def stability_map(d_in: int) -> float:
        return float(d_in)  # one row changes one group's size, so it moves each score
         ↪ by at most 1

    return dp.t.make_user_transformation(
        input_domain=input_domain,
        input_metric=input_metric,
        output_domain=dp.vector_domain(dp.atom_domain(T=float, nan=False)),
        output_metric=dp.linf_distance(T=float),  # not monotonic: a person's rows may
↪  be added and removed
        function=function,
        stability_map=stability_map,
    )
```

The output metric is the L-infinity distance, the largest change in any one score, because noisy max compares scores. The map is the identity, because one row belongs to one group of each candidate, so it moves each score by at most one.

The documentation declares the metric monotonic, which halves the cost of noisy max. That is right when neighbours differ by one row added or one row removed, because every score then moves the same way. Here a person has up to 36 rows, and a neighbouring dataset can add some and remove others, so the scores can move both ways. So the script keeps the metric plain and pays the factor of two (the full edition's chapter on sensitivity).

The measurement chains the scores into noisy max and names the chosen candidate, and it is registered so that a context can run it.

```
def make_select_grouping_cols(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str,
↪  ...]], min_rows: int, scale: float
) -> dp.Measurement:
    """Score the candidates, pick one by the exponential mechanism, and name its
     ↪ columns."""

    def name(index: int) -> tuple[str, ...]:
        return candidates[index]  # post-processing: the index is already private

    scores = make_grouping_cols_score(input_domain, input_metric, candidates, min_rows)
    return scores >> dp.m.then_noisy_max(dp.max_divergence(), scale) >> name


dp.register(make_select_grouping_cols)


def frame_domain() -> dp.Domain:
```

```
    """A domain for a Polars DataFrame: the library has none, so membership is a Python
     ↪  check."""

    def member(value: object) -> bool:
        return isinstance(value, pl.DataFrame)

    return dp.user_domain("PolarsDataFrameDomain", member=member)
```

On the survey the candidates are every non-empty subset of six columns. Five are `SEX`, `ILOSTAT`, `YEAR`, `QUARTER` and `AGE`, in seven bands, and the sixth is the person identifier `PIDENT`. A group counts in a score when it has at least 100 rows, a public choice. The context is built over the collected frame, with the user domain passed as `domain`, the privacy unit of 36 contributions and a budget of $\varepsilon = 1$. The script leaves out the scale, so that the context searches for one against the map of the plugin.

```
LFS: 3,811,932 rows; candidate columns SEX, ILOSTAT, YEAR, QUARTER, AGE, PIDENT; 63
 ↪  candidate sets
distinct values: SEX 2 ILOSTAT 4 YEAR 9 QUARTER 4 AGE 7 PIDENT 1,682,306
score: groups of at least 100 rows
rank 1: SEX, ILOSTAT, YEAR, QUARTER, AGE score 996 of 1,105 groups
rank 2: ILOSTAT, YEAR, QUARTER, AGE score 504 of 570 groups
rank 3: SEX, YEAR, QUARTER, AGE score 432 of 470 groups
sets with PIDENT: 32, largest score 0
select scale 72: map(36)=1.0 monotonic would be 0.5
context query: param scale=72.0
within 910 of the best: 10 candidates
```

The scores, computed at development time on this public survey, have the shape the documentation describes. The five columns without the identifier score best, and the next set scores about half of that. Every set with `PIDENT` scores zero, because no person has a hundred rows. The map is twice the sensitivity over the scale, so the search finds a scale of 72 for an $\varepsilon$ of 1.

By the argument used for the medians, a set that scores more than 910 below the best wins at most once in ten thousand runs. Ten sets are within that bound. The best beats the runner-up by 492, under seven noise scales, so it wins most runs, but any of the ten can appear. That spread is the price of the person-level unit, because a person can move each score by 36 while the scores are only in the hundreds. A higher minimum group size lowers every score and makes the choice more random. A larger budget is the only other lever.

```
released (one run):
  python laplace of 10.0: 9.6979  (within 6.0 with probability 0.95)
  repeated sum of ['0', '1', '2', '3'] with laplace scale 1: 13  (true 12)
  theil-sen: slope 2.1549 intercept 1.1111  (medians at the cuts -2.1212 and 4.3434)
  grouping columns: SEX, ILOSTAT, YEAR, QUARTER, AGE
```

The Python Laplace landed within its error bar, and the repeated sum within one of the truth. Both private medians fell inside their candidate ranges, so the slope and the intercept are near 2 and 1, and the chosen columns are the five that score best. Your run can differ in each of these.

## 13.8. Summary

Table 13.1.: The plugin constructors, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.enable_features("honest-but-curious")` | Unlocks the plugin constructors | The library cannot check a Python map; the proof is yours |
| `make_user_measurement(...)` | A measurement from your function and privacy map | Compare the map with a built-in cousin where one exists |
| `make_user_transformation(...)` | A transformation from your function and stability map | Justify the map and the output domain |
| `user_domain`, `user_distance`, `user_divergence` | Your own domain, metric and measure | Identifiers name one thing; a user measure cannot be composed |
| `new_privacy_profile`, `new_function` | A profile from a curve; a post-processor | No global state, no data-dependent exceptions |
| `dp.register(make_thing)` | `context.query().thing(...)` | Take `input_domain` and `input_metric` first |
| A plugin in a context | Charged its share of the budget, not its map | A cheap plugin does not give its share back |
| Theil-Sen from plugins | Pair, predict at two cuts, two private medians, solve | Use the vetted estimator for real work |
| Grouping columns by noisy max | Score each set by its groups above a minimum, pick one | Monotonic only for one-row units |

## 13.9. References


- OpenDP documentation: “Plugins”, “Measurement example”, “Transformation example”, “Context API Plugins”, “Thiel-Sen Regression” and “Privately Selecting Grouping Columns” in the API user guide.
- Theil (1950) and Sen, “Estimates of the regression coefficient based on Kendall’s tau” (1968): the estimator behind the regression example.
- McSherry and Talwar, “Mechanism Design via Differential Privacy” (2007): the exponential mechanism behind noisy max.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the maps a plugin must satisfy.
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

# 14. Statistical modelling

Every release so far has been a table of statistics, such as a count, a sum, a mean or a quantile per group. Often the question is about a relation instead, such as how many hours people actually work for each hour they usually work, or which directions of a table carry its variance. OpenDP ships two models for such questions, a differentially private linear regression and a differentially private principal component analysis (PCA), both with interfaces like those of scikit-learn. The one idea of this chapter is that a fitted model is a release like any other. Its cost is the privacy map of the measurement behind `fit`, not the argument named `epsilon`.

You will fit the regression on the documentation's generated points and on the Labour Force Survey (LFS), read its privacy loss, and see why its sensitivity is bounded. You will fit the PCA on the documentation's generated table and compare it with NumPy's exact decomposition. Then you will meet the NumPy domain under both estimators, and the questions a reviewer should ask about a private model.

The script opendp_statistical_modelling.py runs every step on OpenDP 0.16.0 with scikit-learn 1.8.0. A seeded NumPy generator makes the toy data and fixes how the regression pairs its rows. So the exact fits, the bounds, the cut points and every privacy map are the same on every run. The released fits change, because OpenDP's generator cannot be seeded, and they appear in one block marked as one run.

```
uv run python code/part04-extending/opendp_statistical_modelling.py
```

## 14.1. A model or a table?

The documentation's page on statistical modelling makes one point before any code. When you know which relation you need, a model made for it spends the whole budget on the few numbers that answer the question. A slope costs two private medians, as the next section shows, while a table fine enough to read the slope from would cost a noisy count in every cell. In the same way, the PCA releases a few directions, not a full covariance matrix.

The flexible alternative is synthetic data, which answers any later question and can go to people who need microdata. The price is accuracy, because a copy that serves every question answers each one less accurately than a direct release under the same budget (AIM and MST). So release a table when the questions are counts, a model when the question is a relation, and synthetic data when the questions are not yet known.

The two models are narrow on purpose. The regression takes one explanatory column and one response, and the PCA takes a numeric table whose rows have a bounded length. For many explanatory variables, use the methods of the full edition's chapter on private empirical risk minimization, which OpenDP does not ship.

## 14.2. Linear regression

A least-squares line is a poor thing to release directly. Its slope is a ratio of sums whose sensitivity is unbounded unless you clip both columns, and clipping changes the line. OpenDP ships a **Theil-Sen regression**, which uses medians instead. It pairs the rows at random, fits the line through each pair, and reads each line at two fixed cut points on the x axis. At each cut point it releases the median of those readings with a private quantile. Then it solves the two medians for a slope and an intercept, which is post-processing and costs nothing.

The sensitivity is bounded because of how the private median chooses. It scores every candidate on a fixed grid by how many readings lie on either side of it. Then the exponential mechanism picks a candidate with a probability that favours high scores. One person's row sits in one pair, so it moves one reading and changes the score of any candidate by at most one. How far the reading moved does not matter, so the estimator needs no clipping and survives a person who reports a thousand hours. The full edition's chapter on selection derives the exponential mechanism and the private quantile, and its chapter on sensitivity in the full edition explains why a count of readings is the quantity to bound.

```
def make_regression(
    x_bounds: tuple[float, float], y_bounds: tuple[float, float], scale: float, runs:
 ↪  int = 1
) -> dp.sklearn.linear_model.LinearRegression:
    """The private Theil-Sen regression under pure differential privacy, with public
     ↪  bounds on x and y."""
    return dp.sklearn.linear_model.LinearRegression(
        dp.max_divergence(),  # the output measure: the privacy loss is read in epsilon
        x_bounds=[x_bounds],  # fixes the two cut points, a quarter and three quarters
 ↪  of the way across
        y_bounds=y_bounds,  # the range over which the candidate medians are spread
        scale=scale,  # the noise scale of each of the two private medians
        runs=runs,  # how many random pairings to pool; each multiplies the privacy
 ↪  loss
    )
```

The first argument is the output measure, and `dp.max_divergence()` asks for pure differential privacy (DP), whose parameter is the privacy loss $\varepsilon$. The bounds are public and never clip the data. The x bounds only place the two cut points, at the fractions `fraction_bounds` of the way across, a quarter and three quarters by default. The y bounds only fix the grid of `candidates_count` evenly spaced candidates for each median, a hundred by default. The scale is the noise scale of each private median, and `runs` repeats the random pairing and pools the readings. Plugins builds the same estimator from plugins, as the documentation does.

```
regression space: NPArray2Domain(num_columns=2, nan=True, T=f64) SymmetricDistance()
 ↪  MaxDivergence
regression map: scale=1 runs=1 -> epsilon=2.0
regression map: scale=2 runs=1 -> epsilon=1.0
regression map: scale=2 runs=3 -> epsilon=3.0
```

The `measurement` attribute of the estimator reads like any other measurement. Its input is a two-column NumPy array under the symmetric distance, so its map at distance 1 is the privacy

loss for adding or removing one row. Two medians at scale 1 cost $\varepsilon = 2$, and doubling the scale halves that. Three runs at scale 2 cost $\varepsilon = 3$, because a row then sits in three pairs and moves three readings. More runs make the median better defined, but at a fixed budget they force a larger scale, so they rarely pay on a large dataset.

The documentation's toy data is a hundred points with a true slope of 2 and an intercept of 1, here from a seeded generator. The script prints the exact values to read a fit against.

```
toy: n=100 cut points=[-1.5, 1.5] candidate step=0.2020
toy least squares: slope=2.0275 intercept=0.9725
toy median line, no noise: slope=2.0379 intercept=1.0786 readings=50
```

The candidate step is the y range divided into 99 intervals, about 0.2. The cut points are 3 apart, so one grid step at one median changes the slope by about 0.07, before any noise. The median line is what the estimator would return with exact medians and the seeded pairing, from the 50 readings of 100 points. It is close to least squares because the noise in the toy data is symmetric.

### 14.2.1. Actual hours against usual hours on the survey

The survey question is how many hours people actually worked in the reference week for each hour they usually work. The script takes the people at work in the fourth quarter of 2013 with both hours recorded. The survey codes hours from 0 to 98, so the bounds are public, and it drops the code 99, "not applicable". It also keeps the first row of each person, so that the privacy unit of the fit is one person (Identifier truncation). These steps run in Polars before the array reaches the measurement, so they are your claim and not the library's.

```
survey: working people in 2013 Q4 with hours recorded: 36,914 (one row each)
survey: of them with 0 actual hours that week: 4,109
survey: bounds=(0.0, 98.0) cut points=[24.5, 73.5] candidate step=0.9899
survey least squares: slope=0.8582 intercept=-0.1935
survey median line, no noise: slope=1.0000 intercept=0.0000 medians=[24.50, 73.50]
survey readings: 16,463 pairs; within one candidate step of the medians: 5,606 and
 ↪  5,252
```

The two exact fits disagree, and that is the lesson. Of the 36,914 people, 4,109 worked no hours that week, through leave or illness, and they pull least squares down to 0.86 actual hours per usual hour. The median line ignores them, because most pairs of people lie on the line where actual equals usual. So its medians are the cut points themselves, and its slope is exactly 1. A private Theil-Sen fit therefore estimates the typical relation, not the average one, and the median line is the exact value to compare it with. The readings are fewer than half the people, because a pair with equal usual hours has no slope, and thousands of them lie within one grid step of each median.

```
model = make_regression(HOURS_BOUNDS, HOURS_BOUNDS, scale=REGRESSION_SCALE)
epsilon = model.measurement.map(1)  # the privacy loss for adding or removing one
 ↪  person's row
survey_slope, survey_intercept = fit_regression(model, hours, seed)
```

```
survey private fit: scale=2 epsilon=1.0
```

The fit at scale 2 costs $\varepsilon = 1$ for one person. Like scikit-learn, `fit` takes the explanatory column as a two-dimensional array and the response as a vector. It returns a fitted scikit-learn `LinearRegression`, so `coef_`, `intercept_` and `predict` work as usual.

## 14.3. Principal component analysis

A PCA finds the directions along which a table varies most. OpenDP's version releases three things under one budget. They are the mean of the rows, the singular values, which say how much variance each direction carries, and the components, which are the directions.

The algorithm, from Amin and others (2019), scales every row into a ball of radius `row_norm` and centres the rows on a private mean. Then it forms the matrix of sums of squares and cross-products. It releases the eigenvalues of that matrix with Laplace noise, because replacing one row moves their sum by at most twice the squared row norm. Then it draws the eigenvectors one at a time with the exponential mechanism on the sphere, which favours directions of high variance, and removes each drawn direction from the matrix.

So two facts must be public. The row norm is the sensitivity of the whole algorithm, and every map below is in units of it. The number of rows is part of the domain, which gives the mean a known divisor. So neighbouring tables have the same size with one row replaced, which is distance 2 under the symmetric distance. The library declares both facts and does not check them, as the reviewer's section shows. The budget is split in three equal parts for the mean, the eigenvalues and the eigenvectors, and the third for the eigenvectors is split again over the draws.

The table is the documentation's, ten thousand centred rows of four columns from a normal distribution with a random covariance. A seeded generator makes the exact decomposition reproducible.

```
pca table: 10,000 rows x 4 columns, row length mean=8.53 max=36.56
pca table: rows longer than 1: 99.9%  longer than 40: 0.0%
pca exact, raw rows: singular values=[901.9, 278.6, 222.2, 156.9]
pca exact, rows clamped to 1: singular values=[80.48, 40.67, 34.29, 26.29]
pca exact, rows clamped to 1: first component=[-0.614, -0.534, -0.526, -0.247]
```

The documentation fits with `row_norm=1.0`, and the first two lines show what that means here. Almost every row is longer than 1, so the clamp scales almost every row onto the unit sphere. The algorithm can then only estimate the singular values of the clamped table, about 80 instead of about 900 for the first. The directions survive the clamp, because scaling a row does not turn it, so the release is compared with the first component of the clamped table. The script also fits with a row norm of 40, which clamps nothing, to show the other side of the trade.

```
def make_pca(epsilon: float, row_norm: float, n_components: int | None = None) ->
 ↪  dp.sklearn.decomposition.PCA:
    """The private PCA of the generated table, with the two public facts the algorithm
     ↪  needs."""
```

```
    return dp.sklearn.decomposition.PCA(
        epsilon=epsilon,
        row_norm=row_norm,  # public: every row is scaled into a ball of this radius
 ↪  before anything else
        n_samples=PCA_ROWS,  # public: the exact number of rows, which fixes the domain
        n_features=PCA_COLUMNS,
        n_components=n_components,  # None releases every component
    )
```

```
pca = make_pca(PCA_EPSILON, ROW_NORM)
measurement = pca.measurement()  # the measurement behind fit(), for its map and for
 ↪  compositors
pca.fit(table)  # the private release: the mean, the singular values and the components
```

```
pca model: PCA(epsilon=1.0, n_components=4, n_features=4, n_samples=10000,
 ↪  row_norm=1.0)
pca space: NPArray2Domain(size=10000, num_columns=4, nan=True, T=f64)
 ↪  SymmetricDistance() MaxDivergence
pca map: map(2)=2.0 map(1)=2.0 check(2, 1.0)=False check(2, 2.0)=True
make_private_pca unit_epsilon=1: map(2)=1.0 map(1)=1.0
pca n_components=2: map(2)=2.0
pca composition of epsilon 1 and 0.5: map(2)=3.0
```

Read the third line twice. The model was built with `epsilon=1.0`, but the map of its measurement at distance 2, one replaced row, is 2.0. And `check` confirms that this fit does not stay within $\varepsilon = 1$.

The reason is in the constructor. The `PCA` class calls the framework's `make_private_pca` with twice its `epsilon`, divided by `n_changes`, the number of rows one person may change. The fourth line shows that `make_private_pca` with a unit budget of 1 has a map of 1.0 at distance 2. So in version 0.16.0 a `PCA(epsilon=1.0)` fit costs $\varepsilon = 2$ per changed row by its own map, and the map is what you should quote. The map is flat below distance 2, because the algorithm is an adaptive composition declared at that distance (Combinators).

The fifth line shows that two components leave the total unchanged, and only the split inside changes. The library draws one eigenvector fewer than there are columns, because the others fix the last direction. So the third for the eigenvectors is split over three draws for four components and over two draws for two, and each draw gets more. The `measurement()` method is there for the compositors. The last line composes a fit at `epsilon=1.0` with one at `epsilon=0.5`, and the maps add to 3.0 at distance 2.

## 14.4. The NumPy domain underneath

Both estimators use `dp.numpy.array2_domain`, the domain of two-dimensional NumPy arrays, described by a number of columns, an optional number of rows and the element type. It can also have a norm, its order `p` and an origin, which say that every row lies within that norm of the origin. A domain with a norm is a promise about the data, and `make_np_clamp` is the transformation that makes the promise true.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
rows = dp.numpy.array2_domain(num_columns=2, T=float)
clamp = dp.numpy.make_np_clamp(rows, dp.symmetric_distance(), norm=1.0, p=2,
 ↪  origin=np.array([1.0, 2.0]))
print(clamp(np.array([[1.0, 2.0], [4.0, 6.0], [1.5, 2.0]])).tolist())  # [[1.0, 2.0],
 ↪  [1.6, 2.8], [1.5, 2.0]]
print(clamp.map(1))  # 1
print(clamp.output_domain)  # NPArray2Domain(origin=[1. 2.], norm=1.0, p=2,
 ↪  num_columns=2, nan=False, T=f64)
```

The clamp leaves the first and third rows alone, because they lie within distance 1 of the origin, and pulls the second row from distance 5 back to the sphere. Its stability map is the identity, because clamping each row on its own neither adds nor removes rows (Transformations). Its output domain records the norm, the order, the origin, and that no value is missing.

That output domain is what `make_private_pca` prefers to receive. Such a domain states that the rows are already bounded around a known centre. So the algorithm skips the private mean and its third of the budget.

```
bounded domain: NPArray2Domain(origin=[0. 0. 0. 0.], norm=1.0, p=2, size=10000,
 ↪  num_columns=4, nan=True, T=f64)
pca on the bounded domain: map(1)=0.50 map(2)=1.00 map(4)=2.00
```

The script passes an explicit split as a `PCAEpsilons`, with half for the eigenvalues, a sixth for each of three eigenvectors and nothing for the mean. The map is then linear in the distance, not flat, because no compositor is declared in between. Use this route when the centre is public, for example for scores on a known scale, and the class when it is not.

## 14.5. What a reviewer should ask

The first question is the privacy unit. Both estimators count rows, so the claim “one person” holds only if each person has one row, which on the survey was your own Polars step. The second is the bounds. The bounds of the regression and the row norm and row count of the PCA must come from the code book or the domain. A bound read from the data leaks it. The third is which features are on, because a feature is a statement about what you accept on trust.

```
regression without honest-but-curious: Attempted to use function that requires
 ↪  "contrib", "honest-but-curious", but not enabled
pca without honest-but-curious: Attempted to use function that requires "contrib",
 ↪  "honest-but-curious", but not enabled
pca without idealized-numerics: Attempted to use function that requires "contrib",
 ↪  "idealized-numerics", but not enabled
regression without idealized-numerics: constructed
pca with n_samples one short of the table: fitted without complaint
```

Both estimators need `honest-but-curious`, because they use user-defined transformations whose functions and stability maps the library takes as written. For example, the pairing step of the regression claims that one row moves `runs` readings, and the library records that claim without proof. The PCA also needs `idealized-numerics`, which concedes that its proofs hold for real arithmetic and not for floating point. Every guarantee of the PCA rests on that concession, and the library's own source notes that it knows no floating-point-safe sampler for the eigenvector draw. The regression constructs without the flag, because its private medians run in the Rust core and only the pairing is Python.

The last line is the trap for a reviewer. The number of rows is part of the domain of the PCA, but the library does not check that the array has that many rows. So a wrong `n_samples` is a wrong guarantee that nothing reports. The clamp enforces the row norm, and the bounds of the regression need no enforcement because its sensitivity does not depend on them. Limitations and troubleshooting collects the other things the library does not check.

## 14.6. The releases of one run

```
released (one run):
  toy private fit: slope=1.9529 intercept=1.0101
  survey private fit: slope=1.0101 intercept=-0.9899
  survey private fit at 35 usual hours: 34.36 actual
  pca row_norm=1: singular values=[80.48, 40.66, 34.28, 26.37]
  pca row_norm=1: |cos| with exact components=[1.000, 0.998, 0.996, 0.996]
  pca row_norm=1: mean=[0.001, 0.002, 0.001, 0.001]  explained variance ratio=[0.648,
 ↪  0.165, 0.117, 0.070]
  pca row_norm=40: singular values=[906.8, 268.6, 212.6, 153.2] (exact [901.9, 278.6,
 ↪  222.2, 156.9])
  pca row_norm=40: |cos| with exact components=[0.996, 0.659, 0.426, 0.086]
  pca n_components=2: returned the model: True  singular values=[80.41, 40.64]
  pca n_components=2: |cos| with exact components=[0.999, 0.999]
```

The toy fit is about one grid step off the least-squares line at one cut point, which is what a private median on 50 readings at $\varepsilon = 2$ gives. The test allows the slope within 0.3 of the least-squares slope and the intercept within 0.5, about three grid steps at either cut point. A candidate three steps from the best scores lower by every reading in between, and the exponential mechanism makes such a candidate rare at this budget.

The survey fit is the same on every run. The medians sit exactly at the cut points, and the nearest candidates on the grid give a slope of 1.0101 and an intercept of about $-1$. Thousands of readings lie within one grid step of each median, so the next candidate scores thousands worse than the best, and noise at scale 2 never overturns that gap. The error of the fit is therefore the grid, not the noise, and a finer `candidates_count` removes it at no cost in privacy. The test allows the slope within 0.1 of the median line and the intercept within 4 hours, two grid steps at the lower cut point. The fit predicts about 34.4 actual hours for someone who usually works 35.

At row norm 1 the PCA recovers the singular values of the clamped table to within a tenth, and all four directions to a cosine above 0.99. Its mean is within a few thousandths of the true zero. The test allows each singular value within 1 and each cosine above 0.9. Both bounds leave a wide

margin, because the Laplace noise on the eigenvalues is a few units against eigenvalues in the thousands, and the four directions are well separated.

At row norm 40 the singular values are those of the raw table, within a few percent, but only the first direction is right. The concentration of the eigenvector sampler falls with the square of the row norm, so the last three directions are close to random. The test checks only the first singular value, within 30, and the first cosine, above 0.9. The two-component fit returns the model itself, and its two directions are as good as the first two of the four-component fit.

## 14.7. Summary

Table 14.1.: The calls and concepts of private modelling, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.sklearn.linear_model.LinearRegression(measure, x_bounds, y_bounds, scale, runs)` | A Theil-Sen regression from two private medians | Bounds fix the cut points and the grid; they never clip the data |
| `model.measurement.map(1)` | The privacy loss of the fit per added or removed row | Two medians at scale $s$ cost $2\,\text{runs}/s$; quote this, not the arguments |
| `candidates_count` | The grid the medians are chosen from | On a large dataset the grid, not the noise, is the error; refine it for free |
| The median line against least squares | What the estimator targets | Compare a Theil-Sen fit with the median line; outliers pull least squares only |
| `dp.sklearn.decomposition.PCA(epsilon, row_norm, n_samples, n_features, n_components)` | A private mean, the singular values and the components | `row_norm` and `n_samples` are public facts the library does not check |
| `pca.measurement()` | The measurement behind `fit`, for maps and compositors | In 0.16.0 its map at distance 2 is twice `epsilon`; read the map |
| `n_components` | How many directions to draw | Fewer draws give each more budget at the same total |
| `row_norm` | The sensitivity scale of the whole algorithm | Tight: magnitudes are the clamped table's, directions good. Loose: magnitudes right, directions poor |
| `dp.numpy.array2_domain(norm, p, origin)` and `make_np_clamp` | The bounded domain and the transformation that enforces it | A known origin skips the private mean and its third of the budget |
| `honest-but-curious`, `idealized-numerics` | Trust the plugins' maps; accept real-arithmetic proofs | Both estimators need the first; the whole PCA rests on the second |

## 14.8. References

- OpenDP documentation: “Statistical Modeling”, “Linear Regression” with the API reference of `opendp.extras.sklearn.linear_model`, “Differentially Private PCA”, and “Theil-Sen Regression” in the plugins user guide.
- Theil, “A Rank-Invariant Method of Linear and Polynomial Regression Analysis” (1950), and Sen, “Estimates of the Regression Coefficient Based on Kendall’s Tau” (1968): the median-of-slopes estimator.
- Dwork and Lei, “Differential Privacy and Robust Statistics” (2009): why robust estimators such as medians are the natural private ones.
- Amin, Dick, Kulesza, Munoz and Vassilvitskii, “Differentially Private Covariance Estimation” (2019): the eigenvalue and eigenvector mechanisms behind the PCA.
- The full edition’s chapters on selection in the full edition, sensitivity in the full edition and private empirical risk minimization in the full edition derive the results this chapter states.

# 15. Limitations and troubleshooting

Every chapter so far has shown what OpenDP guarantees, and this one shows where the guarantee stops. The one idea is that the guarantee covers exactly what passes through a measurement with a checked proof. The rest is yours to account for, such as the arithmetic that the proof assumes, the code before the context, and channels other than the output. Most refusals of the library keep you on the right side of that line, so an error message is usually the guarantee doing its job.

The chapter follows the documentation's pages on limitations, on features and on troubleshooting. You will read what the library does not promise, see what each feature flag gates, and mark out what lies outside the guarantee. Then you will meet the error messages of the earlier chapters again, each with its cause and its fix, and see what a reviewer checks before trusting a release.

The script opendp_limitations.py reproduces every message on this page with OpenDP 0.16.0 and Polars 1.36.1. It uses tiny lists for the framework calls and the France Labour Force Survey (LFS) sample behind a context for the Polars ones. The messages, the sensitivities, the scales and the error bar are the same on every run. OpenDP's noise cannot be seeded, so the one released count changes, and it appears at the end in a block marked as one run.

```
uv run python code/part04-extending/opendp_limitations.py
```

## 15.1. What OpenDP does not promise

The documentation's page on limitations names five things, and each changes how you deploy the library. The first is arithmetic. A privacy proof is about real numbers, but a computer adds floating-point numbers, which round. Casacuberta, Shoemate, Vadhan and Wagaman (2022) found that several libraries computed the sensitivity of a sum as if the arithmetic were exact. The rounding made the true sensitivity larger, so the noise was too small and the stated $\varepsilon$ was wrong.

OpenDP puts the rounding into the proof. To see this, the script builds a float sum and an integer sum of values between 0 and 80 (Transformations). Then it asks each for its sensitivity when 36 rows are added or removed.

```
def sum_sensitivities(d_in: int) -> tuple[float, int]:
    """The sensitivity of a float sum and of an integer sum of values in [0, 80], at
     ↪  `d_in` added or removed rows."""
    float_sum = (dp.vector_domain(dp.atom_domain(bounds=HOURS_BOUNDS)),
 ↪  dp.symmetric_distance()) >> dp.t.then_sum()
    int_sum = (dp.vector_domain(dp.atom_domain(bounds=(0, 80))),
 ↪  dp.symmetric_distance()) >> dp.t.then_sum()
    return float_sum.map(d_in), int_sum.map(d_in)
```

```
float sum of 36 rows in [0, 80]: sensitivity 2880.0000007451
int sum of 36 rows in [0, 80]: sensitivity 2880
```

In exact arithmetic both would be 36 times 80. The map of the float sum adds a small term for the rounding error, and the integer sum needs none, because integers add exactly. The term grows with the number of rows, so a float sum on the LFS needs a public bound on that number (Essential statistics). The library puts any component that still assumes exact arithmetic behind the `idealized-numerics` flag. So prefer integer statistics where you can, and otherwise read the scale of a float statistic from the library, not from the formula.

The second limitation is side channels. OpenDP is not hardened against them, so the time, the memory and the cache lines of a query may depend on the data. No proof covers what an attacker learns from them. So the deployment question is who can interact with the running system. An analyst who submits queries through a queue is in a different position from one with a shell on the machine. Keep the second kind out, or accept that the guarantee is weaker than the $\varepsilon$ you record.

The third is vetting. Every component of OpenDP is meant to come with a written proof and a review of that proof, and until the review is finished the component needs the `contrib` flag. The documentation expects the review to find bugs in code and in proofs. So every release in this book rests on proofs that no second reader has checked yet. Record which components a release used, so that you can revisit it if one is corrected.

The fourth is stability. Until version 1.0.0, OpenDP's interfaces may change between minor releases. So pin the version, as this book pins 0.16.0. Keep a test that reruns your release code, so that a changed default fails the test before it changes a number. The fifth is software quality, because no deployment can rule out an unknown bug. So keep a record of the run from which you can compute the guarantee again, as Release and governance in the practitioner edition in the practitioner edition describes.

## 15.2. The feature flags

The feature flags are how the library asks for your consent to each of those limitations. There are four, plus a deprecated alias, and all are off until you call `dp.enable_features`. A first release switched `contrib` on without showing what happens otherwise, so the script switches it off, tries the Laplace mechanism of that chapter, and switches it back on.

```
dp.disable_features("contrib")
without_contrib = refusal(lambda: space >> dp.m.then_laplace(scale=1.0))
dp.enable_features("contrib")
```

```
laplace without contrib: OpenDPException: Attempted to use function that requires
 ↪  "contrib", but not enabled. See
 ↪  https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪  enable_features("contrib")
```

The helper `refusal` runs a call and returns the type and message of the exception. Every flag gives the same message, which names the flag and the call that enables it. So to find which calls of a script depend on a flag, run the script with that flag off.

The second flag is `honest-but-curious`. It gates constructors whose guarantee depends on the correctness of your arguments, mainly the plugins of Plugins. For example, `make_user_measurement` takes a privacy map as a Python function, and the library cannot check that the function tells the truth. The name describes the adversary the flag assumes, who is honest when building the measurement and only curious when reading its outputs. The script builds the smallest possible plugin, which releases its input unchanged and claims a privacy loss equal to the distance between inputs.

```
def plugin_measurement() -> dp.Measurement:
    """A plugin measurement whose privacy map is only a claim: the library cannot check
     ↪  `identity_map`."""
    return dp.m.make_user_measurement(
        dp.atom_domain(T=float, nan=False),
        dp.absolute_distance(T=float),
        dp.max_divergence(),
        function=identity,
        privacy_map=identity_map,
    )
```

```
plugin without honest-but-curious: OpenDPException: Attempted to use function that
 ↪  requires "contrib", "honest-but-curious", but not enabled. See
 ↪  https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪  enable_features("contrib", "honest-but-curious")
plugin with honest-but-curious: map(1.0)=1.0
```

With the flag on, the plugin reports whatever its map says, here a loss of 1 at distance 1, and nobody verified that number. So turn the flag on where you write and prove plugins, never where an untrusted analyst submits queries, because such an analyst could register a map that claims any loss. The same flag gates `OnceFrame.lazy()`, which gives back a released Polars plan that could run twice.

The third flag is `idealized-numerics`, which marks components whose proof assumes exact arithmetic. In 0.16.0 none of the constructors in this book needs it. The float Laplace mechanism and the float sums above build with the flag off, because their proofs account for rounding. The older name of the flag, `floating-point`, still works and gives a warning.

```
float laplace without idealized-numerics: no error
enable_features("floating-point"): DeprecationWarning: "floating-point" is deprecated.
 ↪  Use "idealized-numerics" instead.
assert_features("idealized-numerics") with the alias on: no error
assert_features("idealized-numerics") after disabling it: OpenDPException: Attempted to
 ↪  use function that requires "idealized-numerics", but not enabled. See
 ↪  https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪  enable_features("idealized-numerics")
assert_features("contrib", "honest-but-curious"): OpenDPException: Attempted to use
 ↪  function that requires "contrib", "honest-but-curious", but not enabled. See
 ↪  https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪  enable_features("contrib", "honest-but-curious")
```

The last three lines use `dp.assert_features`, which raises the error of a gated constructor without building anything. Every gated constructor calls it first. At the top of a script, one such line also states every flag that the script relies on. The fourth flag, `rust-stack-trace`, changes no guarantee, because it only adds the Rust frames to an error message, as the section on reading an exception shows.

## 15.3. What lies outside the guarantee

The guarantee covers what happens to the data inside a measurement, and four things happen outside it in every analysis. The first is loading the data, which the documentation leaves to a trusted curator. One reason is that a CSV reader infers the type of each column from its values. The script reads three two-row files that differ in one value, then a longer file whose odd value comes after the rows that the inference reads.

```
for raw in (b"x\n1\n2\n", b"x\n1\n2.5\n", b"x\n1\nabc\n"):
    print(f"csv {raw!r}: column x inferred as
     ↪  {pl.read_csv(io.BytesIO(raw)).schema['x']}")
late_text = b"x\n" + b"1\n" * 200 + b"abc\n"  # a text value after 200 numbers
failed = refusal(lambda: pl.read_csv(io.BytesIO(late_text), infer_schema_length=100))
print(f"csv with a text value on row 201, inferred from 100 rows:
 ↪  {first_line(failed)}")
as_strings = pl.read_csv(io.BytesIO(late_text), infer_schema=False).schema["x"]
print(f"the same file with infer_schema=False: column x read as {as_strings}")
```

```
csv b'x\n1\n2\n': column x inferred as Int64
csv b'x\n1\n2.5\n': column x inferred as Float64
csv b'x\n1\nabc\n': column x inferred as String
csv with a text value on row 201, inferred from 100 rows: ComputeError: could not parse
 ↪  `abc` as dtype `i64` at column 'x' (column number 1)
the same file with infer_schema=False: column x read as String
```

So one person's value decides whether the column is integer, float or text, and a late value decides whether the load succeeds at all. If the result of the load is visible outside, it is a release that nothing accounted for. The documentation gives two fixes. Read a format that carries its schema, as this book does with Parquet. Or read every column as text with `infer_schema=False`, then cast it yourself. The LFS loader in `practitioner_datasets` converts the CSV to Parquet once, which is the curator's step, outside any guarantee.

The second thing outside the guarantee is public information. Every bound, imputed value, key list and margin in this book comes from the domain or from a published source. A bound read from the private data is a release of the data, and the library cannot tell the two apart.

The third is code that runs outside the context, which is why the data and the context told you to delete your reference to the frame. Collecting the frame yourself is not a query, so nothing counts it. The fourth is the error channel. Whether a query succeeds or fails is itself an output, so a failure that depends on the data is a release. OpenDP guards against this inside a query, for

example by replacing some Polars arguments so that an expression cannot fail on one value. The loading step and your own code have no such guard.

## 15.4. The error messages you will meet

The documentation's troubleshooting page has one entry, the type mismatch, and the earlier chapters met several more. This section gives each with its cause and its fix. The type mismatch comes first. OpenDP fixes the type of every argument from the domain of the data, and it refuses an argument of another type instead of converting it. The script puts three integers behind a context and clamps them with float bounds.

```
float bounds on integer data: TypeError: inferred type is f64, expected i32. See
 ↪  https://docs.opendp.org/en/stable/getting-started/trouble-shooting.html#inferred-type-is-xxx-expe
integer bounds on integer data: scale=10.0
```

The message names the type it inferred from your argument and the type of the data. To fix it, pass bounds of the type of the data, `(0, 10)` for integers, or cast the data first. The documentation names one common cause, a scalar $\varepsilon$ where a measurement expects an $(\varepsilon, \delta)$ pair. To show it, the script asks an approximate measurement, built with `make_approximate` as in Combinators, whether it stays within a scalar budget.

```
check(1.0, 1.0) on an approximate measurement: TypeError: Cannot coerce a non-tuple
 ↪  type to a tuple
check(1.0, (1.0, 0.0)) on the same measurement: True
```

The privacy loss of an approximate measurement is a pair, so its budget must be a pair too, and a pure budget is the pair $(\varepsilon, 0)$. The same message comes from `binary_search_param` with a scalar `d_out` on an approximate chain (Utilities and the Context API).

The next message comes from a domain that allows missing values. The library infers a list of Python floats as a domain that may contain NaN. A clamp refuses such a domain, because NaN is neither inside nor outside a bound.

```
clamp on floats that may be nan: OpenDPException: FailedFunction("Domain contains nan")
impute, then clamp: scale=80.0
```

The fix is the one of the first chapter, `impute_constant` before `clamp`, which maps the domain to one without NaN. In Polars the same need appears as `fill_null` before a `dp.sum`.

The next message comes from mismatched domains in a chain. The `>>` operator checks that the output domain of the left side equals the input domain of the right side, and refuses otherwise. The script clamps to one range and then chains a sum built for a wider one.

```
vectors = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = vectors >> dp.t.then_clamp((0.0, 10.0))
wide_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 100.0))),
 ↪  dp.symmetric_distance())
mismatch = refusal(lambda: clamp >> wide_sum)  # the clamp's output has bounds [0, 10],
 ↪  the sum wants [0, 100]
chained = clamp >> dp.t.then_sum()  # a partial constructor takes the clamp's output
 ↪  space, so they agree
```

```
clamp to [0, 10] then a sum built for [0, 100]: OpenDPException:
 ↪  DomainMismatch("Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
clamp to [0, 10] then then_sum(): sensitivity 10.0
```

The message prints both domains, so you can read that the bounds differ. The fix is the partial constructor `then_sum()`, which takes its input space from the output of the clamp (Transformations).

The remaining messages come from the Polars side, with the LFS behind a context of one query and a budget of $\varepsilon = 1$. The unit is the 36 rows of the data and the context.

```
def make_context(
    frame: pl.LazyFrame, delta: float | None = None, margins:
 ↪  Sequence[dp.polars.Margin] | None = None
) -> dp.Context:
    """The LFS behind a unit of 36 rows and a budget of epsilon = 1, with or without a
     ↪  delta, for one query."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=delta) if delta else
 ↪  dp.loss_of(epsilon=EPSILON),
        split_evenly_over=1,
        margins=margins,
    )
```

A float sum needs a margin with `max_length`, for the reason of the first section. Its sensitivity includes a rounding term that grows with the number of rows, so the proof needs a public bound on that number.

```
float sum without a margin: OpenDPException: MakeTransformation("must specify
 ↪  'max_length' in a margin with by=[]")
float sum with max_length=5,400,000: scale=2,881.04
```

The fix is the documentation's margin, `dp.polars.Margin(max_length=150_000 * 36)`. The scale is then a little above 2880, the sensitivity of the sum over $\varepsilon = 1$ in exact arithmetic. The

extra is the rounding term at 5.4 million rows. A grouped sum needs the same bound in the margin of its grouping, which is what `by=[]` in the message refers to.

The next three messages come from one query. The script builds a count and tries to collect it before it releases it. Then it releases the count, collects the result twice, and submits another query to a context whose one share is spent.

```
count_query = context.query().select(dp.len(signed=True))
before_release = refusal(lambda: count_query.collect())  # the plan has no data; only a
 ↪  released OnceFrame does
once = count_query.release()
dp_count = int(once.collect()["len"][0])
twice = refusal(lambda: once.collect())  # a second collection would be a second
 ↪  release
exhausted = refusal(lambda: context.query().select(dp.len(signed=True)).release())
```

```
count: scale=36.0 95% accuracy=108.34
collect before release: ValueError: You must call `.release()` before executing a
 ↪  query.
collect a OnceFrame twice: OpenDPException: FailedFunction("OnceFrame has been
 ↪  exhausted")
query after the budget is spent: ValueError: Privacy allowance has been exhausted
```

A `LazyFrameQuery` is a plan without data, so `collect` on it would compute nothing private. The library refuses anyway, so that the habit of collecting a plan never meets a frame that holds data. You can collect the `OnceFrame` from `release` only once, because the noise is drawn when the plan runs, so a second run would be a second release. The third message is the one that the first chapter ended on. None of the three is a bug to work around, so keep the DataFrame from the first `collect` if you need a value twice.

The last message is from Contingency tables. A contingency table whose keys or cuts cover every column has no private key set to protect, so its release spends no $\delta$. The library refuses a budget with a $\delta$ instead of leaving it unspent. The script uses $\delta = 10^{-7}$, the value of the grouping chapters.

```
contingency table with keys for every column under (epsilon, delta): ValueError: delta
 ↪  (1e-07) must be zero because keys and cuts span all columns
the same table under pure epsilon: ContingencyTableQuery
```

So state the budget as pure $\varepsilon$ when the keys are all public, or as the zero-concentrated budget $\rho$ of Utility and noise. Keep $\delta$ for the tables whose keys the library must discover. Two more messages from earlier chapters belong in this list. A plain `mean` is refused, because OpenDP has no proof for it. A grouping without keys or a $\delta$ is refused, because its key set is private (Grouping).

## 15.5. Reading an OpenDPException

Most of the messages above came as an `OpenDPException`, which is how an error in the Rust core reaches Python. Its `variant` is the kind of error, from a fixed list such as `DomainMismatch` or

`FailedFunction`, which code can match on. Its `message` is the text, and its `raw_traceback` is the Rust stack at the point of the error, which is recorded even if you do not display it. The script raises the domain mismatch again and reads the three, then switches on `rust-stack-trace` and prints the exception again.

```
try:
    _ = clamp >> wide_sum
except dp.OpenDPException as err:
    print(f"variant: {err.variant}")
    print(f"message: {first_line(err.message or '')}")
    print(f"rust frames recorded: {err.raw_traceback is not None}")
    dp.enable_features("rust-stack-trace")
    print(str(err).strip())
    dp.disable_features("rust-stack-trace")
```

```
variant: DomainMismatch
message: Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
rust frames recorded: True
Continued Rust stack trace:
    opendp_combinators__make_chain_tt
    opendp::combinators::chain::make_chain_tt::<opendp::ffi::any::AnyDomain,
 ↪  opendp::ffi::any::AnyDomain, opendp::ffi::any::AnyDomain,
 ↪  opendp::ffi::any::AnyMetric, opendp::ffi::any::AnyMetric,
 ↪  opendp::ffi::any::AnyMetric>
    opendp::combinators::chain::mismatch_error::<opendp::ffi::any::AnyDomain>
  DomainMismatch("Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
```

The frames read from the outside in: the entry point that Python called, the chaining function, and the helper that formats the mismatch. They help when the message does not say where in a long chain the library stopped, and they locate a bug in the Rust source. The list of variants may change between versions, so match on a variant only in tests you control. The errors that are not an `OpenDPException` come from the Python layer, a `TypeError` from argument conversion and a `ValueError` from the context and the query classes.

## 15.6. What a reviewer verifies

A release made with OpenDP claims that this table satisfies $(\varepsilon, \delta)$-differential privacy for this privacy unit, and Table 15.1 collects the checks of that claim. The first is the unit, because an $\varepsilon$ means nothing until you know what it protects. For example, `dp.unit_of(contributions=36)` protects a person only if nobody has more than 36 rows (Identifier truncation). The second is the budget, read back from the context.

The third is the flags, because with `honest-but-curious` the reviewer must read each privacy map that the library took on trust. The fourth is the source of each bound and key, and the fifth

is what ran outside the context. The full edition's chapter on the definition in the full edition says what the finished claim must state.

Table 15.1.: What a reviewer verifies before trusting a release made with OpenDP.

| Check | Question | Where to look |
|---|---|---|
| Unit | What does one privacy unit contain, and does anyone exceed it? | `privacy_unit` in the context; the identifier and any truncation |
| Budget | What was spent, under which measure, and was any release collected twice? | `context.current_privacy_loss()`; one `collect` per `OnceFrame` |
| Features | Which flags were on, and if `honest-but-curious`, which maps were claimed? | the `enable_features` line; every `make_user_*` call and `lazy()` |
| Public information | Where did each bound, imputed value, key list and margin come from? | the constants of the script, traced to their sources |
| Outside the context | What touched the data before and after the context? | the load and its schema; any code holding the frame |

## 15.7. Summary

Table 15.2.: The flags, the limits and the error messages of this chapter, with the rule of thumb for each.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.enable_features("contrib")` | Opts in to components whose proofs are not yet vetted | Every script in this book needs it; record it with the release |
| `honest-but-curious` | Opts in to constructors whose guarantee rests on your arguments, such as plugins and `OnceFrame.lazy()` | Only where you wrote and proved the map, never where an untrusted analyst queries |
| `idealized-numerics` | Opts in to components whose proof assumes exact arithmetic | No constructor in this book needs it in 0.16.0; its alias `floating-point` warns |
| `dp.assert_features(...)` | Raises the flag error without building anything | One line at the top of a script states what it relies on |
| `rust-stack-trace` | Adds the Rust frames to an exception's text | Switch on to locate a bug; it changes no guarantee |
| Loading the data | Outside the guarantee; CSV inference reads private values | Read Parquet, or `infer_schema=False` and cast |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `inferred type is f64, expected i32` | An argument's type differs from the data's | Match the data's type or cast first; a scalar $\varepsilon$ for a pair is the common case |
| `Domain contains nan` | A float domain may hold NaN, and clamp refuses it | `impute_constant` or `fill_null` before `clamp` |
| `DomainMismatch` | Two chained components disagree on the domain between them | Use `then_*` constructors; read the two domains in the message |
| `must specify 'max_length' in a margin` | A float sum needs a public bound on the rows | `dp.polars.Margin(max_length=...)` from a public source |
| `You must call .release() before executing a query` | A plan was collected before it was released | `release()` first, then `collect()` once |
| `OnceFrame has been exhausted` | A released frame was collected twice | Keep the DataFrame from the first `collect` |
| `Privacy allowance has been exhausted` | Every share of the budget is spent | The refusal is the guarantee; a new context is a new budget |
| `delta (...) must be zero because keys and cuts span all columns` | A table with public keys spends no $\delta$ | Budget the query with pure $\varepsilon$ or with $\rho$ |
| `OpenDPException` | Carries `variant`, `message` and `raw_traceback` | Match on the variant only in tests; expect the list to change before 1.0 |

## 15.8. References

- OpenDP documentation: "Limitations" and the feature listing in the API user guide; "Troubleshooting" and "Next Steps" in Getting Started; the note on data loading in "Working with Tabular Data" and the margin and float-sum notes in "Essential Statistics".
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the finite-precision sensitivity errors in earlier libraries, and the corrected proofs that OpenDP's sums follow.
- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (2012): the floating-point attack on the Laplace sampler.
- Jin, McMurtry, Rubinstein and Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (2022): timing and floating-point side channels in differential privacy libraries.
- The full edition's chapters on the definition in the full edition and discrete and floating-point-safe mechanisms in the full edition derive the results this chapter states; the practitioner edition's Release and governance in the practitioner edition gives the record a reviewer asks for.

# Part V.

# Synthetic data

The last part releases a dataset instead of a statistic. A synthetic dataset is a table of made-up rows whose statistics match those of the real table as closely as a privacy budget allows. OpenDP makes one in two steps. First it releases a contingency table, which holds the noisy counts of every combination of a few columns at a time. Then it fits a model to those noisy marginals and samples rows from it, which is post-processing and costs no more budget.

The thread of the part is that two choices decide the quality of the synthetic data: which marginals you measure, and how you split the budget among them. Contingency tables releases the marginals of a fixed workload with the `Fixed` algorithm. It explains keys, cuts and thresholds, and the `ContingencyTable` object with its error estimates. Synthetic data with AIM and MST lets the library choose the marginals, one round at a time with AIM or along a spanning tree with MST, and generates the rows. Evaluating and tuning synthetic data compares the result with the real data on the marginals, on a query outside the workload and on a downstream model. It also shows how the budget and the workload change the result.

Table 15.3.: The chapters of this part and the questions they answer.

| Chapter | Question it answers |
|---|---|
| Contingency tables | How do I release the noisy counts of a set of marginals, and what does the table object hold? |
| Synthetic data with AIM and MST | How do I let the library choose the marginals, and how do I generate synthetic rows? |
| Evaluating and tuning synthetic data | How good is the synthetic data, and what changes it? |

At the end of this part you can generate a private synthetic copy of a table, state its guarantee, and measure how far it is from the original.

# 16. Contingency tables

Suppose you want to publish a whole table, such as how many people of each age group usually work each number of hours, by employment status and sex. A **contingency table** is that table, with a count for every combination of the values of several columns. Grouping released one for two columns, and this chapter asks what changes with four columns, or forty. The answer is that you release a few **marginals**, the same counts for two or three columns at a time, and fit one model that agrees with all of them. The model then answers any count over those columns, consistently, from the budget already spent.

The chapter follows the documentation's page on contingency tables. You will set up a context under zero-concentrated differential privacy (zCDP) and say which columns have known keys or cut points. Then you will release a fixed workload of marginals and read the `ContingencyTable` that comes back. At the end you will read a projection of the table against its error bar and add a marginal that the first release left out.

The script opendp_contingency_tables.py runs every step on OpenDP 0.16.0 with `mbi` 1.1.0, the Private-PGM version that OpenDP pins, on the Labour Force Survey (LFS) sample. The budgets, scales, thresholds, keys and error bars it prints are the same on every run. OpenDP's noise cannot be seeded, so the page shows the released counts in one block marked as one run.

```
uv run python code/part05-synthetic-data/opendp_contingency_tables.py
```

## 16.1. From grouped counts to marginals

The grouped counts of Grouping fail in two ways as you add columns. The first is size, because each column multiplies the number of cells by its number of values, and the four raw columns of this sample already make 4,648 cells. The second is noise. The noise on a cell does not depend on how many records the cell holds. So when the same records spread over more cells, each count gets smaller while its noise stays the same. Soon most cells are noise.

A marginal is the contingency table of a subset of the columns. Once the columns are binned as below, the two-way marginal of `AGE` and `HWUSUAL` has sixteen cells, each with a count large enough to survive its noise. Several low-order marginals keep the relations between the columns that share a marginal and give up the higher-order ones. The documentation's workload has two marginals, `AGE` with `HWUSUAL` and `HWUSUAL` with `ILOSTAT`.

Noisy marginals released separately do not agree. For example, both marginals above contain the one-way counts of `HWUSUAL`, and two noisy copies of the same count differ. So OpenDP fits one probability model over all the columns whose marginals come as close as possible to the noisy ones, and every count you read comes from that model.

The method of the fit is **Private-PGM** (McKenna, Sheldon and Miklau, 2019), in the `mbi` package. Its model is a graphical model with one factor per measured marginal, fitted by mirror descent, a gradient method that keeps the model a valid distribution at every step. The fit sees only the noisy marginals, so it is post-processing and costs no budget (the full edition's chapter on query release).

Install the extra with `pip install 'opendp[mbi]'`, which brings `mbi` 1.1.0, and the documentation's code then runs as written. Every algorithm fits its model with `mbi.estimation.mirror_descent` at the default of 1,000 steps, and the check at the end of the chapter asks whether that is enough on this data.

## 16.2. The context under zCDP, with a delta

The context is the one of Assessing utility and choosing the noise, with a larger budget. The privacy unit is the book's 36 contributions per person (the data and the context). The budget is a privacy parameter $\rho$ of 0.2 under zCDP, which bounds the privacy loss on average, with a slack $\delta$ of $10^{-7}$ for the stable keys. The two queries of the chapter take $\rho = 0.1$ and $\rho = 0.05$, so part of the budget stays unspent.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: 36 contributions per person, rho = 0.2 of zCDP with delta = 1e-7,
     ↪  spent query by query."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at
↪  most 36 quarters
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # the delta pays for the stable
↪  keys of ILOSTAT
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.2, 1e-07))
```

The budget is under zCDP because a contingency table is many counts released together. Under zCDP the Gaussian noise on all of them composes by adding their values of $\rho$. That sum grows more slowly than the values of $\varepsilon$ of pure differential privacy would (Combinators, and the full edition's chapter on concentrated differential privacy in the full edition). A budget in $\varepsilon$ also works, with Laplace noise, but then the error bars below are only approximate.

The $\delta$ is there because one column, `ILOSTAT`, has neither keys nor cuts, so its keys come from the data by the stable-keys release of Grouping. A key is shown only when its noisy count clears a threshold, set so that one person alone reaches it with probability at most $\delta$. The slack pays for that threshold and for nothing else, and the library enforces this in both directions. It refuses a $\delta$ when every column has keys or cuts, and it refuses $\delta = 0$ when a column has neither. The script provokes both refusals on this data.

```
delta with keys for every column: delta (1e-07) must be zero because keys and cuts span
 ↪  all columns
no delta with ILOSTAT unkeyed: delta (0.0) must be nonzero because keys and cuts don't
 ↪  span all columns
```

The snippet shows the first refusal on a toy frame. It also shows that when every column has keys, nothing is spent on discovering keys, so the query reports no one-way scale or threshold.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
frame = pl.LazyFrame({"SEX": [1, 2, 1, 2]})
fixed = dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))])
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=frame, privacy_unit=unit,
 ↪  privacy_loss=dp.loss_of(rho=0.1), split_evenly_over=1)
query = pure.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
print(query.oneway_scale, query.oneway_threshold)  # None None
slack = dp.loss_of(rho=0.1, delta=1e-7)
approximate = dp.Context.compositor(data=frame, privacy_unit=unit, privacy_loss=slack,
 ↪  split_evenly_over=1)
try:
    approximate.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
except ValueError as err:
    print(err)  # delta (1e-07) must be zero because keys and cuts span all columns
```

## 16.3. Keys and cuts

The model needs a fixed, finite set of values for every column, because its cells are indexed by them. OpenDP takes that set from one of three places. For a categorical column you pass **keys**, the public list of its values, as `keys={"SEX": [1, 2]}` from the survey's codebook. For a numeric column you pass **cuts**, the public bin edges, as `cuts={"AGE": [20, 40, 60]}`, which make four left-open intervals, from `(-inf, 20.0]` to `(60.0, inf]`. For a column with neither, here `ILOSTAT`, the library spends part of the budget on **stable keys**, the values whose noisy counts clear the threshold.

Each of the three sources can miss values, and the library handles every miss the same way. A value outside the key list, such as a code 3 in `SEX`, becomes `null`, and so does a stable key with too small a count. So `null` stands for "something else", and a true null in the data joins it. To keep true nulls apart, recode them to a value of their own with `with_columns` before the query, as the documentation advises. A cut column has no null bin, so a null `AGE` or `HWUSUAL` lands in the last bin. The check at the end confirms that on the 255 rows without an age and the 20,819 without usual hours.

The codebook of `ILOSTAT` lists six codes, from 1 for working to 9 for a child under 15. Codes 4 and 5 do not occur in this sample, so the stable keys should find four. Then `null` is the cell of the two missing codes, with no rows behind it.

## 16.4. The Fixed algorithm and its workload

`dp.mbi.Fixed` releases a workload you name in advance, as a list of `Count` queries, one per marginal. It is the right tool when you already know which relations you need, while the algorithms of the next chapter choose the marginals themselves. The query names the columns, the keys and cuts, and the algorithm.

```
def workload_query(context: dp.Context) -> ContingencyTableQuery:
    """The documentation's query: keys for SEX, cuts for AGE and HWUSUAL, stable keys
     ↪  for ILOSTAT, two counts."""
    return (
        context.query(rho=TABLE_RHO, delta=DELTA)
        .select("SEX", "AGE", "HWUSUAL", "ILOSTAT")
        .contingency_table(
            keys={"SEX": [1, 2]},  # public: the codebook's two codes
            cuts={"AGE": [20, 40, 60], "HWUSUAL": [1, 20, 40]},  # left-open bins:
 ↪  (-inf, 20], (20, 40], ...
            algorithm=dp.mbi.Fixed(
                queries=[dp.mbi.Count(("AGE", "HWUSUAL")), dp.mbi.Count(("HWUSUAL",
 ↪  "ILOSTAT"))],
                oneway_split=ONEWAY_SPLIT,  # the share of rho for the one-way marginal
 ↪  of ILOSTAT
            ),
        )
    )
```

The budget of the query has two parts. A share `oneway_split` goes to the one-way marginals of the columns without keys, which give the stable keys of `ILOSTAT`. The rest goes to the workload, equally over its counts unless you give them weights. By default `Fixed` takes half the budget times the fraction of columns without keys, here a half of a quarter. The documentation sets this one eighth explicitly, and the script prints the split.

```
oneway split: 0.125 of rho=0.1 -> rho=0.0125 one-way, rho=0.04375 per count
oneway scale: 227.68  oneway threshold: 1364
```

Read the two attributes before you release. The scale is the standard deviation of the Gaussian noise on the one-way counts of `ILOSTAT`. The threshold is the count that a noisy key must exceed to be kept. Both follow from the privacy unit. One person's 36 rows can all carry the same status, so a person changes one count by up to 36, which is the sensitivity of the one-way marginal. The Gaussian scale that spends $\rho$ on a count of sensitivity $\Delta$ is $\Delta/\sqrt{2\rho}$ (Assessing utility and choosing the noise), as the script checks for both parts of the budget.

```
scale from the formula 36 / sqrt(2 x 0.0125): 227.68
scale of each count of the workload, 36 / sqrt(2 x 0.04375): 121.70
```

So each cell of the two measured marginals carries Gaussian noise with standard deviation 121.70. The threshold 1364 is set as in Grouping. At scale 227.68, a key held by one person alone, in all 36 of their rows, clears it with probability at most $\delta$. The sensitivity 36 is the price of the privacy unit, and a truncation to one row per person and year (Identifier truncation) would lower it.

## 16.5. The released table

```
table = query.release()
ilostat_std = table.std(("ILOSTAT",))
ilostat_accuracy = dp.gaussian_scale_to_accuracy(ilostat_std, alpha=ALPHA)
```

The release returns a `ContingencyTable`. It holds the key sets that the model was fitted on, the cuts, the thresholds that produced any stable keys, and the data type of the keys of each column.

```
keys: SEX=[1, 2, null]  AGE=[(-inf, 20.0], (20.0, 40.0], (40.0, 60.0], (60.0, inf]]
 ↪  HWUSUAL=[(-inf, 1.0], (1.0, 20.0], (20.0, 40.0], (40.0, inf]]  ILOSTAT=[1, 2, 3, 9,
 ↪  null]
cuts: AGE=[20.0, 40.0, 60.0]  HWUSUAL=[1.0, 20.0, 40.0]
thresholds: {'ILOSTAT': 1364}
schema: {'SEX': Int64, 'AGE': String, 'HWUSUAL': String, 'ILOSTAT': Int64}
```

Every key set that is not a cut set ends in `null`, the cell for values outside the keys. So `SEX` has one, although the data hold only 1 and 2. The stable keys of `ILOSTAT` are the four codes with rows, plus `null`. The schema shows that a cut column now holds strings, the bin labels, so a query on the table names bins and not numbers.

The method `project` sums the model over every column except the ones you name, and returns the counts as an array in the order of the keys. The method `project_melted` returns them as a data frame. Both are post-processing. You can project onto any set of columns, even ones that no marginal measured, which the next section shows is a mixed blessing.

The method `std` estimates the standard deviation of a projection, and `dp.gaussian_scale_to_accuracy` turns it into an error bar. Projecting a two-way marginal onto one of its columns sums four noisy cells, so the standard deviation doubles, from 121.70 to 243.40. So with probability 0.95, each count of `ILOSTAT` is within 477.06 of the truth. That estimate leaves out the one-way marginal of `ILOSTAT`, because it measured only the released keys, and OpenDP counts no cell-wise precision for such a measurement. The column `HWUSUAL` sits in both measured marginals, and the two estimates combine by their inverse variances to 181.42.

```
std ILOSTAT: 243.40  95% accuracy: 477.06
std AGE x HWUSUAL: 121.70  95% accuracy: 238.53
std HWUSUAL: 181.42  95% accuracy: 355.58
```

## 16.6. Adaptive estimation

`SEX` has keys, but no marginal of the workload contains it. The model knows the total number of rows and nothing about how they split by sex. So it spreads the total evenly over the three keys, and `std` refuses to put a bar on that.

```
std SEX: attrs (('SEX',)) are not covered by the query set
```

A second query repairs this without starting over. Passing the released table as `table=` tells the library to keep its keys, cuts, marginals and model, measure the new marginals, and fit the model again. Every column now has keys, including the ones that the first release found, so this query must carry $\delta = 0$ and spends nothing on key discovery.

```
def sex_query(context: dp.Context, table: dp.mbi.ContingencyTable) ->
 ↪  ContingencyTableQuery:
    """A second query that adds the marginal of SEX to the released table's model;
     ↪  every key is known, so delta = 0."""
    return (
        context.query(rho=SEX_RHO, delta=0.0)
        .select("SEX")
        .contingency_table(
            table=table,  # keep the keys, cuts, marginals and model of the first
 ↪  release
            algorithm=dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))]),
        )
    )
```

The new marginal is a one-way count at $\rho = 0.05$ with the same sensitivity of 36, so its scale is 113.84. The three cells of `SEX` then have an error bar of 223.13. The model now has four marginals, and the context has spent $\rho = 0.15$ of its 0.2. The budget of the second query was chosen after seeing the first table, which is the adaptive composition of Combinators. The new fit starts from the first model and runs its own 1,000 steps of mirror descent.

```
std SEX after the second query: 113.84  95% accuracy: 223.13
marginals in the model: [('ILOSTAT',), ('AGE', 'HWUSUAL'), ('HWUSUAL', 'ILOSTAT'),
 ↪  ('SEX',)]
privacy loss spent: (0.15, 1e-07)
```

## 16.7. Checking the release against the truth

The data are public, so the script computes the true values that the table estimates. Polars gives the true one-way counts of `ILOSTAT` and the true `AGE` by `HWUSUAL` table for the documentation's cuts. Every null age or null hours goes in the last bin, as the library places them.

```
true ILOSTAT counts: 1=1,517,011  2=155,661  3=1,450,283  9=688,977
true AGE x HWUSUAL (a null AGE or HWUSUAL counts in the last bin):
  AGE (-inf, 20.0]: (-inf, 1.0]=2,733  (1.0, 20.0]=13,651  (20.0, 40.0]=95,686  (40.0,
 ↪  inf]=1,051,617
  AGE (20.0, 40.0]: (-inf, 1.0]=1,441  (1.0, 20.0]=31,415  (20.0, 40.0]=370,616  (40.0,
 ↪  inf]=258,601
  AGE (40.0, 60.0]: (-inf, 1.0]=1,285  (1.0, 20.0]=46,743  (20.0, 40.0]=437,820  (40.0,
 ↪  inf]=309,224
  AGE (60.0, inf]: (-inf, 1.0]=1,502  (1.0, 20.0]=28,084  (20.0, 40.0]=122,405  (40.0,
 ↪  inf]=1,039,109
```

The releases of one run follow, to read against the error bars above. They show more than the noise.

```
released (one run):
  ILOSTAT 1: 1,515,589  (true 1,517,011, off by -1,422)
  ILOSTAT 2: 155,622  (true 155,661, off by -39)
  ILOSTAT 3: 1,450,332  (true 1,450,283, off by +49)
  ILOSTAT 9: 688,890  (true 688,977, off by -87)
  ILOSTAT null: 1,142  (true 0, off by +1,142)
  AGE x HWUSUAL largest |released - true|: 521  (accuracy 238.53)
  at AGE (60.0, inf] HWUSUAL (40.0, inf]: 1,038,588  (true 1,039,109)
  last bin of both, the nulls' cell: 1,038,588  (true 1,039,109, off by -521)
  SEX before the second query: 1,270,525 1,270,525 1,270,525  (uniform)
  SEX after: 1: 1,828,537  (true 1,828,992)  2: 1,982,506  (true 1,982,940)  null: 610
↪  (true 0)
  ILOSTAT after the second query:
    ILOSTAT 1: 1,516,288  (true 1,517,011, off by -723)
    ILOSTAT 2: 155,608  (true 155,661, off by -53)
    ILOSTAT 3: 1,450,301  (true 1,450,283, off by +18)
    ILOSTAT 9: 688,867  (true 688,977, off by -110)
    ILOSTAT null: 590  (true 0, off by +590)
```

Start with the cells that the noise alone explains. The largest of the sixteen two-way errors is 521 rows, past twice the bar of 238.53 per cell but inside three times it. It sits in the last bin of both columns, which holds the rows without usual hours or age, so the run confirms where the library puts a null. The `null` keys of `ILOSTAT` and `SEX` have no rows behind them, but their estimates are positive, because a fitted distribution cannot go negative. Keep that in mind for a cell whose truth may be zero.

The fit, not the noise, explains the other errors. The working code 1 of `ILOSTAT` came out 1,422 rows short, past twice its bar of 477.06. The empty `null` key holds 1,142 rows. The fit starts from the uniform model, so `null` begins with a fifth of all rows. A key with no rows is the slowest cell to fit, because the model can empty it only in the limit. So 1,000 steps leave about a thousand rows in it, and those rows are missing from the other keys.

The second query shows the same thing in `SEX`. Its fit starts from the first model, in which the `null` key of `SEX` holds a third of all rows. After 1,000 steps that key still holds 610 rows, more than twice its bar. So the two real keys are short by 455 and 434 rows. The same steps continued to fit `ILOSTAT`, whose working code is now 723 rows short.

To settle it, the script fits the four released marginals of the second table again from scratch, with 10,000 steps of `mbi.estimation.mirror_descent` instead of the default 1,000. That touches no data, so it is post-processing and a dev-time check, never part of a release.

```
  the same marginals refitted at 10,000 steps (dev time, post-processing):
    ILOSTAT 1: 1,516,829  (true 1,517,011, off by -182)
    ILOSTAT 2: 155,587  (true 155,661, off by -74)
    ILOSTAT 3: 1,450,277  (true 1,450,283, off by -6)
    ILOSTAT 9: 688,859  (true 688,977, off by -118)
    ILOSTAT null: 103  (true 0, off by +103)
    SEX: 1: 1,828,829  (true 1,828,992)  2: 1,982,798  (true 1,982,940)  null: 27
↪  (true 0)
```

```
AGE x HWUSUAL largest |refit - true|: 295  (accuracy 238.53)
```

With more steps, the `null` key of `ILOSTAT` keeps 103 rows and that of `SEX` 27, inside their bars. Code 1 is within 182 rows of the truth and each real key of `SEX` within 163. The largest two-way error is 295 rows, inside three times its bar. So a fit that stops early biases the empty cells upward and the largest ones downward, and nothing in the release warns you. A dev-time check against public or historical data is how you notice it.

The test behind the script holds each two-way cell and each count of the longer fit within three times its bar. For the default fit, it checks that each `null` holds less than a tenth of a percent of the rows. It also checks that each real key is within three times its bar plus the rows left in `null`.

## 16.8. Summary

Table 16.1.: The calls and concepts of a contingency-table release, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.loss_of(rho=0.2, delta=1e-7)` | A zCDP budget with a slack for stable keys | Gaussian noise composes tightly over many marginals; the $\delta$ is needed exactly when a column has neither keys nor cuts |
| `keys={"SEX": [1, 2]}` | The public values of a categorical column | Values outside the list become `null` |
| `cuts={"AGE": [20, 40, 60]}` | Left-open bins of a numeric column | Three cuts make four bins; a null lands in the last one |
| A column with neither | Stable keys, bought with `oneway_split` of the budget | Rare values fall below the threshold and join `null` |
| `dp.mbi.Fixed(queries=[dp.mbi.Counts(...), ...], oneway_split=1/8)` | A workload named in advance | The rest of the budget is shared equally over the counts, or by their weights |
| `query.oneway_scale`, `query.oneway_threshold` | The noise and the cut-off of the key discovery | Read them before releasing; both are `None` when every column is keyed |
| `table.keys`, `table.cuts`, `table.thresholds`, `table.schema` | What the model was fitted on | Cut columns come back as bin labels, not numbers |
| `table.project`, `table.project_melted` | Any count over the table's columns, from the model | Post-processing; a column no marginal covers comes back uniform |

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `table.std` with `dp.gaussian_scale_to_accuracy` | The error bar of a covered projection | Raises for an uncovered projection; a cell near zero is biased upward |
| `contingency_table(table=table, ...)` | Adds marginals to an existing model | Every key is then known, so $\delta = 0$ |
| `mbi.estimation.mirror_descent(domain, marginals, iters=...)` | Refits the released marginals with more steps | Post-processing; the default 1,000 steps leave rows in the empty `null` keys on this data, so check the fit on public data |

## 16.9. References

- OpenDP documentation: "Contingency Tables" in Getting Started, and the opening of "Synthetic Data", which lists the algorithms and the rules for null values.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM, the model that one consistent table is fitted from, and the mirror-descent fit.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP and the Gaussian mechanism under it.
- The full edition's chapters on query release in the full edition, sensitivity in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

# 17. Synthetic data with AIM and MST

Contingency tables released the marginals you asked for, which works when you know the questions in advance. The analyst of a synthetic dataset usually does not, and a workload that guesses wrong leaves the relations they care about to chance. So this chapter lets the library choose. The **AIM** algorithm picks its marginals one round at a time, each time the one that its current model gets most wrong. The **MST** algorithm picks a spanning tree of column pairs in one step. Both fit a model to the noisy marginals, and `synthesize()` turns the model into rows, which is post-processing and costs nothing.

The chapter follows the documentation's synthetic-data page and the docstrings of `dp.mbi.AIM`, `dp.mbi.MST`, `dp.mbi.Sequential` and `ContingencyTable.synthesize`. You will release three tables over four survey columns and read which marginals each algorithm chose. Then you will generate a synthetic frame of 3.8 million rows, state the guarantee, and compare the synthetic marginals with the real ones. That comparison shows a problem with nulls, which you will fix.

The script opendp_aim_and_mst.py runs everything on the Labour Force Survey (LFS) sample with OpenDP 0.16.0 and `mbi` 1.1.0, the Private-PGM version that OpenDP pins, with JAX on the CPU. The noise scales, the threshold, the budget arithmetic, the conversion to $\varepsilon$ and the facts about the data are the same on every run. OpenDP's noise cannot be seeded, so the chosen marginals, the counts, the rows and the distances to the real data change. The page shows them in blocks marked as one run. The whole script takes about 75 seconds.

```
uv run python code/part05-synthetic-data/opendp_aim_and_mst.py
```

## 17.1. When you do not know the questions

A fixed workload spends the budget on the marginals you name and on nothing else. For a pair of columns you did not name, the model answers as if the two columns were independent, as `SEX` came out uniform in the previous chapter. With four columns you could name every pair and every triple. But the budget would then spread over ten marginals, most of which the model fits well anyway. The question is how to spend the budget where the model is wrong.

AIM answers it adaptively. It first measures the one-way marginal of every column with half the budget, which also releases the keys of a column without keys. Then it fits a model and runs rounds. Each round scores every candidate marginal of up to three columns by how far the model's version is from the true one, minus what measuring it would cost in noise. Candidates that overlap the workload weigh more, so the three-way marginals, which are the workload itself, weigh most.

A noisy selection with a tenth of the round's budget picks one candidate, by the report-noisy-max form of the exponential mechanism (the full edition's chapter on selection). The other nine tenths measure the chosen marginal with Gaussian noise, and AIM fits the model again.

The budget per round adapts. It starts at the budget divided by the number of rounds. When a measurement moves the model by less than its noise, the round taught little, so the next round gets four times the share. The rounds stop when a privacy filter finds the budget spent. The `rounds` argument sets the number of planned rounds, by default sixteen per column, which is 64 here, and the script uses 10 so that a release takes seconds. The `max_size` argument caps the memory of the model in megabytes, which never binds with four small columns.

```
import opendp.prelude as dp

dp.enable_features("contrib")
aim = dp.mbi.AIM()
print(aim.queries, aim.measure_split, aim.max_size, aim.rounds)  # 3 0.9 80.0 None
print(dp.mbi.MST().num_selections, dp.mbi.MST().measure_split)  # None 0.9
print(dp.mbi.Count(("SEX", "ILOSTAT")))  # Count(by=('SEX', 'ILOSTAT'), weight=1.0)
```

The `queries` argument is either the largest number of columns of a candidate, 3 by default, or a workload of weighted `dp.mbi.Count` objects. The `measure_split` of 0.9 is the share of each round that goes to measuring. MST has the same split and one argument of its own, `num_selections` (Section 17.3).

## 17.2. The AIM release

The context is the documentation's, with three times the budget because this chapter releases three tables. The privacy unit is 36 contributions per person, as in the data and the context. The budget is a privacy parameter $\rho$ of 0.3 under zero-concentrated differential privacy (zCDP), with Gaussian noise. The parameter bounds the privacy loss on average (the full edition's chapter on concentrated differential privacy (DP)). The slack $\delta$ of $3 \times 10^{-7}$ pays for releasing the keys of `ILOSTAT`, which the query does not name.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: a privacy unit of 36 rows, a budget of rho = 0.3 with delta = 3e-7,
     ↪  one share per release."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪   remove one person's 36 rows
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # zCDP with Gaussian noise;
↪   delta pays for ILOSTAT's keys
    )
```

Every release goes through one function, because the three queries differ only in the algorithm. The query takes a third of the budget, $\rho = 0.1$ with $\delta = 10^{-7}$. It selects the documentation's four columns, gives the keys of `SEX` and the cuts of `AGE` and `HWUSUAL`, and leaves `ILOSTAT` to the threshold. The `fill` argument is for later.

```
def table_query(context: dp.Context, algorithm: Algorithm, fill: bool = False) ->
↪  ContingencyTableQuery:
    """The documentation's contingency-table query at rho = 0.1 and delta = 1e-7, with
    ↪  the algorithm plugged in."""
    query = context.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    if fill:
        query = query.with_columns(fill_nulls())  # a row-by-row transformation before
↪  the table
    return query.select(*COLUMNS).contingency_table(
        keys=KEYS,  # public categories of SEX; ILOSTAT's are released with the
↪  threshold
        cuts=CUTS,  # bins for the two numeric columns
        algorithm=algorithm,
    )
```

The first release uses AIM with ten rounds. Before the release, the query reports the scale and the threshold of its one-way step, which cost nothing to read.

```
aim_query = table_query(context, dp.mbi.AIM(rounds=ROUNDS))
oneway_scale, oneway_threshold = aim_query.oneway_scale, aim_query.oneway_threshold
table_aim, seconds_aim = timed_release(aim_query)
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.3, 3e-07))
query budget: rho=0.1 delta=1e-07
aim: oneway scale=227.6840 threshold=1364
aim: rho per one-way marginal=0.0125  scale 36/sqrt(2 rho)=227.6840
aim: rounds=10  rho per round=0.0050  measure=0.0045  select=0.0005  scale of a round's
↪  marginal=379.4733
aim keys ILOSTAT: [1, 2, 3, 9, None]  thresholds: {'ILOSTAT': 1364}
```

The scales follow from the budget split. Half of the query's $\rho$ goes to the one-way step, where AIM measures all four one-way marginals, so each gets 0.0125. One person changes the cells of a marginal by at most 36 rows in all, possibly all in one cell. So the sensitivity in the Euclidean norm is 36. Write $\Delta_2$ for that sensitivity and $\sigma$ for the standard deviation of the Gaussian noise. The mechanism is $\rho$-zCDP when

$$\sigma = \frac{\Delta_2}{\sqrt{2\rho}}, \tag{17.1}$$

and that formula gives the 227.68 the query reports.

The other half of the share is divided over the ten rounds. Nine tenths of the 0.005 of a round goes to measuring, and the formula gives 379.47 for the marginal of a round. The threshold of 1364 is the noisy count that a category of `ILOSTAT` must clear to be released. The keys are the four codes in the data plus the null that stands for anything else.

```
aim release (one run): 13.4 s
aim measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
```

```
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=189.74
aim measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL') std=379.47
aim measured: ('AGE', 'HWUSUAL') std=379.47
```

The `marginals` attribute of the table holds every measurement with its clique, the columns it spans, and its noise. In this run AIM measured all four three-way marginals before any pair, as the weights predict. One of them appears twice, the second time at half the scale, which is the adaptive rule at work. A round had moved the model by less than its noise, so the next round took four times the share, which was all that was left. So there were seven measurements and not ten. Another run chooses differently, so read this list after every release.

## 17.3. A spanning tree of pairs with MST

MST, the mechanism that won the 2018 NIST synthetic-data contest, is simpler than AIM. After the same one-way step it scores only pairs of columns, by how far the model's version is from the true one. Then it picks pairs one at a time by noisy maximum, each time among the pairs that join two columns not yet connected. With four columns that gives three pairs, which form a tree that spans every column, hence the name. The `num_selections` argument sets how many pairs to pick, by default one fewer than the number of columns. Then MST measures all the selected pairs at once with nine tenths of the remaining budget, and fits the model.

```
mst_query = table_query(context, dp.mbi.MST())
table_mst, seconds_mst = timed_release(mst_query)
```

```
mst: oneway scale=227.6840 threshold=1364
mst: pairs=3  rho per pair=0.0150  scale of a pair=207.8461
mst release (one run): 3.3 s
mst measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst measured: ('AGE', 'ILOSTAT') std=207.85
mst measured: ('SEX', 'ILOSTAT') std=207.85
mst pairs form a spanning tree: True
```

The one-way step is identical, so its scale and threshold are the same. The three pairs share nine tenths of the remaining budget, 0.015 each, and the formula gives 207.85. So each pair is measured more precisely than a round of AIM, because MST commits to three measurements where AIM keeps budget back for rounds it may need. In this run the tree is a star on `ILOSTAT`, the column most related to the other three. The script checks that the pairs connect all four columns without a cycle, as the selection guarantees.

## 17.4. A first look at the one-way marginals

The data are public, so the script can compare each one-way marginal of the synthetic rows, with their numeric columns put back into bins, with the real one. It reports the total variation distance,

which is half the sum of the absolute differences of the shares of the two marginals. It also reports the largest error of any cell in rows, and the standard deviation of that marginal from `std`.

The last column is a bound. Write $k$ for the number of cells of the marginal and $n$ for the number of rows. If every cell were off by three standard deviations at once, the distance would be

$$\frac{3k\sigma}{2n}. \tag{17.2}$$

A well-fitted marginal stays far under that, because the cell errors have different signs and few reach three standard deviations.

```
one-way SEX (aim): tv=0.00007 largest cell error=2,813 rows  std=204.6 bound=0.00024
one-way AGE (aim): tv=0.00015 largest cell error=1,547 rows  std=201.1 bound=0.00032
one-way HWUSUAL (aim): tv=0.00200 largest cell error=11,310 rows  std=192.3
 ↪  bound=0.00030
one-way ILOSTAT (aim): tv=0.00039 largest cell error=1,670 rows  std=792.7
 ↪  bound=0.00156
one-way SEX (mst): tv=0.00004 largest cell error=2,779 rows  std=204.5 bound=0.00024
one-way AGE (mst): tv=0.00017 largest cell error=1,981 rows  std=204.5 bound=0.00032
one-way HWUSUAL (mst): tv=0.00257 largest cell error=13,620 rows  std=204.5
 ↪  bound=0.00032
one-way ILOSTAT (mst): tv=0.00040 largest cell error=1,963 rows  std=227.7
 ↪  bound=0.00045
```

`SEX` and `AGE` are right as shares, with distances below their bounds. But their largest cell errors of two or three thousand rows are more than the noise explains. `HWUSUAL` is wrong even as shares. Its distance is many times its bound in both releases, and one cell is off by more than eleven thousand rows, over fifty standard deviations. Both algorithms measured that marginal at the same scale as the others, so noise cannot explain it, and something systematic is wrong with the column.

## 17.5. Nulls and the key set

The model behind every table needs a fixed set of keys for every column, which the documentation's section on nulls explains. The keys come from you, through `keys` and `cuts`, or from the stable-key release. Either way, every value outside the set is replaced by null before counting. A cut column has no null key, so the four bins of `HWUSUAL` are its whole key set, and the column has 20,819 real nulls, hours that were not recorded.

```
rows: 3,811,932  AGE nulls: 255  HWUSUAL nulls: 20,819
HWUSUAL = 99 (not applicable): 2,294,921
```

In version 0.16.0 those nulls take two different paths. The one-way step counts a cut column by joining its counts onto the bin labels, so it does not count a null row at all. The multi-way step gives a row whose bin is null the last index, so it counts that row in the bin above 40 hours. So the model is fitted to measurements that disagree about the number of rows, and it compromises. The script shows the fitted one-way marginal of `HWUSUAL` against the real counts with the nulls in the last bin.

```
aim model total (one run): 3,806,638.8  real rows 3,811,932
HWUSUAL bins (aim, one run): fitted count against the real count with nulls in the last
 ↪  bin
  (-inf, 1.0]: fitted 8,953  real 6,961  difference +1,992
  (1.0, 20.0]: fitted 121,660  real 119,893  difference +1,767
  (20.0, 40.0]: fitted 1,028,785  real 1,026,527  difference +2,258
  (40.0, inf]: fitted 2,647,240  real 2,658,551  difference -11,311
```

The last bin is eleven thousand rows short and every other bin is two thousand rows over. The total of the model, which it estimates from the one-way marginals, is five thousand rows below the real count. Those are the missing nulls, spread by the fit.

The documentation's remedy is to turn true nulls into a key before the table, with `select` or `with_columns`, which is what the `fill` argument of the query does. It is a row-by-row transformation inside the query, so it changes nothing about the privacy unit. The script sends the nulls of `HWUSUAL` to 99, the survey's code for not applicable. It sends the 255 nulls of `AGE` to zero, the youngest bin. The code a null gets is a modelling decision, but every row must land in a key, the same way in both steps.

## 17.6. Guaranteeing marginals with Sequential

Adaptive selection has a cost, because AIM may not measure a marginal that the release must contain. `Sequential` runs a list of algorithms one after another on the same table, each from the model of the previous one. It splits the budget evenly between them unless `weights` says otherwise. So with a `Fixed` workload first and AIM second, the named marginals are always measured, and AIM spends the rest where the model is still wrong. The third release does this with the pairs of `ILOSTAT` with `SEX` and with `AGE`, and it fills the nulls.

```
sequential = dp.mbi.Sequential(
    algorithms=[
        dp.mbi.Fixed(queries=[dp.mbi.Count(by) for by in GUARANTEED]),  # these two
 ↪  marginals, whatever happens
        dp.mbi.AIM(rounds=ROUNDS),  # then AIM chooses the rest
    ],
)
sequential_query = table_query(context, sequential, fill=True)
table_seq, seconds_seq = timed_release(sequential_query)
```

```
sequential: rho per guaranteed marginal=0.0125  scale=227.6840
sequential: rho per AIM round=0.0025  scale of a round's marginal=536.6563
sequential release (one run): 12.1 s
sequential model total (one run): 3,811,712.7  real rows 3,811,932
sequential measured: ('SEX', 'ILOSTAT') std=227.68
sequential measured: ('AGE', 'ILOSTAT') std=227.68
sequential measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=268.33
sequential measured: ('SEX', 'AGE', 'ILOSTAT') std=536.66
```

```
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'HWUSUAL') std=536.66
sequential guaranteed marginals present: True
```

The one-way step again takes half, and the rest is split evenly between the two algorithms. `Fixed` divides its quarter between its two marginals, at the familiar 227.68. AIM runs its ten rounds on the other quarter, so each of its marginals is noisier than in the first release, 536.66 against 379.47. That is the price of the guarantee, and `weights` can shift it either way. The total of the model is now 219 rows from the truth, within one standard deviation of a one-way measurement, so the nulls were the problem.

```
one-way SEX (sequential): tv=0.00004 largest cell error=240 rows  std=198.2
 ↪  bound=0.00023
one-way AGE (sequential): tv=0.00006 largest cell error=233 rows  std=200.7
 ↪  bound=0.00032
one-way HWUSUAL (sequential): tv=0.00009 largest cell error=273 rows  std=206.2
 ↪  bound=0.00032
one-way ILOSTAT (sequential): tv=0.00008 largest cell error=360 rows  std=271.5
 ↪  bound=0.00053
```

Every distance is under its bound by a factor of three or more, and no cell is more than 360 rows off, under two standard deviations. The largest error of `HWUSUAL` fell from eleven thousand rows to 273.

## 17.7. Generating the rows

A `ContingencyTable` is a model, and `synthesize()` turns it into a frame. The model is the Private-PGM fit of the previous chapter, a graphical model over the four columns whose marginals match the noisy measurements as closely as possible at once. Drawing rows from it looks only at released values, so it is post-processing and spends nothing, however often you call it.

```
synthesize (aim, one run): shape (3806638, 4)  model total 3,806,638.8  11.9 s
┌─────┬──────────┬──────────┬──────────┐
│ SEX ┆ AGE      ┆ HWUSUAL  ┆ ILOSTAT  │
│ --- ┆ ---      ┆ ---      ┆ ---      │
│ i64 ┆ f64      ┆ f64      ┆ i64      │
╞═════╪══════════╪══════════╪══════════╡
│ 1   ┆ 19.813324 ┆ 0.482324 ┆ 1        │
│ 1   ┆ 19.500078 ┆ 0.94265  ┆ 1        │
│ 1   ┆ 19.338927 ┆ 0.741979 ┆ 9        │
│ 1   ┆ 19.031755 ┆ 0.118982 ┆ 9        │
│ 1   ┆ 19.232563 ┆ 0.676039 ┆ 1        │
└─────┴──────────┴──────────┴──────────┘
synthetic AGE from 19.00 to 61.00  HWUSUAL from 0.00 to 41.00
synthetic dtypes: {'SEX': Int64, 'AGE': Float64, 'HWUSUAL': Float64, 'ILOSTAT': Int64}
synthetic SEX values: [1, 2]  ILOSTAT values: [1, 2, 3, 9]
```

Without a `rows` argument the frame has as many rows as the total of the model, a noisy estimate of the row count. The four columns keep their original types, and `SEX` and `ILOSTAT` take values from their keys. The default `method="round"` scales the expected counts of the model to the number of rows and rounds them at random. So the synthetic marginals match those of the model to within a row per cell. The alternative, `method="sample"`, draws each row independently, which adds sampling noise of about the square root of each count, and suits rows that should behave like a sample.

```
sample = table_aim.synthesize(rows=SAMPLE_ROWS, method="sample")  # draws rows
 ↪  independently from the model
```

```
synthesize rows=100,000 method=sample: shape (100000, 4)
```

Look again at the numeric columns, because the first rows all show an `AGE` just under 20. The model knows only which bin a row is in, so for a cut column `synthesize()` draws a value uniformly inside the bin, or within one unit of the edge for the two open-ended bins. Every synthetic age therefore lies between 19 and 61. The rows coded 99 all become hours just above 40.

So a mean age from this frame reflects the bins and the sampling rule, not the data. If the numbers matter, give finer cuts, at the cost of more cells per marginal and more noise per cell (Evaluating and tuning synthetic data).

The script seeds NumPy's global generator, which the rounding and the draws inside a bin use, so the frame is a function of the released table.

## 17.8. The privacy statement

The context has spent its three shares, and the script writes the statement of the whole release from its variables.

```
budget spent: (0.3, 3e-07)
rho=0.3 at delta=1e-07: epsilon=4.2709
whole context converted: epsilon=4.2709 delta=4e-07
```

A budget in $\rho$ is stated to readers in $\varepsilon$, with the conversion of Assessing utility and choosing the noise. The combinators `make_zCDP_to_approxDP` and `make_fix_delta` convert the budget of 0.3 to $\varepsilon = 4.27$, at a further $\delta$ of $10^{-7}$. The same two combinators on the `accountant` of the context give the same $\varepsilon$ with the total $\delta$, three parts for the keys and one for the conversion. So three adaptive releases cost $\varepsilon = 4.27$, which is a loose guarantee.

```
print(
    f"This release of three contingency tables and the synthetic rows drawn from them
     ↪  satisfies "
    f"(rho = {RHO:g}, delta = {DELTA:g})-zero-concentrated differential privacy, which
     ↪  is "
    f"(epsilon = {epsilon:.2f}, delta = {total_delta:g})-differential privacy, "
```

```
    f"for adding or removing all of one person's survey responses, at most
     ↪ {CONTRIBUTIONS} of them, "
    f"under the central model, using OpenDP {dp.__version__} with Gaussian noise. "
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; the keys
     ↪ of ILOSTAT were released "
    f"with a threshold of {oneway_threshold}. Generating the rows is post-processing
     ↪ and spent nothing."
)
```

The statement covers the synthetic rows, because post-processing adds nothing to the loss. It names what was public, since a cut chosen by looking at the data would leak it. The unit of 36 contributions is the expensive part, because every scale above carries the factor 36. But Identifier truncation and bounds showed that hardly anyone answers more than a few times. So the script builds the same query under the identifier unit with a truncation to four rows per person, to read the scale, and does not release it.

```
context_id = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),  # neighbours add or
 ↪ remove one person
    privacy_loss=dp.loss_of(rho=RHO_QUERY, delta=DELTA_QUERY),
)
query_id = (
    context_id.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    .truncate_per_group(TRUNCATE)  # at most 4 rows per person, so a person moves a
 ↪ count by at most 4
    .select(*COLUMNS)
    .contingency_table(keys=KEYS, cuts=CUTS, algorithm=dp.mbi.AIM(rounds=ROUNDS))
)
```

```
identifier unit, 4 rows per person: oneway scale=25.2982 threshold=142
identifier unit: scale ratio 9.00 = 36/4
```

The one-way scale falls nine-fold, to 25.30, because a person now moves a count by at most 4 rows. The threshold falls from 1364 to 142, and every other scale of the release carries the same factor. The cost is the truncation, which drops every row beyond the fourth of anyone who answered more often.

## 17.9. The measured marginals against the real ones

The last check compares every measured multi-way marginal of the three releases with the real one, as for the one-way marginals. Each release spent its budget on its measured marginals, so the synthetic data should be closest to the truth there. The lines are from the same run as the lists above.

```
measured (AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=2,992 rows  std=224.1
 ↪  bound=0.00141
measured (SEX, HWUSUAL) (aim): tv=0.00200 largest cell error=5,730 rows  std=242.4
 ↪  bound=0.00114
measured (AGE, HWUSUAL, ILOSTAT) (aim): tv=0.00219 largest cell error=1,382 rows
 ↪  std=379.5 bound=0.01195
measured (SEX, AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=1,556 rows  std=169.7
 ↪  bound=0.00321
measured (SEX, AGE, ILOSTAT) (aim): tv=0.00093 largest cell error=699 rows  std=379.5
 ↪  bound=0.00896
measured (SEX, HWUSUAL, ILOSTAT) (aim): tv=0.00200 largest cell error=1,940 rows
 ↪  std=379.5 bound=0.00896
measured (AGE, ILOSTAT) (mst): tv=0.00116 largest cell error=2,366 rows  std=207.8
 ↪  bound=0.00164
measured (HWUSUAL, ILOSTAT) (mst): tv=0.00258 largest cell error=3,700 rows  std=207.8
 ↪  bound=0.00164
measured (SEX, ILOSTAT) (mst): tv=0.00041 largest cell error=1,085 rows  std=207.8
 ↪  bound=0.00123
measured (AGE, ILOSTAT) (sequential): tv=0.00027 largest cell error=387 rows  std=221.1
 ↪  bound=0.00174
measured (HWUSUAL, ILOSTAT) (sequential): tv=0.00068 largest cell error=1,035 rows
 ↪  std=351.3 bound=0.00276
measured (SEX, HWUSUAL) (sequential): tv=0.00017 largest cell error=338 rows  std=342.9
 ↪  bound=0.00162
measured (SEX, ILOSTAT) (sequential): tv=0.00019 largest cell error=335 rows  std=218.1
 ↪  bound=0.00129
measured (SEX, AGE, HWUSUAL) (sequential): tv=0.00094 largest cell error=712 rows
 ↪  std=240.0 bound=0.00453
measured (SEX, AGE, ILOSTAT) (sequential): tv=0.00099 largest cell error=751 rows
 ↪  std=536.7 bound=0.01267
measured (SEX, HWUSUAL, ILOSTAT) (sequential): tv=0.00073 largest cell error=567 rows
 ↪  std=536.7 bound=0.01267
```

Two things stand out. In the two releases that left the nulls alone, every marginal with `HWUSUAL` breaks its bound. The pairs without it are also further from the truth than in the third release, because the wrong total leaks into every marginal. In the release that filled the nulls, every measured marginal is under its bound by a factor of four or more. The bound counts three standard deviations in every cell at once, which never happens, so a three-way marginal with a distance of one thousandth is a good fit.

This check does not say how the synthetic data answers a question that nobody measured, or how a model trained on it compares with one trained on the real rows. Evaluating and tuning synthetic data asks those questions, and shows what changes with the budget, the cuts or the number of rounds.

## 17.10. Summary

Table 17.1.: The calls and concepts of synthetic data with AIM and MST, what each does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| `dp.mbi.AIM(rounds=10)` | One-way marginals, then rounds that measure the marginal the model fits worst | Fewer rounds for a quick release; read `table.marginals` to see what it chose |
| `queries`, `measure_split`, `max_size` | The candidates or a weighted workload; the share of a round spent measuring; the memory cap | The defaults serve a handful of columns |
| `dp.mbi.MST()` | One-way marginals, then pairs that form a spanning tree, measured at once | Faster than AIM when pairs are what matter; it never measures a triple |
| `dp.mbi.Sequential([Fixed, AIM])` | Run algorithms in turn on one table, splitting the budget evenly or by `weights` | Put the marginals you must have in `Fixed`; AIM spends the rest |
| `query.oneway_scale`, `query.oneway_threshold` | The one-way step's noise scale and key threshold, before anything is spent | Under the identifier unit, 4 rows per person cost a ninth of 36 |
| `table.synthesize(rows, method)` | Rows drawn from the fitted model; `round` matches its marginals, `sample` draws independently | Post-processing, so free; the row count defaults to the model's noisy total |
| Cut columns in the synthetic frame | Values drawn uniformly inside the bin, within one unit of the edge for the open bins | Right as bins, wrong as numbers; give finer cuts if the numbers matter |
| Nulls in a cut column | Dropped by the one-way step and counted in the last bin by the multi-way step in 0.16.0 | Fill them with a public code in `with_columns` before the table |
| Total variation distance against `std` | How far a synthetic marginal is from the real one, against three standard deviations per cell | A measured marginal over the bound points to the data, not the noise |

## 17.11. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi` (`AIM`, `MST`, `Sequential`, `Fixed`, `ContingencyTable`).
- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the selection score, the budget schedule and the experiments behind the defaults.

- McKenna, Miklau and Sheldon, "Winning the NIST Contest: A scalable and general approach to differentially private synthetic data" (2021): the MST mechanism and the Private-PGM engine that fits the model.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM itself, the inference step that both algorithms share.
- The full edition's chapters on selection in the full edition, query release in the full edition and concentrated DP in the full edition derive the results this chapter states.

# 18. Evaluating and tuning synthetic data

Synthetic data with AIM and MST ended with a synthetic table and a privacy statement. The statement says how little the table reveals about any one person, but nothing about whether the table is any good. The one idea of this chapter is that "good" is three separate questions, and only the release itself can answer the first. You will compare an AIM release with the real data on the marginals it measured, on a table it never measured, and through a classifier trained on its rows. Then you will repeat the comparison at three budgets and for MST, and write down what to publish with the rows.

The documentation stops once the rows are generated, so this chapter goes further, with methods from the AIM paper and a benchmark of synthetic-data generators. The comparison is possible only because the Labour Force Survey (LFS) sample is public. On private data, run the same comparison on a public or historical stand-in before the release, never on the data you protect.

The script opendp_evaluating_synthetic_data.py runs every step on OpenDP 0.16.0 with `mbi` 1.1.0, and imports the functions it shares with the script of the previous chapter. The budgets, noise scales, thresholds, conversions to the privacy parameter $\varepsilon$ and the classifier trained on real rows are the same on every run. OpenDP's noise cannot be seeded, so the releases, the fitted models and every distance to the truth change. The page shows them in blocks and tables marked as one run. The whole script, with four releases, took 55 seconds.

```
uv run python code/part05-synthetic-data/opendp_evaluating_synthetic_data.py
```

## 18.1. What good means

A synthetic table can be judged by three questions, the checks of the benchmark by Tao and coauthors. Do the marginals that the algorithm measured match the real ones? Do queries it never measured come out right, such as a count over all four columns or a share within a small group? And does a model trained on the synthetic rows work on real rows? Each check catches failures that the other two miss.

Only the first question has an error bar from the release. Every measured marginal carried Gaussian noise of a known scale, so the `std` method of the table says how far its fitted counts should be from the truth (Contingency tables). A table that no measurement covers has no such bar, and `std` refuses it, and a downstream model has none either. So you can answer the second and third questions only against real data. On a real release, that means public or historical data of the same kind (the full edition's chapter on public data).

The release under evaluation is the AIM release of the previous chapter with its nulls filled. It has the same privacy unit of 36 contributions (the data and the context), the same four columns with their keys and cuts, and ten rounds. The budget is a privacy parameter $\rho$ of 0.1 under

zero-concentrated differential privacy (zCDP), which bounds the privacy loss on average, with Gaussian noise. A slack $\delta$ of $10^{-7}$ pays for releasing the keys of `ILOSTAT`. Two functions build the context and the query from a budget, so that later sections can change it.

```
def make_context(frame: pl.LazyFrame, rho: float) -> dp.Context:
    """A context for one release at budget rho, with the previous chapter's unit and
     ↪  slack."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
 ↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(rho=rho, delta=DELTA),  # zCDP with Gaussian noise;
 ↪  delta pays for ILOSTAT's keys
    )


def table_query(context: dp.Context, rho: float, algorithm: Algorithm) ->
 ↪  ContingencyTableQuery:
    """The previous chapter's query with its nulls filled, at the budget rho, with the
     ↪  algorithm plugged in."""
    return (
        context.query(rho=rho, delta=DELTA)
        .with_columns(synth.fill_nulls())  # every null lands in a public key before
 ↪  anything is counted
        .select(*COLUMNS)
        .contingency_table(keys=synth.KEYS, cuts=synth.CUTS, algorithm=algorithm)
    )
```

The release is one line. The next line puts the real rows through the same steps as the query, filling the nulls, cutting the numbers into bins and mapping codes outside the keys to null. So every real marginal has exactly the cells of the synthetic one.

```
aim_query = table_query(make_context(frame, RHO), RHO, dp.mbi.AIM(rounds=ROUNDS))
table_aim, seconds_aim = synth.timed_release(aim_query)
real = synth.binned(frame, table_aim.keys, fill=True).collect()  # the real rows in the
 ↪  table's keys and bins
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
aim rho=0.1: oneway scale=227.6840 threshold=1364  scale of a round's marginal=379.4733
 ↪  epsilon at delta=1e-07: 2.3484
rows: 3,811,932  keys ILOSTAT: [1, 2, 3, 9, None]
aim rho=0.1 release (one run): 13.7 s
aim rho=0.1 measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'ILOSTAT') std=189.74
```

The query is that of the previous chapter, so the scales are too, 227.68 for the one-way marginals and 379.47 for the marginal of each round. At a further $\delta$ of $10^{-7}$, the budget converts to $\varepsilon = 2.35$. In this run AIM measured all four three-way marginals and then a pair at half the scale.

## 18.2. The workload error

The first question asks how far each measured marginal of the model is from the real one. The AIM paper calls the average of that distance over a workload the **workload error**, which AIM itself tries to make small. The distance is the total variation distance of the previous chapter, half the sum over cells of the absolute differences between the shares. It is also the largest difference between the shares that the two tables give to any set of cells. So the worst distance below, 0.00102, means that no question about those columns gets a share off by more than that.

The function below computes it for a list of cliques, the sets of columns that the marginals span. It reads the counts of the model with `project_melted` instead of drawing rows, because drawing rows adds noise of its own (Section 18.6). Next to each distance it puts the bound of the previous chapter, the distance if every cell of the marginal were off by three standard deviations at once.

```
def marginal_errors(
    table: ContingencyTable, real: pl.DataFrame, cliques: Sequence[tuple[str, ...]]
) -> list[MarginalError]:
    """Each clique's distance between the fitted model's marginal and the real one,
     ↪  with the table's own standard
    deviation for it and the bound that three standard deviations in every cell would
 ↪  give."""
    errors: list[MarginalError] = []
    for clique in cliques:
        fitted = table.project_melted(clique)  # the model's counts: no rows are drawn
        tv, worst = distance(synth.marginal(real, clique), fitted, clique)
        std = table.std(clique)
        bound = synth.tv_bound(synth.cells(table, clique), std, real.height)
        errors.append(MarginalError(clique, tv, worst, std, bound))
    return errors
```

Table 18.1.: The workload error of the AIM release at $\rho = 0.1$ in one run: the total variation distance between each one-way and each measured marginal of the model and the real one, the largest cell error, the table's own standard deviation, and the bound from it.

| Marginal | Distance | Largest cell error (rows) | `std` | Bound |
|---|---|---|---|---|
| `SEX` | 0.00004 | 158 | 196.2 | 0.00023 |
| `AGE` | 0.00005 | 146 | 219.9 | 0.00035 |
| `HWUSUAL` | 0.00006 | 246 | 219.9 | 0.00035 |
| `ILOSTAT` | 0.00013 | 307 | 201.0 | 0.00040 |
| (`SEX`, `ILOSTAT`) | 0.00022 | 425 | 178.9 | 0.00106 |
| (`AGE`, `HWUSUAL`, `ILOSTAT`) | 0.00102 | 509 | 379.5 | 0.01195 |
| (`SEX`, `AGE`, `HWUSUAL`) | 0.00081 | 503 | 379.5 | 0.00717 |
| (`SEX`, `AGE`, `ILOSTAT`) | 0.00072 | 568 | 379.5 | 0.00896 |
| (`SEX`, `HWUSUAL`, `ILOSTAT`) | 0.00065 | 470 | 379.5 | 0.00896 |

Every distance is under its bound by a factor of three or more, and no cell is off by more than a few hundred rows out of 3.8 million. The one-way standard deviations are not the one-way scale

of 227.68, because the model combines each one-way measurement with every measured marginal that contains the column. That is why `ILOSTAT`, which sits in four marginals, gets 201.0.

Read the table as a check of the pipeline, not only of the noise. A distance over its bound means something systematic, such as the nulls of the previous chapter. On private data you cannot fill in this table, because it needs the real marginals. You can fill in the `std` column, which comes from the release, and that is the error bar you publish.

## 18.3. A query nobody measured

The second question asks about a table that no measurement covers. The candidates of AIM have at most three columns, so it never measures the four-way table of `SEX`, `AGE`, `HWUSUAL` and `ILOSTAT`, whose 240 cells hold every combination. The model still answers, with the distribution that agrees with the measured marginals and assumes nothing else (the full edition's chapter on query release). But the table can no longer put a bar on the answer.

```
try:
    four_std = f"{table_aim.std(COLUMNS):.1f}"
except ValueError as err:
    four_std = str(err)  # no measured marginal covers all four columns, so the table
 ↪  has no error bar for it
```

```
four-way cells: 240  measured by AIM: False
std of the four-way table: attrs (('SEX', 'AGE', 'HWUSUAL', 'ILOSTAT')) are not covered
 ↪  by the query set
four-way (aim rho=0.1, one run): tv=0.00141 largest cell error=450 rows  best measured
 ↪  (SEX, ILOSTAT) tv=0.00022  worst measured tv=0.00102
cells of the share with at least 1,000 real rows: 26
share working by SEX, AGE, HWUSUAL (aim rho=0.1, one run): largest error=2.05 points at
 ↪  {'SEX': 1, 'AGE': '(-inf, 20.0]', 'HWUSUAL': '(1.0, 20.0]'}  real=100.00%
 ↪  model=97.95%
```

In this run the four-way table is further from the truth than every measured marginal. Its error comes from the noise of the triples that make it up, and from any dependence of all four columns that no triple shows. Here the four measured triples overlap so much that little is left to guess, so the distance stays near the worst measured one. A release with fewer marginals leaves more to the assumptions of the model, as Section 18.5 shows for MST.

A conditional share is a harder test, because it divides one small count by another. From the four-way table, the script reads the share of people at work in the 26 combinations of `SEX`, `AGE` band and `HWUSUAL` band with a thousand real rows or more. The largest error in this run was two points, for men under 20 who usually work 1 to 20 hours. Everyone in that cell works, since usual hours imply a job, but the model gave it a share of 97.95%. Noise is added per count, not per person, so a small group gets the largest relative error. If an analyst will ask about small groups, measure them, for example with `Fixed` inside `Sequential`.

## 18.4. Train on synthetic, test on real

The third question is whether a model trained on the synthetic rows works on real rows, which the benchmark calls "train on synthetic, test on real". The task is to predict whether a person works, `ILOSTAT` equal to 1, from `SEX`, the `AGE` band and the `HWUSUAL` band. The model is a logistic regression from scikit-learn on eleven indicator columns, one per key of each feature.

The script draws 200,000 synthetic rows with `method="sample"`, independently from the model, as a sample from the population would be. It also takes 400,000 real rows in one seeded Polars shuffle. The first 200,000 train the yardstick, a model on real rows, and the rest score both models.

```
rows = synth.synthesize(table, seed, rows=ROWS, method="sample")  # independent draws
 ↪  from the model
rows = synth.binned(rows.lazy(), table.keys, fill=False).collect()  # the numbers back
 ↪  in their bins
model = classifier(rows, table.keys, seed)  # train on synthetic
```

```
def classifier(train: pl.DataFrame, keys: dict[str, Any], seed: int) ->
 ↪  LogisticRegression:
    """A logistic regression that predicts ILOSTAT == 1 from SEX, the AGE band and the
     ↪  HWUSUAL band, fitted on
    `train`, which is either synthetic rows or real rows."""
    return LogisticRegression(max_iter=1000, random_state=seed).fit(design(train,
     ↪  keys), labels(train))


def accuracy(model: LogisticRegression, test: pl.DataFrame, keys: dict[str, Any]) ->
 ↪  float:
    """The share of the held-out real rows the model labels correctly."""
    return float((model.predict(design(test, keys)) == labels(test)).mean())
```

```
held-out real rows: 200,000  real training rows: 200,000  synthetic training rows:
 ↪  200,000
features: 11 indicators
share working in the held-out rows: 0.3976
train on real, test on real: accuracy=0.9313
standard error of an accuracy on 200,000 rows: 0.0006
train on synthetic (aim rho=0.1, one run), test on real: accuracy=0.9313  gap=+0.0000
 ↪  mean probability difference=0.0015
```

The model trained on real rows labels 93.13% of the held-out rows correctly, against 39.76% working. The model trained on synthetic rows scored the same in this run, so the gap is zero, well inside the standard error of 0.0006. The test allows a gap of one percentage point, the size of gap that a model would show if it got a few small cells wrong.

But accuracy is a coarse check, because two models that put the probability of every cell on the same side of one half make the same predictions. So the script also reports the mean difference between the predicted probabilities of the two models, 0.0015 in this run. For a single model, fitting it privately and directly (Statistical modelling) is often better than training on synthetic rows.

## 18.5. What the budget and the algorithm buy

Tuning means running the evaluation under different settings and choosing between them. The script repeats it with AIM at a fifth of the budget, $\rho = 0.02$, and at five times it, $\rho = 0.5$, then runs MST at the original budget. Each release gets a context of its own, because you would publish only one of them. Released from the same private data, the four would compose to $\rho = 0.72$, so tuning on private data spends budget too. Tune on a public stand-in, and release once.

```
mst_query = table_query(make_context(frame, RHO), RHO, dp.mbi.MST())
table_mst, seconds_mst = synth.timed_release(mst_query)
mst_eval = evaluate("mst rho=0.1", table_mst, seconds_mst, real, test_real, baseline,
 ↪  seed)
```

```
aim rho=0.02: oneway scale=509.1169 threshold=3004  scale of a round's
 ↪  marginal=848.5281  epsilon at delta=1e-07: 0.9951
aim rho=0.5: oneway scale=101.8234 threshold=630  scale of a round's marginal=169.7056
 ↪  epsilon at delta=1e-07: 5.6709
mst rho=0.1: oneway scale=227.6840 threshold=1364  scale of a pair=207.8461  epsilon at
 ↪  delta=1e-07: 2.3484
mst rho=0.1 measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('AGE', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('SEX', 'ILOSTAT') std=207.85
share working by SEX, AGE, HWUSUAL (mst rho=0.1, one run): largest error=24.03 points
 ↪  at {'SEX': 2, 'AGE': '(-inf, 20.0]', 'HWUSUAL': '(-inf, 1.0]'}  real=100.00%
 ↪  model=75.97%
budget the four contexts add up to if all were released from the same data: rho=0.72
```

The scales follow the formula of the previous chapter, the sensitivity of 36 over the square root of twice the budget. So twenty-five times the budget divides every scale by five, from 509.12 to 101.82 for the one-way marginals. In $\varepsilon$, the smallest budget is 1.00 and the largest 5.67 (the full edition's chapter on choosing epsilon discusses which a release can defend).

Table 18.2.: The same evaluation at three budgets with AIM and at one with MST, in one run: $\varepsilon$ at $\delta = 10^{-7}$, the seconds the release took, the number of marginals measured, the total variation distances of the one-way marginals (mean), of the worst measured marginal and of the four-way table, the largest error of the share working, and the mean difference in predicted probability from the model trained on real rows.

| Release | $\varepsilon$ | Seconds | Measured | One-way mean | Worst measured | Four-way | Share error (points) | Probability difference |
|---|---|---|---|---|---|---|---|---|
| AIM, $\rho = 0.02$ | 1.00 | 6.4 | 4 | 0.00016 | 0.00157 | 0.00252 | 15.64 | 0.0016 |
| AIM, $\rho = 0.1$ | 2.35 | 13.7 | 5 | 0.00007 | 0.00102 | 0.00141 | 2.05 | 0.0015 |

| Release | $\varepsilon$ | Seconds | Measured | One-way mean | Worst measured | Four-way | Share error (points) | Probability difference |
|---|---|---|---|---|---|---|---|---|
| AIM, $\rho = 0.5$ | 5.67 | 7.1 | 6 | 0.00005 | 0.00052 | 0.00077 | 1.65 | 0.0014 |
| MST, $\rho = 0.1$ | 2.35 | 3.3 | 3 | 0.00021 | 0.00114 | 0.05610 | 24.03 | 0.0280 |

The budget buys accuracy everywhere, but not evenly. Across the three budgets, the worst measured marginal and the four-way table improve steadily, though less than the scales shrink. The one-way marginals, already close at $\rho = 0.1$, improve least. The share error falls fastest at the low end, from 15.64 points to 2.05, because small groups are the first to drown in noise. A larger budget also lets AIM measure more marginals, four, five and six in this run. All four classifiers scored the same accuracy of 0.9313, so for this task the budget makes no visible difference.

The algorithm changes more than the budget does. MST measures three pairs, all with `ILOSTAT` in this run, so it treats `SEX`, `AGE` and `HWUSUAL` as independent once employment status is known. That is false, since hours depend on sex and age among those who work. So its four-way table is tens of times further from the truth than that of AIM at the same budget.

The share error of MST reaches 24 points, for women under 20 who report at most one usual hour. The predicted probabilities move by 0.028 on average, but the accuracy still does not change. MST is the better choice when the questions are about pairs and the release must be fast, 3.3 seconds against 13.7 for AIM in this run.

The cuts and the number of rounds are the other two settings. Finer cuts multiply the cells of every marginal, each with noise of the same scale, so the bound on each marginal grows. More rounds give each round a smaller share, so AIM measures more marginals, each less precisely.

## 18.6. How many rows, and how to draw them

The evaluation of the marginals read the model, not the rows, because drawing a fixed number of rows independently adds a sampling error to the error of the model. The script computes its expected size for each one-way marginal from the shares of the table, at 200,000 rows and at the size of the real table.

```
sampling noise on one-way SEX: expected tv at 200,000 rows=0.00090  at 3,811,932
 ↪  rows=0.00021
sampling noise on one-way AGE: expected tv at 200,000 rows=0.00152  at 3,811,932
 ↪  rows=0.00035
sampling noise on one-way HWUSUAL: expected tv at 200,000 rows=0.00100  at 3,811,932
 ↪  rows=0.00023
sampling noise on one-way ILOSTAT: expected tv at 200,000 rows=0.00140  at 3,811,932
 ↪  rows=0.00032
aim rho=0.1 one-way tv summed (one run): model=0.00029  round=0.00028  sample=0.00745
aim rho=0.1 four-way tv (one run): model=0.00141  round=0.00139  sample=0.00483
```

```
mst rho=0.1 four-way tv (one run): model=0.05610  round=0.18410  sample=0.05791
mst rho=0.1 pair ('AGE', 'HWUSUAL') tv (one run): model=0.01504  round=0.12370
 ↪  sample=0.01261
```

At 200,000 sampled rows the sampling error of `SEX` is many times the model error of Table 18.1, so the rows are much worse than the release. Even at the size of the real table the sampling error stays above the model error, because the privacy noise at this budget is small against 3.8 million rows.

The default `method="round"` avoids the sampling error, and for AIM its marginals matched those of the model in this run. But in `mbi` 1.1.0 it draws each column by the rank of a row within the group of its parent columns. So two columns with the same parents are sorted together, which ties them. The star of MST makes `AGE` and `HWUSUAL` such a pair, and rounding moved their marginal from 0.01504 to 0.12370, while sampling left it near the model. So sample when the joint distribution of the rows matters, and draw as many rows as the total of the model if the marginals must be close. Check the pairs before you trust rounded rows.

## 18.7. What to publish

A synthetic dataset needs a statement of its guarantee and enough description for a user to judge what it can answer. The practitioner edition's chapter on release and governance builds such a statement for a trained model, and the same rules apply. The numbers come from the variables of the script, and the statement names what was public.

```
print(
    f"This synthetic dataset of {ROWS:,} rows was generated by AIM (OpenDP
     ↪  {dp.__version__}, mbi "
    f"{importlib.metadata.version('mbi')}, rounds={ROUNDS}, queries up to 3 columns)
     ↪  from one contingency table "
    f"over {', '.join(COLUMNS)} released under (rho = {RHO:g}, delta =
     ↪  {DELTA:g})-zero-concentrated differential "
    f"privacy, which is (epsilon = {epsilon:.2f}, delta = {2 * DELTA:g})-differential
     ↪  privacy, for adding or "
    f"removing all of one person's survey responses, at most {CONTRIBUTIONS} of them,
     ↪  under the central model. "
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; nulls were
     ↪  recoded to "
    f"{synth.FILL}; the keys of ILOSTAT were released with a threshold of
     ↪  {aim_query.oneway_threshold}. "
    f"Drawing the rows by independent sampling is post-processing and spent nothing."
)
```

```
This synthetic dataset of 200,000 rows was generated by AIM (OpenDP 0.16.0, mbi 1.1.0,
 ↪  rounds=10, queries up to 3 columns) from one contingency table over SEX, AGE,
 ↪  HWUSUAL, ILOSTAT released under (rho = 0.1, delta = 1e-07)-zero-concentrated
 ↪  differential privacy, which is (epsilon = 2.35, delta = 2e-07)-differential
 ↪  privacy, for adding or removing all of one person's survey responses, at most 36 of
 ↪  them, under the central model. The keys of SEX and the cuts of AGE and HWUSUAL were
 ↪  fixed in advance; nulls were recoded to {'AGE': 0.0, 'HWUSUAL': 99.0}; the keys of
 ↪  ILOSTAT were released with a threshold of 1364. Drawing the rows by independent
 ↪  sampling is post-processing and spent nothing.
```

The statement covers the rows without a further term, because drawing rows from a released model is post-processing, which cannot weaken the guarantee (the full edition's chapter on properties). It covers this one table only, and the other releases of this chapter, from the same people, would add to it.

Table 18.3.: What to publish with a synthetic dataset, and where this chapter found each item.

| Item | What to publish | In this chapter |
|---|---|---|
| The guarantee | $\rho$ and $\delta$, their conversion to $\varepsilon$, the privacy unit and the trust model, printed from the variables | $\rho = 0.1$, $\varepsilon = 2.35$, 36 contributions, central model |
| The columns | Each column with its keys or cuts, the null recoding, and which keys were released | `SEX` keys, `AGE` and `HWUSUAL` cuts, `ILOSTAT` keys at threshold 1364 |
| The algorithm | Its name and parameters, the library versions, and how the rows were drawn | AIM with 10 rounds, OpenDP 0.16.0, `mbi` 1.1.0, 200,000 sampled rows |
| The error bars | The measured marginals and their `std`, which come from the release and cost nothing | Table 18.1, the `std` column |
| The utility | The marginal errors and the downstream check, measured on public or historical data | Never on the private data itself |

The last row is the one most often done wrong. A distance computed on the private data depends on that data, so publishing it is a new release that the statement does not cover. As Limitations and troubleshooting put it, the guarantee covers what passes through a measurement and nothing computed beside it. The error bars from `std` are safe to publish, and the distances in this chapter were safe only because the data are public.

This is the last chapter of the book. You can now release statistics, tables, models and synthetic rows through a context. You can also measure their quality on data you may look at, and publish them with a guarantee that says what it covers.

## 18.8. Summary

Table 18.4.: Evaluating and tuning synthetic data, what each step does, and the rule of thumb that goes with it.

| Call or concept | What it does | Rule of thumb |
|---|---|---|
| Three questions | Measured marginals, unmeasured queries, a downstream model | Only the first has an error bar from the release |
| `table.project_melted(clique)` | The fitted model's counts for a marginal, without drawing rows | Evaluate the model first, then the rows |
| Total variation distance against `std` | Workload error of a marginal against three standard deviations in every cell | A distance over the bound points to the pipeline, not the noise |
| `table.std` on an uncovered table | Raises `ValueError`: not covered by the query set | No measurement, no error bar; check unmeasured queries on public data |
| Train on synthetic, test on real | A model fitted to synthetic rows, scored on held-out real rows | Accuracy is coarse; compare predicted probabilities too |
| One context per setting | Alternatives that you compare but release only one of | On the same private data their budgets add up; tune on a stand-in |
| The budget | Divides every scale by the square root of its ratio | Small groups and unmeasured tables gain most; one-way marginals least |
| AIM against MST | Triples chosen adaptively, or a tree of pairs | MST is fast but loses what no pair shows |
| `synthesize(rows, method="sample")` | Independent rows, with sampling error of their own | Draw as many rows as the model's total when the marginals must be close |
| `method="round"` in `mbi` 1.1.0 | Matches the model's marginals but ties columns drawn from the same parents | Check the pairs before trusting rounded rows |
| What to publish | Guarantee, columns, algorithm, error bars, utility on public data | Never publish a distance computed on the private data |

## 18.9. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi.ContingencyTable` (`std`, `project_melted`, `synthesize`). The documentation gives no evaluation; this chapter's is its own.
- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the workload error and the selection that

minimizes it.

- Tao, McKenna, Hay, Machanavajjhala and Miklau, “Benchmarking Differentially Private Synthetic Data Generation Algorithms” (2021): marginal errors and downstream tasks as the measures of quality.
- The practitioner edition’s chapter on release and governance in the practitioner edition: the statement of a guarantee and the evidence that goes with it.
- The full edition’s chapters on post-processing and composition in the full edition, query release in the full edition, public data in the full edition and choosing epsilon in the full edition derive the results this chapter states.

# A. Glossary

Acronyms used in the book and the pages that use them.

**API** Application programming interface. Used in Getting started, A first release, The data and the context, Grouping, Assessing utility and choosing the noise, The framework, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Extending the library, Plugins, Statistical modelling, Limitations and troubleshooting, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

**CPU** Central processing unit. Used in Synthetic data with AIM and MST.

**CSV** Comma-separated values. Used in The data and the context, Limitations and troubleshooting.

**DP** Differential privacy. Used in Preface, Essential statistics, Assessing utility and choosing the noise, A framework to understand differential privacy, Measurements, Combinators, Statistical modelling, Synthetic data with AIM and MST.

**JSON** JavaScript Object Notation. Used in Utilities and the Context API inside.

**LFS** Labour Force Survey. Used in Preface, Getting started, The data and the context, Essential statistics, Grouping, Identifier truncation and bounds, Preparing microdata, Assessing utility and choosing the noise, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Plugins, Statistical modelling, Limitations and troubleshooting, Contingency tables, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

**PCA** Principal component analysis. Used in Statistical modelling.

**RDP** Rényi differential privacy. Used in Combinators.

**SQL** Structured Query Language. Used in Identifier truncation and bounds.

# B. Index

adaptive composition, 115, 116, 125
approximate differential privacy, 79, 126

basic composition, 116

clipping, 29, 67
composition, 10, 18, 29, 37, 79, 80, 91, 115, 116, 125, 126, 141, 194

differential privacy, 4, 10, 17, 20, 29, 37, 47, 67, 69, 79, 91, 103, 115, 116, 125, 126, 128, 141, 150, 160, 171, 182, 183, 193, 194
discrete Gaussian distribution, 67, 103, 116
discrete Laplace distribution, 102, 103

empirical risk minimization, 150
exponential mechanism, 29, 141

fully adaptive composition, 115

Gaussian mechanism, 10, 29, 67, 79, 80, 91, 104, 126, 141, 171
group privacy, 18

honest-but-curious, 114, 115, 141, 149, 159

Laplace mechanism, 10, 18, 29, 47, 80, 103

one-way marginal, 182, 189, 193

permute-and-flip, 91, 103
post-processing, 10, 18, 29, 37, 80, 91, 114, 116, 162, 170, 171, 182, 194
privacy filter, 115
privacy loss, 69, 116, 149
privacy odometer, 115
privacy profile, 79, 115, 116
pure differential privacy, 10, 17, 67, 79, 102, 115, 116, 125

randomized response, 103, 104
report-noisy-max, 103
Rényi differential privacy, 116

sensitivity, 10, 18, 29, 37, 47, 58, 65, 67, 79, 80, 91, 103, 104, 141, 149, 150, 160, 171
smooth sensitivity, 58
sparse vector technique, 103

total variation distance, 182, 186, 189, 193

variance, 47, 67, 91

zero-concentrated differential privacy, 20, 65, 67, 79, 102, 115, 116, 125, 126, 170, 171